

CHMTRN

Reference Manual

LHASA Group
Harvard University

November 2002

Version 19.1

CREDITS

The first edition of this Manual, commonly called the "CHMTRN Manual", was written by Harry W. Orf for the PDP-10 version of LHASA. The Manual was revised and updated for the VAX-11 version of LHASA in January, 1982, by Leo Joncas (Harvard), Alan Long (Harvard), Tony Baxter (ICI Organics), and Susan Bostock (ICI Organics). A subsequent major revision was made in May, 1984, by Alan Long, Jerry Lotto (Harvard), and Stewart Rubenstein (Harvard). Since then, revisions have been made as modifications to the CHMTRN language occurred. The manual was substantially overhauled and converted to its current form in August, 1997, by Craig Marby (Harvard).

PROGRAM DISTRIBUTORS

Access to the LHASA or DEREK programs may be arranged directly with Harvard University, from:

Dr. Alan K. Long
Department of Chemistry and Chemical Biology
Harvard University
12 Oxford Street
Cambridge, MA 02138
USA
e-mail:aklong@fas.harvard.edu

COPYRIGHT NOTICE

Copyright © 1981-2002 President and Fellows of Harvard College. All rights reserved.

Permission is granted, to all licensed LHASA or DEREK users, to make verbatim copies of this manual for personal use, provided the copyright notice and this permission notice are preserved on any such copies. Creating derivatives of this manual, or any portion thereof, is prohibited.

DISCLAIMER

The information contained in this manual is believed to be accurate and reliable. However no representation of warranty is made with respect to this document or the software described herein. That software is distributed only in accord with a written license from a program distributor. Distributors disclaim any implied warranties of merchantability or fitness of this document for any purpose. Information in this document is subject to change without notice and does not represent a commitment on the part of Harvard University or any licensed program distributor.

TRADEMARKS

All trademarks are the properties of their respective holders.

TABLE OF CONTENTS

ABOUT THIS MANUAL.....	5
Release Notes for CHMTRN 19.1	5
Release Notes for CHMTRN 17.1 (February 1999).....	5
Release Notes for CHMTRN 16.1 (December 1997).....	5
Section 1: INTRODUCTION	7
Section 2: HEADER OPERATORS.....	11
2.1 Title Operators	11
2.1.1 LHASA Title Operators	11
2.1.2 DEREK Title Operators	12
2.1.3 Title Operators for Screens.....	12
2.2 Rating Operators	13
2.3 Pattern Operators.....	15
2.4 LHASA Screening Set Operators	16
2.4.1 LHASA Functional Group Screening Set Operators.....	17
2.4.1 LHASA Screening Set Operators for Transforms	18
2.4.2 LHASA Screening Operators for Olefin Chemistry.....	23
2.5 DEREK Screening Set Operators	24
2.6 LHASA Parameters for Calculations	25
2.7 LHASA Ring Parameters	25
2.8 Miscellaneous Language Constructs.....	26
Section 3: TARGET SPECIFICATION	28
3.1 Functional Group Specifiers	29
3.1.1 Explicit Functional Groups	29
3.1.2 Generic Functional Groups	33
3.1.3 Generic Functional Groups Used in Mechanisms	34
3.1.4 Other Generic Functional Group Specifiers.....	34
3.2 Atom and Bond Specifiers.....	37
3.2.1 Explicit Atom Types	37
3.2.2 Generic Atom Types	39
3.2.3 Stereochemical Atom and Bond Specifiers.....	40
3.2.4 Ring Atom and Bond Specifiers.....	42
3.2.5 Comparison Atom and Bond Specifiers.....	47
3.2.6 Miscellaneous Atom and Bond Specifiers	50
3.3 Miscellaneous Specifiers	53
Section 4: LOCANT SPECIFICATION	54
4.1 Distance Specification	54
4.2 Atom and Bond Specification	55
4.2.1 Atom Locants	55
4.2.2 Bond Locants.....	59
4.3 Additional Locant Specification	62
Section 5: OPERATION CODES	68
5.1 Mechanism and Subgoal Operations.....	69
5.2 Operations Requiring Locant Specifiers.....	82
5.3 Operations Using an Address or Table Location	84
5.4 Operations Specifying Text	88

5.5	Miscellaneous Operations.....	90
5.6	Condition Statement Operators.....	99
Section 6:	CONDITIONAL AND LOGICAL QUALIFIERS	105
6.1	Conditional Execution of Qualifiers	105
6.2	Logical Combination of Qualifiers	105
Section 7:	CHMTRN ARITHMETIC.....	108
7.1	Numerical Values.....	108
7.2	Arithmetic Declarations.....	108
7.3	Set Expressions	109
7.4	Variable Expressions.....	111
7.5	Calculated Values	112
INDEX		114

ABOUT THIS MANUAL

This manual is a reference for version 19.1 of the CHMTRN language. This manual contains very particular descriptions of the uses of the individual words, or "specifiers", that make up CHMTRN. The reader is assumed to be a fairly experienced CHMTRN writer of LHASA transforms and/or DEREK rules. This manual documents the entire CHMTRN language and indicates (where appropriate) features that are specific to LHASA or DEREK.

For a less advanced discussion please refer to the manual entitled "CHMTRN Writers' Guide". The Writers' Guide documents the ways in which the language can be used to write transforms or rules, the correct formats for those transforms or rules, and the makeup of the data tables that comprise the knowledge bases used by the LHASA or DEREK programs.

As of version 19.1, the CHMTRN compiler is supported under the same operating systems as the current LHASA 19.1 and DEREK 19.1. It is available for various flavors of Unix. The behavior of the CHMTRN compiler is equivalent for all supported operating systems.

Release Notes for CHMTRN 19.1

The END*PRESCREEN keyword has been removed from the CHMTRN language. The functionality was not used consistently in transforms, and with recent increases in speed of transform evaluation, is no longer worthwhile.

Release Notes for CHMTRN 17.1 (February 1999)

The CHMTRN 17.1 compiler now generates all index files by recompiling the entire LHASA and/or DEREK knowledge base whenever a CHMTRN source file is compiled. This new behavior avoids a bug present in the CHMTRN 16.1 compiler.

Please note that CHMTRN 17.1 will be the last version of the CHMTRN compiler to support the RISK and NEW*RISK keywords as synonyms for HAZARD and NEW*HAZARD. Users are advised to update CHMTRN source files to use the hazard keywords.

Release Notes for CHMTRN 16.1 (December 1997)

Since this manual was last printed, there have been numerous changes to both the CHMTRN compiler and the CHMTRN manual itself. The changes are summarized in this section.

More seasoned users of CHMTRN will note that this manual has been substantially overhauled since it was last printed. It has been entirely reformatted and now has a similar look to the other LHASA and DEREK manuals. This manual now documents the entire CHMTRN language; there is no supplement detailing DEREK keywords and constructs. DEREK documentation is completely integrated into this manual.

As of version 16.1, the following CHMTRN keywords have been officially removed from the language: SAME*RATING, CALL*C, CALL*L, CALL*N, CALL*FORT, LOGP*ACTIVE, LOGP*VALUE, PERCUT*ABS*ACTIVE, PERCUT*ABS*VALUE, RESTORE*DEBUG, DEBUG*OFF, DEBUG*ON, RATING*SCHEME, END*RATING*SCHEME.

For this printing, the CHMTRN keywords documented herein have been carefully checked against the compiler and as a result, documentation for well over 100 additional previously undocumented or newly defined keywords has been added to the manual.

Keywords that were previously used, but undocumented in this manual, include: DEREK Title Operators (see section 2.1.2), Pattern Operators (see section 2.3), LHASA Screening Operators for Olefin Chemistry (see section 2.4.2), LHASA Parameters for Calculations (see section 2.6), LHASA Ring Parameters (see section 2.7), the program keywords LHASA, APSO, PROTECT, DEREK (see end of section 5.5), the literature citation keywords REFERENCES and END*REFERENCES for LHASA (see section 2.1.1), the literature citation keywords REMARKS and END*REMARKS for DEREK (see section 2.1.2), the DEREK keyword TOXOPHORE (see section 2.1.2), the LHASA screening set operators NON*ENANTIOSELECTIVE and NON*DIASTEREOSELECTIVE (see section 2.4.1), and last, but not least, the most recent incarnation of CHMTRN arithmetic syntax is now fully documented in section 7.

The keywords MUST*BE, MUST*NOT*BE and FORBIDDEN (see beginning of section 5) together with the DEREK screening set operators (e.g. CARCINOGENICITY, IRRITANCY, etc.) (see section 2.5) have been available for sometime though they have yet to be used. These keywords and screening set operators were added to the language as a result of DEREK customer feedback and their use is encouraged.

Title keywords SCREEN, NAME and TACTIC*SCREEN for screen definition files are new for version 16.1 and are documented in section 2.1.3. The keywords HAZARD and NEW*HAZARD for DEREK have been created as synonyms for RISK and NEW*RISK to more accurately describe what they refer to. The RISK and NEW*RISK keywords will be made obsolete in a future release of CHMTRN.

The most significant change to the CHMTRN compiler is that it has recently been ported to Unix. A major advantage of the port is that significant optimizations to the compiler were made. The CHMTRN compiler is now a *single* executable requiring no supporting command scripts. The compiler performs as much work as possible in memory and creates fewer output files. The "PUNMASK" procedure and output files have been made obsolete. The PATRAN compiler for 1-D patterns has been entirely rewritten and merged into the main compiler; the 2-D pattern compiler (known as PARSE) has also been merged into the main compiler. Furthermore, the output file "subobj.sbj" is now created directly by the compiler. The listing file (if requested by the user) now contains an annotated listing representing a merger of the three individual listing files that could previously be requested. As a result of all these improvements, version 16.1 of the CHMTRN compiler now runs approximately twice as fast under OpenVMS VAX compared to the previous versions of the compiler.

Section 1: INTRODUCTION

The CHMTRN compiler compiles, or converts to machine-readable form, one or more CHMTRN source files. The CHMTRN source files contain the individual transforms or rules of the LHASA or DEREK knowledge base, respectively. For a given input file (suffix “.src”), up to four output files may be generated. A binary output file (suffix “.bin”) is always created and contains the converted CHMTRN qualifiers and the directory sets (or indexing information). A reference text file (suffix “.ref”) is also always created and contains the literature text specified in the input file. If 1-D patterns (*vide infra*) are present in the input file, then the necessary data are written to an output file with suffix “.pat”. If 2-D patterns (*vide infra*) are present in the input file, then the necessary data are written to an output file with suffix “.prs”. After all CHMTRN source files have been compiled the master index files are also updated.

This manual is concerned primarily with the CHMTRN specifiers or “qualifiers” that comprise the binary output file “.bin”. Other aspects of CHMTRN (for example the 1-D and 2-D pattern languages) are described in greater detail in the companion manual entitled “CHMTRN Writers’ Guide”. The chemical questions or “CHMTRN qualifiers”, specified in CHMTRN source files and then compiled in to the binary output file, are fashioned after “chemical English”. By designing the language this way, the knowledge bases are more comprehensible to chemists or toxicologists with little or no programming experience. The language is very readable, and powerful, allowing quite complicated questions to be asked.

A “standard” qualifier such as

ADD 20 IF ALPHA TO ATOM*3 IS NOT BENZYLIC

is composed of three parts: (1) a "target" specifier (BENZYLIC), which designates the particular structural or chemical feature to be queried; (2) a "locant" specifier (ALPHA TO ATOM*3), which designates the site(s) in the current molecule to be examined; (3) an "operation" code (ADD 20), which dictates the action to be taken when the site(s) referred to by the locant specifier contains the feature designated in the target specifier.

A number of words with no real meaning to the compiler are accepted but immediately discarded. These so called “buzzwords” are: A, ALSO, AN, ANOTHER, ANY, ANYWHERE, ARE, AS, AT, ATOM(S), ATTACHED, AWAY, BE, BEARING, BETTER, BONDED, BY, CAN, CENTER, CENTRE, CHANGE, CHARACTER, CHARGE, CODE, CONTAIN(S), CONTAINING, COPY, DIMENSIONED, DOES, DUPLICATE, ELEMENT(S), EMPTY, EXACTLY, EXO, FACE, FOR, FROM, GROUP(S), HAS, HAVE, IDENTICAL, IF, IN, INCLUDING, INTO, ION, IS, LENGTH, LIST, MECHANISM, MEMBER(S), MOLECULE, MUST, NEXT, OF, OFFSPRING, OLD, OLEFIN*TYPE, ON, ONLY, ORDER, PARENT, PATH(S), PRECURSOR, PRESENT, PREVIOUS, REQUIRED, SHORTEST, SIMULTANEOUSLY, STRUCTURAL, TARGET, TEST, THAT, THE, THERE, TO, UNITS, UP, VALUE(S), WHICH, WITH, WORD(S), and WRITE. The purpose of “buzzwords” is to improve readability, and their use is strongly encouraged in writing good CHMTRN. Note that the example qualifier above contains 3 “buzzwords”.

Another purpose of “buzzwords” is to allow qualifiers to be represented compactly. The rest of this section describes the detailed storage of CHMTRN qualifiers in the binary output file. It is probably only of interest to those developing or maintaining the LHASA or DEREK programs or for the particularly curious who wish to delve into the more intricate recesses of CHMTRN.

The CHMTRN language has been designed to allow most qualifier sentences to be stored in one or two (and occasionally three) 32-bit computer words. Each CHMTRN word is defined as a unique value or combination of values in one of 22 different word categories or "fields". Each field occupies one or more bits in one of the three types of "CHMTRN computer words", which are shown in Figure 1.1. Associated with each field is a range of numerical values, each of which defines a word or other element in the CHMTRN language. In addition to the fields indicated in Figure 1.1, there are three fields which overlap other fields and that are used with particular CHMTRN constructs. These fields are QLNUMB (bits 1-10 in a type "A" word), QLADDR (bits 1-16 in a type "A") word, and QLCONST (bits 1-32 in a type "B") word.

Type "A" Word			Type "B" Word			Type "C" Word		
1			1	QLFORE0	0-1	1		
2	QLDIST	0-7	2	QLBOND	0-1	2	QLRING	0-
3			3	QLNOTL	0-1	3	_JSIZE	31
4			4	QLNOTT	0-1	4		
5			5			5		
6			6	QLLOGC	0-7	6		
7	QLFROM	0-127	7			7	QLRING	0-
8			8	QLBRKT	0-1	8	_JSIZE	31
9			9			9		
10			10			10		
11			11			11		
12			12			12		
13	QLINCR	0-63	13			13		
14			14			14		
15			15			15		
16			16	QLQUAL		16		
17	QLNOT	0-1	17			17		
18			18			18	Unused	
19	QLOPER	0-7	19			19		
20			20	24		20		
21			21	Separate		21		
22			22	Bits		22		
23			23			23		
24			24			24		
25	QLTARG	0-511	25			25		
26			26			26		
27			27			27		
28			28			28	QLRING	
29			29			29	_QUAL	
30	QLCONT	0-1	30			30		
31	QLTHEN	0-1	31			31	6 bits	
32	QLNEXT	0-1	32			32		

Figure 1.1 Structure of the Packed Computer Words Used for the Compact Numerical Representation of CHMTRN Qualifiers.

Qualifier sentences are packed into computer words on a line-by-line basis, i.e. each line of CHMTRN is stored in a different computer word. Consider

SUBTRACT 25 FOR EACH ALKYL GROUP ON BETA &
TO ATOM*2 OFFPATH ONRING.

The above example, though spanning multiple lines of text, would still be considered a single line of CHMTRN and hence would be stored as a single word.

Certain types of qualifiers require additional words. Certain fields, such as QLADDR, QLNEXT, and QLNUMB, are only used with particular keywords. The most common type of multiple-word qualifier is that which specifies multiple locants, each of which must be stored in separate words. Each separator keyword packs a 1 into the QLCONT (qualifier continuation) field and causes subsequent CHMTRN specifiers to be packed into the next computer word. In the qualifier

KILL IF THE APPENDAGES FROM ATOM*1 TOWARDS &
HETERO1*1 AND FROM ATOM*4 TOWARDS HETERO2*1 &
ARE NOT IDENTICAL

for example, the four locant specifiers (ATOM*1, HETERO1*1, ATOM*4, and HETERO 2*1) are stored in successive words.

Numerous other specifiers (e.g. SAME, CONTAINED, PRIMARY, SECONDARY, TERTIARY, and BRIDGE(D), have also been defined as separators. Most of these keywords require two locant specifiers and always appear between those two specifiers in the qualifier sentence. In the qualifier

ADD 30 IF ATOM*5 IS THE SAME AS ALPHA TO ATOM*2

for example, the word SAME, which is stored in the first word as QLTARG 316 and QLCONT 1, causes anything following it on the same line to be coded into the next word. Depending on usage, certain stereochemical specifiers (e.g. CIS, TRANS, SYN, ANTI, ABOVE, BELOW, DEFINED*CIS, DEFINED*TRANS, DEFINED*SYN, DEFINED*ANTI, DEFINED*ABOVE, DEFINED*BELOW) can be defined as either single or double (post- or both pre- and post-) word separators.

Subsequent sections of this manual detail the various keywords and constructs of the CHMTRN language. Where appropriate the values of the fields in the binary output file are also given alongside the keyword heading. For example KETONE [QLTARG 1]. This information can be particularly useful when debugging or when making changes to the CHMTRN language.

Section 2: HEADER OPERATORS

The specifiers described in this section are used at the beginning of each transform or rule to provide labeling and indexing information. To see how the header operators and the chemical qualifiers go together to produce a complete transform or rule, the reader is referred to the companion manual, the CHMTRN Writers' Guide.

2.1 Title Operators

2.1.1 LHASA Title Operators

TRANSFORM

For LHASA knowledge base entries, the TRANSFORM specifier must appear in the first line of every group- or pattern-oriented transform. TRANSFORM is followed on the same line by an identifying number that is unique to that transform, such as

TRANSFORM 1342

The transform number appears along with the transform NAME (*vide infra*) whenever a precursor resulting from the successful application of that transform is displayed in LHASA. The reader is referred to the CHMTRN Writers' Guide for documentation regarding the allocation of LHASA transform numbers.

NAME

For LHASA knowledge base entries, the NAME specifier follows the TRANSFORM line. NAME is followed on the same line by the textual name of the transform (limited to 58 characters), as in

NAME Aldol Condensation

Unlike a transform number (*vide supra*), which must be unique for each LHASA transform entry, the name of a transform may be repeated whenever necessary. The transform name appears along with the transform number whenever a precursor resulting from the successful application of that transform is displayed.

REFERENCE(S)

END*REFERENCE(S)

These two keywords must immediately follow the NAME of a transform and are used to delimit the literature and other reference information for a particular transform. The lines between REFERENCES and END*REFERENCES are copied verbatim to the LHASA reference display. The journal abbreviations that are used in the LHASA references are listed in an appendix to the LHASA User Guide. There is no restriction on the number of lines of a reference. The special sequence of characters @@@@ appearing alone on a line indicates a page break on the LHASA reference display.

2.1.2 DEREK Title Operators

RULE

In DEREK rulebase entries, RULE must appear as the first line. A unique number for the rule (which is not displayed on the DEREK Toxophore Display) follows:

RULE 23

The reader is referred to the CHMTRN Writers' Guide for documentation regarding the allocation of DEREK rule numbers.

HAZARD

(synonym **RISK**)

In DEREK rulebase entries, the HAZARD specifier follows the RULE line. HAZARDS can occupy two lines, and the line break is indicated by //. HAZARDS may be continued across lines using the standard CHMTRN continuation character "&."

HAZARD Mutagenesis // Sensitization

TOXOPHORE

In DEREK rulebase entries, TOXOPHORE follow the HAZARD specifier. As with HAZARDS, TOXOPHORES may occupy two lines with the line break indicated by //. An example of a TOXOPHORE statement follows:

TOXOPHORE Nitro group on heterocyclic or alicyclic ring

REMARK(S)

END*REMARK(S)

These two keywords must immediately follow the TOXOPHORE of a rule and are used to delimit the text of the literature citation, remarks or references for a particular rule. The lines between REMARKS and END*REMARKS are copied verbatim to the DEREK reference display. The special sequence of characters @@@@ appearing alone on a line indicates a page break on the reference display.

2.1.3 Title Operators for Screens

Screens are used by both the CHMTRN compiler and LHASA during execution. The screening files with suffix ".lds" are used for creation of the directory sets in the CHMTRN binary output files. The other screening files (having suffix ".ldp") are used for screening by LHASA. Screening files use the keywords documented in the section and contain one or more 1-D patterns defining the actual screen.

SCREEN

This keyword marks the beginning of a screen definition. It is followed by a number unique to the file. Screens are numbered starting at 1 (occasionally 0) in each file.

NAME

This keyword follows the SCREEN line and is used to associate a name to a screen.

TACTIC*SCREEN

This optional keyword, when present, indicates that a given screen is a tactic screen.

2.2 Rating Operators**RATING**

The traditional RATING statement simply specifies the rating, for example:

RATING 25

However, the only instances where this keyword should still be used are LHASA tactical combinations (which always have a RATING 30 statement) and DEREK rules (which always have a RATING 50 statement). In these cases, the RATING statement exists solely for historical reasons and is not a rating in the conventional sense.

Initial ratings are now specified by using a newer, more precise method. This replacement system uses eight rating categories together with scale keywords. The individual rating categories together with their weightings are listed below:

TYPICAL*YIELD	10
RELIABILITY	8
REPUTATION	5
HOMOSELECTIVITY	4
HETEROSELECTIVITY	4
ORIENTATIONAL*SELECTIVITY	4
CONDITION*FLEXIBILITY	3
THERMODYNAMICS	2

The scale keywords together with their numerical equivalents are as follows:

BAD	0
POOR	1
FAIR	2
GOOD	4
VERYGOOD (synonym NOT*APPLICABLE)	6
EXCELLENT	8

The reaction corresponding to the transform is rated in each category on a scale from BAD to EXCELLENT. Guidelines for the selection of scale keywords are included in the CHMTRN Writers' Guide. A typical rating block would be

TYPICAL*YIELD	GOOD
RELIABILITY	GOOD
REPUTATION	GOOD
HOMOSELECTIVITY	BAD
HETEROSELECTIVITY	GOOD
ORIENTATIONAL*SELECTIVITY	EXCELLENT
CONDITION*FLEXIBILITY	FAIR
THERMODYNAMICS	EXCELLENT

The initial rating is computed by summing the numerical value of the scale keyword multiplied by its weight for each category, normalizing to 100 and rounding to the nearest multiple of 5. The CHMTRN log file (if one is requested) lists the calculated value of the rating block.

In either the old or the new format, the CHMTRN compiler encodes the rating in the first word that appears for each transform or rule in the binary output file, i.e. the first qualifier read during transform or rule evaluation. The QLNUMB field of the rating qualifier encodes the number of words in the CHMTRN binary output file which encode the mechanism – i.e. the “mechanism length”.

Definitions of the rating category keywords are included below. In a rating block, the category keywords may only appear in the order specified below.

TYPICAL*YIELD

This keyword is the 1st rating block category. TYPICAL*YIELD is the only category for which it is legal to use VERY*GOOD.

RELIABILITY

This keyword is the 2nd rating block category. RELIABILITY is a measure of the predictability and (if applicable) stereoselectivity of the reaction.

REPUTATION

This keyword is the 3rd rating block category. REPUTATION is a measure of the familiarity, or track-record, of the reaction.

HOMOSELECTIVITY

This keyword is the 4th rating block category. HOMOSELECTIVITY is a measure of the selectivity of the reagents for the reaction towards functional groups of the same type as the reacting group(s).

HETEROSELECTIVITY

This keyword is the 5th rating block category. HETEROSELECTIVITY is a measure of the selectivity of the reagents for the reaction towards functional groups of different types than the reacting group(s).

ORIENTATIONAL*SELECTIVITY

This keyword is the 6th rating block category. ORIENTATIONAL*SELECTIVITY is a measure of the selectivity of the reagents for the reaction towards one site versus an alternative one, e.g. the selectivity of a borane reagent for one end or the other of a reacting CC. In the event that the reaction does not involve a choice of orientation, the keyword NOT*APPLICABLE (a synonym for VERY*GOOD) may be used.

CONDITION*FLEXIBILITY

This keyword is the 7th rating block category. CONDITION*FLEXIBILITY is a measure of the number of different types of reagents that can be used to carry out the reaction.

THERMODYNAMICS

This keyword is the 8th rating block category. THERMODYNAMICS is a measure of the tendency of the reaction to go to products without intervention from the chemist.

2.3 Pattern Operators

The CHMTRN compiler recognizes two alternative pattern syntaxes to represent the retron and precursor of a LHASA transform. For DEREK rules and screening files, the 1-D pattern is the only syntax permitted. For a detailed description of the two pattern syntaxes the reader is referred to the CHMTRN Writers' Guide.

1D*PATTERN

This keyword indicates that the following lines contain a retron or toxophore pattern written in the CHMTRN one-dimensional pattern syntax, also known as PATRAN (PATtern TRANslator). All DEREK rules must contain 1-D patterns. For LHASA, 1-D patterns are recommended but not required. In LHASA any transform containing 1-D patterns is a candidate for processing by the pattern chemistry executive.

2D*PATTERN

This keyword indicates that the following lines contain a retron and precursor pattern written in the CHMTRN two-dimensional pattern syntax. 2-D patterns were developed after 1-D patterns and are intended to be easier to read by humans (as opposed to the CHMTRN compiler). Either or both 1-D and 2-D patterns may be specified for a LHASA transform. 2-D patterns are required for tactical combination transforms, and for any transform that is to be a candidate for a goal transform by the tactical combinations executive.

END*PATTERN

This keyword indicates the end of a preceding one- or two-dimensional pattern. Starting another pattern also ends a particular pattern, for example,

```

1D*PATTERN
  C[FGS=NITRO]% C% C% C% C% C% @1
  {1, 6, 5, 4, 3, 2}
2D*PATTERN
  NO2      H
  |        |
  Ph  =>   Ph
END*PATTERN

```

Multiple patterns may be specified in a transform or rule. The rating of each pattern is taken as the most recently preceding rating block.

1D*DEFAULTS
END*DEFAULTS

For 1D patterns, a statement block enclosed by the above two keywords is used to specify nonstandard defaults for all 1D patterns in a CHMTRN table, for example,

```

1D*DEFAULTS
  ARYL=EITHER
  CHARGE=EITHER
  FUSION=EITHER
END*DEFAULTS

```

Such statement blocks, if present, must occur as the first statements of a CHMTRN source file.

2.4 LHASA Screening Set Operators

Screening sets are created in the directory structures of all CHMTRN tables. The directory structures provide indexing and quick access to group- and pattern-oriented transforms in the LHASA knowledge base. These sets are used either to provide additional important information about certain transforms or to provide access to transforms of a similar type.

2.4.1 LHASA Functional Group Screening Set Operators

In order to provide quick access to one- and two-group transforms, the keywords in this section are used to specify the functional group prerequisites in the retron for a LHASA group-oriented transform. These keywords are specified immediately following the transform rating.

GROUP*1 (synonym SUBJECT)
GROUP*2 (synonym OBJECT)

These keywords indicate what functional group types key entry into the LHASA transform, as in

GROUP*1 CAN BE KETONE OR ALDEHYDE OR ESTER &
 OR AMIDE*1 OR AMIDE*2 OR AMIDE*3.

The synonyms SUBJECT and OBJECT are used in the FGI tables to indicate, respectively, the group types keying entry into a transform and the group type produced by the successful application of that transform.

Any functional group type listed after the GROUP*1(2) statement must be an explicit functional group. Generic names such as AMIDE and CARBOXYL are not allowed.

The word OR, separates the individual group types. When a GROUP*1(2) statement exceeds one line in length, as shown in the example above, the word OR must never appear at the end of a line.

The functional group types contained in the GROUP*1 and GROUP*2 statements are stored only in the directory sets used for accessing the individual transforms. The GROUP*1 and GROUP*2 lines do not appear in the compiled transform.

SUBGOAL*1
SUBGOAL*2

These keywords follow the GROUP*x statements in one- and two-group transforms that are allowed to request FGI subgoals. They list those keying group types that may be requested as subgoals. Consider, for example, a transform where

GROUP*1 CAN BE ALCOHOL OR ESTERX OR ETHER.

Since any functional group that could be converted (retrosynthetically) to an alcohol would probably also be converted to an esterox or ether via the alcohol, it is not necessary to let all the keying groups be requested as subgoals. In this case, the statement

SUBGOAL*1 MUST BE ALCOHOL

would suffice.

2.4.1 LHASA Screening Set Operators for Transforms

To indicate that a transform belongs in a special set, the keyword representing that set must appear in the transform just after the GROUP*1(2) or SUBGOAL*1(2) statements documented in the previous section. All special sets presently in use are described below along with their representative keywords.

BROKEN*BOND(S)

This keyword is followed by bond locant specifiers that identify all the bonds broken by this transform, for example

BROKEN*BONDS BOND*1 BOND2*1

These keywords identifies the first path bond and the first bond in the second keying group as bonds broken by the transform. When more than one bond is specified the additional locant specifiers are separated by a space. The bond locant specifiers that can be used with the BROKEN*BONDS statement include path bond (BOND*1-BOND*10), the group bond (BOND1*1-BOND2*3) specifiers, and the offpath bonds (OFFPATH*ON*ATOM1-OFFPATH*ON*ATOM12). However, these locants are not always sufficient to identify all broken bonds. In these cases, the OTHER*BONDS specifier is added to the BROKEN*BONDS statement.

The BROKEN*BONDS statements are used to create directory sets that are important for the prescreening of transforms. Therefore, it is desirable that each group- and pattern-chemistry transform contain a BROKEN*BONDS statement that identifies all the bonds broken in the mechanism.

DISCONNECTIVE

Transforms containing this keyword involve a retrosynthetic disconnection and correspond synthetically to reactions that join two or more fragments or to intramolecular ring-forming reactions. This set is particularly important in LHASA Two-Group Strategic Bond Mode, where only disconnective transforms are considered, but can be used in pattern transforms, functional group interchange transforms, and tactical combinations of transforms as well.

EXTENDED*SUBGOAL

Transforms containing the EXTENDED*SUBGOAL keyword have been given the ability to call extended subgoals (sequential FGI, double parallel FGI, FGA, and sequential FGI-FGA) for the purposes of disconnecting strategic bonds. This special set label is only used in two-group transforms, one-group transforms, and tactical combinations of transforms.

FGA*NOT*ALLOWED

This keyword labels transforms in the pattern chemistry knowledge base that are not allowed to request functional group addition (FGA) subgoals to rectify mismatches between the target structure and their keying patterns.

INTERMOLECULAR

Reactions corresponding to INTERMOLECULAR transforms are those that are not of synthetic value when attempted intramolecularly (e.g. organometallic additions to carbonyls). The INTERMOLECULAR set prevents transforms of this sort from being considered in Strategic Bond Mode when the bond to be disconnected is a ring bond. INTERMOLECULAR is only used in one-group and two-group transforms.

LAST*RESORT

The LAST*RESORT keyword is used to label so-called "last-resort" functional group addition transforms in the FGATBx tables. These transforms are accessed only in LAST*RESORT SUBGOALS mode, selectable on the LHASA Debug menu. LAST*RESORT SUBGOALS mode is exclusive in that it does not allow any other types of subgoals to be performed. The transforms bearing the LAST*RESORT label typically disconnect appendages blocking the addition of the desired functional group without regard for the existence of chemical precedents for such disconnections.

LEGAL*REQUEST

LEGAL*REQUEST takes atom locant specifiers as arguments. LEGAL*REQUEST restricts a transform to attaching a requested group only on the atom locant specified.

LOW*UTILITY**MEDIUM*UTILITY**

These keywords reflect the general utility of an FGI transform and are used in ranking and comparing similar sequential FGI routes. Three classes of transform utility are presently recognized: high, medium, and low. Transforms are considered as high utility unless otherwise specified by the LOW*UTILITY or MEDIUM*UTILITY keyword. Low utility transforms are often characterized by severe reaction conditions, fair-to-poor yields, or lack of stereospecificity. Examples in this category are the Hunsdieker decarboxylation and the acid-catalyzed dehydration of an alcohol. At the opposite extreme, transforms with the high utility designation are more synthetically reliable, e.g. hydrolyses of esters, reduction of nitriles, Wittig reactions, etc. LOW*UTILITY can also be used to label pattern transforms of questionable usefulness.

MECHANISTIC

The MECHANISTIC special set label identifies a group of pattern chemistry transforms that do not correspond to published synthetic methods. These transforms simply perform mechanisms or partial mechanisms that are often constituent steps in heterocycle syntheses in hopes of revealing new potential synthetic methods.

NON*DIASTEREOSELECTIVE**NON*ENANTIOSELECTIVE**

These keywords are only relevant to transforms that are also designated STEREOSELECTIVE. They are therefore only utilized in the LHASA STEREOSELECTIVE tactic. Note that the above keywords are mutually exclusive and should not both appear in a transform.

NON*OPPORTUNISTIC

This keyword appears in one-group, two-group, and pattern transforms and in tactical combinations of transforms that are not to be used in simple opportunistic, group- and pattern-oriented search. Transforms in this category are usually non-simplifying and are only used in bond-oriented searches or by higher level executives. They can be tested manually as well.

ORIGIN*INDEPENDENT

This keyword appears in one-group transforms that are keyed by multi-origin functional groups and that give identical precursors regardless of which origin is used to key the transform. The ORIGIN*INDEPENDENT keyword prevents such transforms from being performed more than once.

PATH*INDEPENDENT

A one-group transform (of path length other than zero) normally is attempted once for each path that can be grown from the keying group origin. This ensures, for example, that all three possible disconnections of a tertiary alcohol back to a ketone via the Grignard transform are examined. Similarly, a tactical combination of transforms is attempted once for each mapping of the tactical combination keying pattern to the target. Some transforms, however, would give the same precursor regardless of the mapping or path grown and, to prevent the unnecessary duplication of such transforms, they are labeled as PATH*INDEPENDENT.

PATH*ONE*SET

The PATH*ONE*SET label is used in the one-group transform table SGTAB1 to distinguish those transforms that are actually Path 1 single-group transforms, that is those transforms that break a bond adjacent to, but not actually in, the keying group, from Path 0 transforms, which break a bond in the keying group. Transforms of these two different single-group path lengths appear in a single table because of an anticipated eventual merger of the single group and functional group interchange tables.

REARRANGEMENT

The great majority of one-group transforms are disconnective and break a bond either in the keying functional group (path 0 transforms), or one, two, or three bonds away (path 1, 2, and 3 transforms). Certain one-group transforms, however, such as the Favorskii rearrangement, the carbonium ion to alcohol transform, and the diazomethane ring expansion transform do not break more C-C bonds than they make or are not particularly simplifying. Such transforms are labeled as REARRANGEMENTs to allow them to be distinguished when processing in One-Group Strategic Bond Mode. The REARRANGEMENT special set is also used for tactical combinations of transforms which do not break more bonds than they make.

RECONNECTIVE

Transforms containing this keyword involve retrosynthetic reconnections and correspond synthetically to fragmentation reactions. The ozonolysis and vicinal glycol cleavage transforms serve as examples. Such transforms must be labeled for quick access since they are of special importance to strategies involving the reconnection of medium and large rings and appendages. RECONNECTIVE can be used in one-group, two-group, or pattern transforms. Note that RECONNECTIVE is also used to label certain functional group interchange transforms, specifically the atom-oriented to bond-oriented path change 0 transforms which add the second functional group origin in the transform mechanism and the atom-oriented to atom-oriented path change 1 transforms which add the entire object group, including the origin, in the mechanism. Path change 0 atom-oriented to atom-oriented functional group interchanges should not be labeled RECONNECTIVE.

RING*4
RING*5
RING*6
RING*7
RING*4*5
RING*4*6
RING*4*7
RING*5*7
RING*6*7

These special set specifiers are used to cross-reference transforms containing 2-D patterns according to required ring sizes. More particularly, each keyword RING*X*Y is followed by atom or bond locant specifiers which correspond to the sequence numbers of 2-D pattern path atoms and bonds which must be in a common ring of size X through Y. For example,

RING*6 ATOM*2 BOND*4
 RING*5*6 ATOM*5 BOND*8

identifies the second path atom and fourth path bond as being in a common ring of size 6 and the fifth path atom and eighth path bond as being in a common ring of size 5 or 6. Note that only path atoms and bonds can be used in the set (i.e., not OTHER*BONDS or OFFPATH*ON*ATOM*X) and that each set can be used only once per transform, although several different sets can be used in the same transform.

RING*REQUIRED

Transforms containing this keyword are those that operate specifically to disconnect ringbonds. RING*REQUIRED is only used in the one-group transforms, two-group transforms, and tactical combinations of transforms. When processing in LHASA one-group Strategic Bond Mode or Appendage Mode, RING*REQUIRED transforms are not accessed if the bond to be broken is acyclic. In two-group chemistry, RING*REQUIRED transforms are excluded if there are no cyclic path bonds.

STEREOSELECTIVE (synonym **STEREOSPECIFIC**)

Transforms labeled STEREOSELECTIVE or STEREOSPECIFIC are the only ones accessed in the LHASA STEREOSELECTIVE tactic. These labels can be used in one-group, two-group, pattern, and functional group interchange transforms and in tactical combinations of transforms.

RACEMIZES*STEREO
CREATES*STEREO
REMOVES*STEREO
INVERTS*STEREO
RETAINS*STEREO
REQUIRES*STEREO

The function of the RACEMIZES*STEREO (etc.) statements is quite analogous to the function of the BROKEN*BONDS statements and the format is identical except that atom locant specifiers (specifically path atoms or CARBONx*y) must be used. The REMOVES*STEREO and CREATES*STEREO sets refer to the retrosynthetic removal or creation of sp³ stereocenters to or

from their non-sp³ counterparts. RACEMIZES*STEREO is used to identify cases where an sp³ stereocenter exists in both the retron and the precursor, but is modified non-selectively in the synthetic direction (i.e. a synthetic racemization). REQUIRES*STEREO indicates the path atoms that must be stereocenters for the target to match a particular transform. The *STEREO statements are used to create directory sets that are important for the stereochemical prescreening of transforms.

STUDENT

The STUDENT keyword is used to label transforms considered to be appropriate for the student knowledge base for the undergraduate teaching version of LHASA known as APSO. STUDENT is a legal keyword in one-group, two-group, pattern, and functional group interchange transforms.

SUBGOAL(S) *ALLOWED

The SUBGOALS*ALLOWED special set label is used in those pattern chemistry transforms that are allowed to request subgoals (functional group interchange, sequential functional group interchange, functional group addition, and functional group removal) to rectify mismatches between the target structure and their keying patterns.

SYMMETRICAL

This keyword applies only to two-group transforms and is used to indicate those transforms that would generate an identical precursor if reentered with GROUP*1 and GROUP*2 interchanged. For example, the word SYMMETRICAL appears in the ozonolysis transform keyed by two carbonyls since the precursor generated would be the same regardless of which carbonyl was labeled as GROUP*1.

UNMASKING

This keyword refers to one-group, two-group, and pattern transforms that generate “core” functionality from disguised and/or uncommon functional groups or group pairs. The hydrazone to ketone and episulfide to olefin transforms are typical examples of unmasking transforms.

COPYRIGHT*LUK

The COPYRIGHT*LUK label indicates that a transform is part of the LHASA extended knowledge base developed at Leeds by LHASA UK. Unless users have licensed this separate product, the LHASA program will not consider transforms labeled COPYRIGHT*LUK. Instead, only transforms in the “core” knowledge base will be available.

2.4.2 LHASA Screening Operators for Olefin Chemistry

These are a number of screening sets and operators used only in the olefin chemistry table “olftb.src”. These keywords are discussed below.

OLEFIN*TABLE

This keyword may appear at the top of a source file and causes the CHMTRN compiler to accept olefin chemistry specifiers.

The screening sets for olefin transforms indicate the required environment of the olefin for the particular transform to be a candidate for evaluation.

RINGBOND**FUNCTIONAL*GROUP****NON*OLEFINIC*FG****TERT*OR*QUAT*CENTER****QUAT*CENTER****AROMATIC*ATOM****NO*FG*WITHIN*ALPHA**

The above specifiers take one or more olefin locants as arguments. There are twelve olefin locant specifiers:

LONE1	CIS1	TRANS1	ACROSS1
LONE2	CIS2	TRANS2	ACROSS2
LONE3	CIS3	TRANS3	ACROSS3

The following olefin special set specifiers do not require locant arguments:

TETRASUBSTITUTED	NO*L2	WELL*DEFINED*L*LAST
TRISUBSTITUED	NO*C2	WELL*DEFINED*C*LAST
E*DISUBSTITUTED	NO*T2	WELL*DEFINED*T*LAST
Z*DISUBSTITUTED	NO*C3	UNDEFINED*L*FIRST
CUPRATE*FOR*RC	T1*NOT*QUARternary	UNDEFINED*C*FIRST
CUPRATE*FOR*RT	SPACING*1*3	UNDEFINED*T*FIRST
BORANE*FOR*RL	SPACING*1*5	UNDEFINED*L*LAST
ORGANOMETALLIC*AT*L1	SPACING*1*4	UNDEFINED*C*LAST
ORGANOMETALLIC*AT*C1	WELL*DEFINED*L*FIRST	UNDEFINED*T*LAST
ORGANOMETALLIC*AT*T1	WELL*DEFINED*C*FIRST	
RC*LARGER*THAN*METHYL	WELL*DEFINED*T*FIRST	

2.5 DEREK Screening Set Operators

The screening sets used in the DEREK rulebase are used to classify the rules.

COPYRIGHT*LUK

Rules developed at Leeds by LHASA UK should include this keyword.

STANDARD**FDA**

These labels indicate that a rule is either part of the STANDARD or FDA rulebase. If neither of

these labels is specified for a rule, then the rule is considered part of the DEVELOPMENTAL rulebase.

COLLABORATIVE
PROVISIONAL
SENSITIZATION (synonym SENSITISATION)
MUTAGENICITY
CARCINOGENICITY
IRRITANCY
REPRODUCTIVE*TOXICITY
NEUROTOXICITY
CORROSIVITY
METHAEMOGLOBINAEMIA
HEPATOTOXICITY
CYTOTOXICITY
OTHER*TOXICITY

These keywords are currently accepted by the CHMTRN compiler. They presently are not used by the DEREK program, but it is anticipated that they will be used in the near future.

2.6 LHASA Parameters for Calculations

DELTA
RHO

These keywords, which must appear before any transform qualifiers, require a positive or negative integer as argument, and the values specified are used in any subsequent Hammett calculations performed in the transform. Presently, DELTA and RHO are only used in the table "pchem5.src".

2.7 LHASA Ring Parameters

A number of the LHASA long-range chemistry tables require particular ring systems in order for them to be applicable. For such tables the equivalent of screening sets are specified in a ring parameter block which is enclosed between the following two keywords:

RING*PARAMETERS
RING*PARAMETERS*END

Within a ring parameter block, specific requirements for the ring system(s) may be defined.

FOR*EACH*ATOM
FOR*EACH*PATH
FOR*EACH*RING

These keywords introduce one or more clauses specifying characteristics which must be true for every atom, path or ring. FOR*EACH* clauses are optional, but if specified they may only appear at the start of a ring parameter block.

NUMBER*OF*REORIENTS
NUMBER*OF*RINGS
NUMBER*OF*RENUMBERS

These keywords, which expect an integer argument, are used to specify the characteristics of the entity specified by a preceding FOR*EACH. The integer argument represents the number of alternative orientations, ring systems, or renumberings of a ring listed, that a given entity may possess. More than one NUMBER*OF phrase may follow a FOR*EACH keyword. For example,

```
FOR*EACH*ATOM      NUMBER*OF*RINGS 1 &
                   NUMBER*OF*REORIENTS 0
```

RING*DEFINITION
MAXIMUM*SIZE
MINIMUM*SIZE
ATOM*NUMBERING
BOND*NUMBERING

These keywords together with numerical arguments make up an actual ring definition, which must be present and follows any FOR*EACH specifiers. Multiple ring definitions may be given. A typical ring definition would be

```
RING*DEFINITION MAXIMUM*SIZE 6 MINIMUM*SIZE 6
                  ATOM*NUMBERING 1 2 3 4 5 6
                  BOND*NUMBERING 1 2 3 4 5 6
```

REORIENT*DEFINITION

This keyword introduces a reorient definition which follows any ring definition clauses and has analogous syntax.

2.8 Miscellaneous Language Constructs

The constructs described in this section are not strictly-speaking header operators.

&

When a qualifier sentence is too long to fit on a single typed line, the special character "&" is used to indicate that the line following it is a continuation of the current qualifier. For example,

```
SUBTRACT 25 FOR EACH ALKYL GROUP ON BETA &
      TO ATOM*2 OFFPATH ONRING.
```

The above example, though spanning multiple lines of text, would still be considered a single line of CHMTRN.

...

Three consecutive dots indicate to the CHMTRN compiler that the rest of a line is a comment. Comments need not necessarily start in the first column of a line. Note that 1-D and 2-D patterns should not contain comments. Also note that reference text is copied verbatim, so such “comments” will actually also be displayed on the LHASA or DEREK reference display. Though well-written CHMTRN should largely be self-documenting, the use of comments for clarification is encouraged.

....

Four consecutive dots are used to enclose the mechanism or a transform or rule. Pre-mechanism qualifiers appear in the transform entry before the first occurrence of and post-mechanism qualifiers after the second occurrence, as exemplified by the following table code (taken from the two-group transform corresponding to Markovnikov addition of HX to an acetylene).

```

.
.
.
KILL IF BOND1*1 IS IN A RING OF SIZE 3 THROUGH 8
....
      TRIPLE BOND1*1
      BREAK BOND2*1
....
SUBTRACT 25 IF A MINUS CHARGE IS BETTER ON &
      ATOM*1 THAN ON CARBON1*2.
```

Section 3: TARGET SPECIFICATION

The QLTARG field houses a number that represents the target specification of a qualifier, i.e. the structural or chemical feature mentioned in the qualifier. When a qualifier is interpreted, the LHASA or DEREK programs decode the number contained in the QLTARG field and locate all of the atoms and bonds in the current molecule that possess the feature represented by that QLTARG number. Bits corresponding to the numbers of the atoms and bonds containing the target feature are turned on, respectively, in the target atom and bond sets. These sets are then used in conjunction with the locant atom and bond sets (see Section 4) to determine whether the conditions of the present qualifier have been met by the molecule currently under examination. The conditions of the qualifier are met (i.e. the qualifier is "true") if the logical AND of the target atom set with the locant atom set (or the target bond set with the locant bond set for qualifiers referring explicitly to bonds) is nonzero.

Occasionally it is useful to identify the atoms and bonds in a molecule that do not contain a certain target feature. In such cases, the QLNOTT (NOT Target) specifier NON is used. The qualifier

IF ATOM*1 IS NON ENOLIZABLE...,

for example, causes the target atom set to be filled with all nonenolizable atoms. This qualifier is thus different from

IF ATOM*1 IS NOT ENOLIZABLE...

in which the target atom set is filled with all enolizable atoms.

When the NON qualifier is applied to a functional group type, either a specific group such as NON ALCOHOL or a generic group such as NON WITHDRAWING, the target atom set is filled with the set of atoms that are origins of functional groups of other than the specified type(s). When dealing with composite functional groups such as GLYCOL, the origins will still be placed in the target atom set since they are also the origins of ALCOHOL, ETHER or ESTERX functional groups.

When no target word is mentioned in the qualifier, as in

ADD 25 FOR EACH ATOM ALPHA TO ATOM*4 OFFPATH

the target atom and bond sets are filled, respectively, with all atoms and bonds in the molecule.

Presently, there are five general categories of target specifiers: functional group specifiers, atom and bond specifiers, ring specifiers, global specifiers, and miscellaneous specifiers. Global specifiers are distinguished from all other QLTARGs in that their definition holds for all values of QLOPER. Other QLTARGs are defined only when QLOPER is less than four, e.g. only when an ADD, SUBTRACT, or KILL operation has been specified (QLOPERs 1, 2, and 3 respectively) or when no operation has been specified (QLOPER 0).

3.1 Functional Group Specifiers

3.1.1 Explicit Functional Groups

The functional group types recognized by LHASA and DEREK are listed below with representative patterns included for convenience. The patterns shown are not part of the CHMTRN language. In each pattern, the first carbon atom is the functional group "origin" (see CARBON1*1, etc.).

Functional Group	Pattern	Field	Notes
KETONE	C(=O) (-C)-C	[QLTARG 1]	
ALDEHYDE	C(=O)-H	[QLTARG 2]	
ACID	C(=O)-OH	[QLTARG 3]	
ESTER	C(=O)-OC	[QLTARG 4]	
AMIDE*1	C(=O)-NH ₂	[QLTARG 5]	
AMIDE*2	C(=O)-NH-C	[QLTARG 6]	
AMIDE*3	C(=O)-N(-C)-C	[QLTARG 7]	
CARBONIUM	C ⁺	[QLTARG 8]	(ION is a buzzword)
ISOCYANATE	C-N=C=O	[QLTARG 9]	
ACID*HALIDE	C(=O)-X	[QLTARG 10]	
THIOESTER	C(=O)-S-C	[QLTARG 11]	
AMINE*3	C-N(-C)-C	[QLTARG 12]	
AZIRIDINE	C---C \ / N	[QLTARG 13]	(secondary and tertiary)
AMINE*2	C-NH-C	[QLTARG 14]	
AMINE*1	C-NH- ₂	[QLTARG 15]	
NITROSO	C-N=O	[QLTARG 16]	
DIAZO	C=N+=N-	[QLTARG 17]	
HALOAMINE	C-N-X	[QLTARG 18]	(any halide)
HYDRAZONE	C=N-N	[QLTARG 19]	
OXIME	C=N-OH	[QLTARG 20]	
IMINE	C=NR	[QLTARG 21]	(primary and secondary)
THIOCYANATE	C-S-C#N	[QLTARG 22]	
ISOCYANIDE	C-N+#C-	[QLTARG 23]	
NITRILE	C#N	[QLTARG 24]	(synonym CYANO)
AZO	C-N=N-C	[QLTARG 25]	
HYDROXYLAMINE	C-N-OH	[QLTARG 26]	
NITRO	C-N(=O)-O-	[QLTARG 27]	
ENAMINE	C(-N(-C)-C)=C	[QLTARG 28]	
THIOL	C-SH	[QLTARG 29]	
EPISULFIDE	C---C \ / S	[QLTARG 30]	(synonym EPISULPHIDE)
SULFIDE	C-S-C	[QLTARG 31]	(synonym SULPHIDE)
SULFOXIDE	C-S(=O)-C	[QLTARG 32]	(synonym SULPHOXIDE)
SULFONE	C-S(=O)(=O)-C	[QLTARG 33]	(synonym SULPHONE)
C*SULFONATE	C-S(=O)(=O)-O	[QLTARG 34]	(synonym C*SULPHONATE)
ANHYDRIDE	C(=O)-O-C=O	[QLTARG 35]	
PHOSPHINE	C-P	[QLTARG 36]	

Functional Group	Pattern	Field	Notes
PHOSPHONATE	C-O-P	[QLTARG 37]	
EPOXIDE	C---C \ / O	[QLTARG 38]	
ETHER	C-O-C	[QLTARG 39]	
PEROXIDE	C-O-OR	[QLTARG 40]	(including hydroperoxides)
ALCOHOL	C-OH	[QLTARG 41]	(synonym HYDROXYL)
ENOL*ETHER	C(-OC)=C	[QLTARG 42]	
O*SULFONATE	C-O-S(=O)=O	[QLTARG 43]	(synonym O*SULPHONATE)
FLUORIDE	C-F	[QLTARG 44]	(<u>not</u> FLUORINE)
CHLORIDE	C-Cl	[QLTARG 45]	(<u>not</u> CHLORINE)
BROMIDE	C-Br	[QLTARG 46]	(<u>not</u> BROMINE)
IODIDE	C-I	[QLTARG 47]	(<u>not</u> IODINE)
GEM*DIHALIDE	C(-X)-Y	[QLTARG 48]	(any halide)
TRIHALIDE	C(-X)(-Y)-Z	[QLTARG 49]	(any halide)
ACETYLENE	C#C	[QLTARG 50]	
OLEFIN	C=C	[QLTARG 51]	
VIC*DIHALIDE	C(-X)-C-Y	[QLTARG 52]	(any halide)
HALOHYDRIN	C(-X)-C-OR	[QLTARG 53]	
GLYCOL	C(-OR)-C-OR'	[QLTARG 54]	
HEMIACETAL	C(-OH)-OC	[QLTARG 55]	
ACETAL	C(-OC)-OC	[QLTARG 56]	
AZIDE	C-N=N+=N-	[QLTARG 57]	
DISULFIDE	C-S-S-C	[QLTARG 58]	(synonym DISULPHIDE)
ALLENE	C=C=C	[QLTARG 59]	
VINYLSILANE	C(-SiR3)=C	[QLTARG 60]	
VINYLW	C=C-W	[QLTARG 61]	(vinyl-extended W-group)
VINYLD	C=C-D	[QLTARG 62]	(vinyl-extended D-group)
ESTERX	C-O-C=O	[QLTARG 63]	(RO end of an ester)
AMIDZ	C-N-C=O	[QLTARG 64]	(R ₂ N end of an amide)
SELENIDE	C-Se-C	[QLTARG 65]	
TRIALKYLSILYL	C-SiR3	[QLTARG 66]	
TRIALKYLSILOXY	C-O-SiR3	[QLTARG 67]	
SILYLENOETHER	C(-O-Si)=C	[QLTARG 68]	
N*UREA*H	C-N-C(=O)-NH	[QLTARG 69]	(or C-NH-C(=O)-N)
N*UREA*C	C-NR-C(=O)-NR2	[QLTARG 70]	
CARBAMATE*H	C(=O)(-NH)-OR	[QLTARG 71]	(not used in transforms)
CARBAMATE*C	C(=O)(-NR2)-OR	[QLTARG 72]	(not used in transforms)
O*CARBAMATE	C-O-C(=O)-N	[QLTARG 73]	(for use in transforms)
N*CARBAMATE	C-N-C(=O)-OR	[QLTARG 74]	(for use in transforms)
O*CARBONATE	C-O-C(=O)-OR	[QLTARG 75]	(RO end of a carbonate)
DITHIOACETAL	C(-SR)(-SR)-H	[QLTARG 76]	
DITHIOKETAL	C(-SR)(-SR)-C	[QLTARG 77]	

These targets are most commonly used to check for functionality on a particular locant, as in

IF THERE IS A KETONE ON ATOM*1...

or, for those transforms that can be keyed by more than one functional group type, to identify within the body of a transform the group that keyed entry into that transform, as in

IF THE FIRST GROUP IS AN ESTER....

The functional group targets are also commonly used in mechanism commands where a specific functional group type is to be put on a locant, as in

ATTACH A NITRO GROUP TO ATOM*3.

In addition, explicit functional group types can be specified in an FGI or FGA subgoal request transform, as in

EXCHANGE THE GROUP FOR AN ALCOHOL ON ATOM*3.

For all functional group targets, the target atom and bond sets are filled to contain all atoms and bonds in the groups of the target type excluding the keying functional groups. This latter restriction is important: it is not possible to use

IF THERE IS AN AMINE*1 ON ATOM*1 THEN...

to find out whether the first keying group (an amine) is primary or not, since the target atom set will not include the origins of the keying groups.

3.1.2 Generic Functional Groups

The following specifiers in this subsection are generic functional group specifiers. They may be used exactly like the explicit functional group specifiers.

METHYLENE [QLTARG 129]

This specifier refers to methylene double bonds. The target bond set is filled with the set of all bonds that are methylene double bonds. The target atom set is filled with the carbons in methylene double bonds that are opposite the double bond carbon bearing the two hydrogens. When used in a subgoal request or mechanism command, as in

EXCHANGE THE GROUP FOR A METHYLENE
or ATTACH A METHYLENE TO ATOM*2

an exo methylene is attached to the specified locant.

CARBONYL [QLTARG 130]

This specifier refers collectively to all ALDEHYDES and KETONES. A subgoal request such as

MAKE A CARBONYL GROUP ON ATOM*6

causes a request for both an aldehyde and a ketone to be generated. When used in a mechanism command, the CARBONYL specifier is equivalent to the explicit functional group specifier KETONE.

CARBOXYL [QLTARG 131]

The CARBOXYL specifier refers collectively to ACIDs, ESTERs, AMIDEs, ACID*HALIDEs, THIOESTERs, and ANHYDRIDEs. When used in a subgoal request, requests for all of the above-mentioned groups are made. When used in a mechanism command, the CARBOXYL specifier is equivalent to the explicit functional group specifier ACID.

HALIDE [QLTARG 132]

HALIDE refers collectively to the functional groups FLUORIDE, CHLORIDE, BROMIDE, and IODIDE. Note that this specifier is meant to refer to those (carbon) locants bearing halides and not specifically to the halogen atoms themselves, as in

IF THERE IS A HALIDE ON ATOM*2....

Explicit halogen atoms are accessed by HALOGEN*ATOM (*vide infra*) or by their individual atomic types FLUORINE, CHLORINE, BROMINE, IODINE.

AMINE [QLTARG 133]

AMIDE [QLTARG 134]

These specifiers refer collectively to all amine (or amide) functional group types (e.g. AMINE*1, AMINE*2, AMINE*3). When used in a subgoal request, all amines (or amides) are requested. In mechanism commands, AMINE is equivalent to the explicit functional group specifier AMINE*1 and AMIDE is equivalent to AMIDE*1.

3.1.3 Generic Functional Groups Used in Mechanisms

The following generic functional group specifiers represent fragments that are often attached in mechanism commands, but not used in subgoal requests.

PHENYL [QLTARG 141]

ISOPROPYL [QLTARG 142]

T*BUTYL [QLTARG 143] (synonym TBUTYL)

These specifiers refer to appendage types that may be used in mechanism commands only. They are useful for building complex substructures with a minimum number of mechanism commands. For example,

```

ATTACH A T*BUTYL GROUP TO THE NEWEST*ATOM
or  INTRODUCE A PHENYL GROUP TO POSITION*1
or  ATTACH AN ISOPROPYL GROUP TO ATOM*1

```

3.1.4 Other Generic Functional Group Specifiers

WITHDRAWING [QLTARG 150]
DONATING [QLTARG 151]

The WITHDRAWING specifier refers collectively to all KETONES, ALDEHYDES, ESTERS, AMIDES, CARBONIUM IONS, ACID*HALIDES, THIOESTERS, OXIMES, IMINES, NITRILES, NITROS, SULFOXIDES, SULFONES, SULFONATES (C*SULFONATE), ANHYDRIDES, TRIHALIDES, and VINYLW groups.

WITHDRAWING can be used to specify either the carbon origins of a withdrawing-group or to specify the sites activated by the "withdrawing bonds" emanating from the withdrawing-group. When used with the buzzword GROUP, as in

IF THERE IS A WITHDRAWING GROUP ON ATOM*3...

the WITHDRAWING specifier refers to the carbon origins of the group. When WITHDRAWING is used with the keyword BOND(S), however, as in

IF THERE IS A WITHDRAWING BOND ON ATOM*3...,

the locants referred to are those activated by the withdrawing-group. Note that the usage with BOND should not be combined with logical specifiers OR: and AND:. For example,

KILL IF THERE IS A WITHDRAWING BOND ON ATOM*3 OR: ON ATOM*5

is illegal and should be split up into two qualifiers. Withdrawing-groups such as KETONE, ESTER, CYANO, etc. ("expandable" withdrawing-groups) activate the locants alpha to the group origin and, consequently, WITHDRAWING GROUP and WITHDRAWING BOND refer to different atom locants. In "nonexpandable" withdrawing-groups (SULFOXIDE, SULFONE, NITRO, and C*SULFONATE), however, WITHDRAWING GROUP and WITHDRAWING BOND refer to the same atom locants since the sites activated by the withdrawing bonds of nonexpandable groups are also the carbon origins of those groups. The two types of groups may be referred to collectively via the specifiers EXPANDABLE*WITHDRAWING and NONEXPANDABLE*WITHDRAWING.

As with all functional group specifiers, atoms and bonds in keying functional groups (and in the case of WITHDRAWING BONDS, those bonds emanating from keying functional groups) are not added to the target atom and bond sets.

Another use of the WITHDRAWING specifier is in a subgoal request such as

MAKE A WITHDRAWING GROUP ON ATOM*1
 or EXCHANGE THE GROUP FOR A WITHDRAWING GROUP ON ATOM*6.

When making a subgoal request for a withdrawing-group, it is important to differentiate between expandable and nonexpandable withdrawing-groups. Clearly, adding a nitro group on a locant is

quite different from adding a ketone there since different sites would be activated by each functional group. To allow either type of withdrawing group to be specified in a subgoal request, the word **WITHDRAWING** is taken to mean an expandable withdrawing group. Using **WITHDRAWING** in this context is equivalent to using **EXPANDABLE*WITHDRAWING**. Requests for a nonexpandable withdrawing group are made by using the specifier **NONEXPANDABLE*WITHDRAWING**, as in

MAKE A NONEXPANDABLE*WITHDRAWING GROUP ON ATOM*1.

The **DONATING** specifier is completely analogous to the **WITHDRAWING** specifier, with all the donating groups (**AMINEs**, **HALOAMINEs**, **HYDROXYLAMINEs**, **THIOLs**, **SULFIDEs**, **PHOSPHINEs**, **ETHERs**, **ALCOHOLs**, **HEMIACETALs**, **ACETALs**, **VINYLDs**, **AMIDZs**, **ESTERXs**, **SELENIDES**, **TRIALKYLSILOXYs**, **N*UREAs**, **N*CARBAMATEs**, **O*CARBAMATEs**, and **O*CARBONATE** groups) classified as nonexpandable.

EXPANDABLE*WITHDRAWING [QLTARG 152]
NONEXPANDABLE*WITHDRAWING [QLTARG 153]

These specifiers refer to all expandable withdrawing groups (**KETONE**, **ALDEHYDE**, **ESTER**, **AMIDE**, **CARBONIUM ION**, **ACID*HALIDE**, **THIOESTER**, **OXIME**, **IMINE**, **NITRILE**, **ANHYDRIDE**, **TRIHALIDE**) and to all non-expandable withdrawing groups (**NITRO**, **SULFOXIDE**, **SULFONE**, and **SULFONATE (C*SULFONATE)**), respectively. They may be used to check for the existence of these types of groups, as in

ADD 10 IF THERE IS AN EXPANDABLE*WITHDRAWING GROUP ON ATOM*4

or in subgoal requests, as in

MAKE A NONEXPANDABLE*WITHDRAWING GROUP ON ATOM*2.

For more details of the definitions of expandable and non-expandable withdrawing groups, see **WITHDRAWING**.

LEAVING [QLTARG 154]

This specifier refers collectively to the set of all leaving groups - **AZIRIDINEs**, **THIOCYANATEs**, **THIOLs**, **EPISULFIDEs**, **SULFIDEs**, **SULFONEs**, **SULFONATEs (C*SULFONATEs and O*SULFONATEs)**, **PHOSPHONATEs**, **EPOXIDEs**, **ETHERs**, **ALCOHOLs**, **FLUORIDEs**, **CHLORIDEs**, **BROMIDEs**, **IODIDEs**, **GEM*DIHALIDEs**, **TRIHALIDEs**, **ACETALs**, **ESTERXs**, **AMIDZs**, **SELENIDES**, **TRIALKYL SILYLOXYs**, **N*UREA*Hs**, **N*UREA*Cs**, **O*CARBAMATEs**, **N*CARBAMATEs**, **O*CARBONATEs**, **DITHIOACETALs**, and **DITHIOKETAL** groups.

GOOD*LEAVING [QLTARG 155]

This specifier refers collectively to the set of all good leaving groups - **AZIRIDINEs**, **EPISULFIDEs**, **PHOSPHONATEs**, **EPOXIDEs**, **O*SULFONATEs**, **CHLORIDEs**, **BROMIDEs**, **IODIDEs**, **GEM*DIHALIDEs**, and **TRIHALIDE** groups.

PRIMARY	[QLTARG 156]
SECONDARY	[QLTARG 157]
TERTIARY	[QLTARG 158]

These specifiers refer to the carbon substitution about a functional group, as in

ADD 20 IF THERE IS A PRIMARY AMINE*3 ON ATOM*4
or EXCHANGE THE GROUP FOR A SECONDARY ETHER.

They are defined also as word incrementors because the functional group specifier following one of them is itself a QLTARG. These specifiers refer only to functional groups and are not to be confused with the PRIMARY*CENTER-QUATERNARY*CENTER (PRIMARY*CENTRE - QUATERNARY*CENTRE) specifiers.

FUNCTIONAL	[QLTARG 159]
-------------------	---------------------

This target specifier, which is used in conjunction with the buzzword GROUP, refers to the atoms and bonds in functional groups excluding keying functional groups and any groups sharing bonds with the keying groups. The target atom set is filled only with the carbon origins of functional groups while the target bond set is filled with the group bonds attached to those carbon origins. The only functional groups considered are the explicit functional groups listed in Section 3.1.1. Note that ALKYL and ARYL are not explicit functional groups. FUNCTIONAL is most commonly used to check for the presence of a group on a particular locant, as in

IF THERE IS A FUNCTIONAL GROUP ON ATOM*5...
or IF BOND*3 IS IN A FUNCTIONAL GROUP....

It may also be used in a counting operation, such as

IF THERE ARE MORE THAN TWO FUNCTIONAL GROUPS ON ATOM*1....

In this case, the target atom set is filled with the number of bits corresponding to the number of functional groups with origins on the specified locant.

3.2 Atom and Bond Specifiers

3.2.1 Explicit Atom Types

HYDROGEN(S)	[QLTARG 211]
--------------------	---------------------

This specifier refers not to all atoms whose type is hydrogen, but rather to all atoms bearing hydrogens, both explicit and implicit. For qualifiers of the type

IF THERE IS A HYDROGEN ON ATOM*2...

the target atom set is filled with those locant atoms bearing hydrogen. For hydrogen counting qualifiers, however, such as

IF THERE ARE THREE HYDROGENS ON ATOM*5...
or FOR EACH HYDROGEN ON HETERO1*1...

the target atom set is filled with the number of bits corresponding to the number of hydrogens on the specified locant.

HYDROGEN*ATOM(S) [QLTARG 212]

Explicit hydrogen atoms are accessed by HYDROGEN*ATOM. For example,

KILL IF THERE IS NO HYDROGEN*ATOM ALPHA TO ATOM*2

OXYGEN(S)	[QLTARG 213]
NITROGEN(S)	[QLTARG 214]
CARBON(S)	[QLTARG 215]
PHOSPHORUS	[QLTARG 216] (synonym PHOSPHOROUS)
SULFUR	[QLTARG 217] (synonym SULPHUR)
FLUORINE	[QLTARG 218]
CHLORINE	[QLTARG 219]
BROMINE	[QLTARG 220]
IODINE	[QLTARG 221]
SILICON	[QLTARG 222]
SELENIUM	[QLTARG 223]
COPPER	[QLTARG 224]
BORON	[QLTARG 225]
ALUMINUM	[QLTARG 226]
TIN	[QLTARG 227] (synonym STANNUM)
TITANIUM	[QLTARG 228]

These specifiers are used to ascertain whether a particular locant is of the indicated atom type, as in

IF ALPHA TO ATOM*1 IS NOT CARBON...
or FOR EACH NITROGEN BETA TO ATOM*2....

the target atom set is filled with all atoms of the indicated type and the target bond set with all bonds emanating from those atoms. These specifiers may also be used in a mechanism command, as in

ATTACH A CARBON TO ATOM*2.

3.2.2 Generic Atom Types

HALOGEN*ATOM(S) [QLTARG 241]

This specifier refers collectively to all halogen atoms - FLUORINE, CHLORINE, BROMIDE, IODINE. The target atom set is filled with the numbers corresponding to all halogen atoms and the target bond set with all bonds emanating from those atoms.

HETERO [QLTARG 242]

The target atom set is filled with all atoms that are neither carbon nor explicit hydrogen, thereby allowing such locants to be referenced, as in

```

      IF ALPHA TO ATOM*3 IS HETERO...
or    F THERE ARE TWO HETERO ATOMS BETA TO ATOM*4...
```

METHYL*ATOM(S) [QLTARG 243]

METHYL*ATOM allows direct access to the carbon atoms of methyl groups (see METHYL). METHYL*ATOM is distinct from METHYL in that the target atom set is returned as the set of actual methyl carbons rather than the set of atoms bearing methyl groups.

ALKYL*ATOM(S) [QLTARG 244]

ALKYL*ATOM allows access to all first carbons in alkyl groups. ALKYL*ATOM is distinct from ALKYL in that the target atom set is returned with the actual alkyl carbons rather than the set of atoms bearing alkyl groups.

METHYL(S) [QLTARG 245]

The METHYL specifier identifies all carbon atoms bearing three hydrogens (either explicit or implicit). METHYL is analogous to HYDROGEN and ALKYL and distinct from METHYL*ATOM in that the target atom set is returned as the set of atoms bearing methyl groups rather than the methyl carbons themselves. Note that either METHYL or METHYL*ATOM may be used in counting qualifiers. For example,

```

      IF THERE ARE TWO METHYL*ATOMS ON ALPHA TO ATOM*1...
and    IF THERE ARE TWO METHYL GROUPS ON ATOM*1...
```

are equivalent except that the former qualifier leaves the actual methyl carbons in the resultant set while the latter qualifier leaves ATOM*1 in the resultant set, provided of course that there are in fact methyl groups on ATOM*1.

ALKYL(S)**[QLTARG 246]**

This specifier is used to identify alkyl groups attached to a particular locant, as in

IF THERE IS AN ALKYL GROUP ON ATOM*1....

An ALKYL group is defined as any sp^3 hybridized carbon atom bearing no hetero atom appendages. ALKYL is sophisticated about the direction from which the request about an alkyl group is made. For example, the oxygen atom in methyl ethyl ether has two alkyl groups on it, but if one asks whether the methyl carbon in the ethyl group bears an alkyl group, the answer will be "no" since the methylene carbon now bears a hetero atom. ALKYL is analogous to HYDROGEN and METHYL and distinct from ALKYL*ATOM in that the target atom set is returned with the set of atoms bearing alkyl groups rather than the alkyl carbons themselves. Thus, the qualifiers

IF THERE IS AN ALKYL GROUP ON ATOM*3...

and IF THERE IS AN ALKYL*ATOM ON ALPHA TO ATOM*3...

are equivalent except that the former qualifier leaves ATOM*3 in the resultant set and the latter leaves the first carbons of any alkyl groups on ATOM*3 in the resultant set, provided that there are in fact one or more alkyl groups on ATOM*3.

As with METHYL and METHYL*ATOM, both ALKYL and ALKYL*ATOM may be used in counting qualifiers (see METHYL).

Note that the definition of ALKYL does not include aromatic atoms. Path atoms are not specifically excluded, however. The preferred method for finding alkyl groups that are not on the path is to use ALKYL*ATOM, as in

ADD 10 FOR EACH ALKYL*ATOM ON ALPHA TO ATOM*2 OFFPATH.

3.2.3 Stereochemical Atom and Bond Specifiers

(See also section 5 for definitions of other stereochemical specifiers).

STEREOCENTER(S)**[QLTARG 256] (synonym) STEREOCENTRE(S)**

This specifier refers to atoms that are stereocenters and to the bonds that are attached to those centers, as in

IF ATOM*3 IS A STEREOCENTER...

or IF BOND*2 IS ATTACHED TO A STEREOCENTER....

CIS**[QLTARG 257]****TRANS****[QLTARG 258]**

These specifiers refer to the stereo-relationship of two locants about a keying reference ring. Since two locants are always required, the words *cis* and *trans* must appear between the two locants in the qualifier, as in

IF SAVED*ATOM 1 IS CIS TO SPECIFIED*ATOM 1 &
ABOUT THE RING CONTAINING BOND*4....

The reference ring may be specified in one of four ways:

- 1) by specifying a bond within the ring, as in the example above;
- 2) by specifying an atom within the ring (note that this method is often less precise than method 1)
- 3) by using two atom specifiers, as in

IF HETERO1*1 IS TRANS TO BOND2*1 ABOUT THE &
RING CONTAINING FROM ATOM*1 TOWARDS ATOM*3...,

in which case the reference ring must contain all bonds in the shortest path from one atom locant (ATOM*1) to the other (ATOM*3); 4) by specifying the CURRENT RING, which is automatically assigned in special substructure tables but can also be DESIGNATED inside a transform or rule, as in

IF HETERO1*2 IS CIS TO SAVED*BOND 1 ABOUT THE CURRENT*RING....

CIS*FUSION	[QLTARG 259]
TRANS*FUSION	[QLTARG 260]

These specifiers refer to all *cis* or *trans* fusion bonds, as in

IF THE BOND ALPHA TO ATOM*4 OFFPATH IS A TRANS*FUSION....

CIS*OLEFIN	[QLTARG 261]
TRANS*OLEFIN	[QLTARG 262]

These specifiers refer only to C=C bonds with one hydrogen on each carbon atom spanning the bond. The target bond set is filled with all such *cis* or *trans* bonds and the target atom set with all the atoms spanning those bonds. Hence, either atom or bond locants may be referenced, as in

IF BOND*2 IS A TRANS*OLEFIN...
or IF ATOM*5 IS IN A CIS*OLEFIN....

R*CENTER(S)	[QLTARG 263] (synonym R*CENTRE(S))
S*CENTER(S)	[QLTARG 264] (synonym S*CENTRE(S))

These specifiers may be used to obtain information about the absolute stereochemistry at an sp³ center, e.g.

IF CARBON1*1 IS AN R*CENTER....

R*CENTER and S*CENTER both follow the Cahn, Ingold, Prelog rules of stereochemical nomenclature.

AXIAL [QLTARG 265]
EQUATORIAL [QLTARG 266]

These specifiers are used to identify locants that are axial or equatorial about a six-membered ring, as in

IF SAVED*ATOM 2 IS AXIAL ABOUT THE RING CONTAINING BOND*4....

The reference ring may be specified in any of the ways used for CIS and TRANS.

BOW*ATOM [QLTARG 280]

This specifier is used in conjunction with the DESIGNATE command to fix a particular boat conformation of a six-membered ring, e.g.

DESIGNATE ATOM*6 AS A BOW*ATOM.

3.2.4 Ring Atom and Bond Specifiers

ARYL [QLTARG 281] (synonym AROMATIC)

This specifier refers to atoms and bonds that exhibit aromatic character, e.g.

IF CARBON2*2 IS AROMATIC...
 or IF BOND*2 IS ARYL....

If the keyword CURRENT RING is used with ARYL, the entire ring is checked for aromaticity. It is therefore important how CURRENT*RING qualifiers are phrased. Consider

IF THE CURRENT*RING IS AROMATIC...
 and not
 IF THERE IS AN AROMATIC ATOM IN THE CURRENT*RING....

The correct phrasing of the latter qualifier would be

IF THERE IS AN AROMATIC ATOM CONTAINED IN THE CURRENT*RING....

BRIDGEHEAD(S) [QLTARG 282]

This specifier refers to the sets of bridgehead atoms and bonds attached to those atoms, as in

IF CARBON1*1 IS A BRIDGEHEAD...
 or IF BOND*3 IS A BOND TO A BRIDGEHEAD....

SPIRO**[QLTARG 283]**

This specifier refers to the sets of all spiro atoms and all bonds attached to spiro atoms, as in

IF THERE ARE TWO SPIRO ATOMS ALPHA TO ATOM*2 OFFPATH...
or IF SAVED*BOND 1 IS A BOND TO A SPIRO ATOM....

FUSION(S)**[QLTARG 284] (synonym FUSED)**

This specifier fills the target atom set with all fusion atoms and the target bond set with all fusion bonds. For example,

IF ATOM*4 IS A FUSION ATOM...
or IF THE BOND ALPHA TO ATOM*3 IS A FUSION....

3*OR*MORE*RINGS**[QLTARG 285]**

This specifier refers to all atoms and bonds that are in three or more rings:

IF BOND*5 IS IN 3*OR*MORE*RINGS THEN....

SMALL*FUSION**[QLTARG 286]**

The SMALL*FUSION specifier refers to fusion bonds between the following ring sizes: 3/3, 3/4, 3/5, 3/6, 4/4, 4/5, 4/6, 5/5.

CORE**[QLTARG 287]**

This specifier refers to fusion and/or bridge bonds that, if disconnected, would leave an undesirably large (eight-membered or greater) ring envelope. A qualifier such as

KILL IF BOND*2 IS A CORE BOND

for example, would be appropriate in many disconnective transforms.

BREDT*STRAINED**[QLTARG 288]**

This specifier refers to all bridgehead bonds that are not in a real or pseudo (envelope) ring of size eight or larger. This specifier may refer to atoms as well as bonds. For example,

IF ATOM*2 IS BREDT*STRAINED....

The BREDT*STRAINED specifier is different from and should not be confused with the STRAINED specifier.

STRAINED**[QLTARG 289]**

This QLTARG refers to all bonds that are either:

- 1) endocyclic in a three- or four-membered ring;
- 2) endocyclic in a five-membered ring that is part of a [2.1.1], [2.2.1], or [3.2.1] bicyclic system;
- 3) *trans* C=C's in rings less than ten-membered; or
- 4) C=C's exocyclic to three- or four-membered rings.

Although the concept of strain is implicitly associated with bonds, the STRAINED specifier may refer to atoms as well as bonds, as in

IF ATOM*2 IS STRAINED....

All atoms spanning strained bonds may be referenced in this manner. This specifier is different from and should not be confused with the BREDT*STRAINED specifier.

DIARYL*FUSION**[QLTARG 290]**

This specifier refers to all fusion bonds between two aromatic rings, as in

IF BOND*2 IS A DIARYL*FUSION....

RING(S)**[QLTARG 292]****SIZE(S)****[QLTARG 293,**

**QLRING_QUAL 1,
QLRING_ISIZE,
QLRING_JSIZE]**

HETEROCYCLIC**[QLTARG 293,**

QLRING_QUAL 2]

CARBOCYCLIC**[QLTARG 293,**

QLRING_QUAL 3]

COMMON**[QLTARG 293,**

QLRING_QUAL 4]

BRIDGED*TO*CURRENT*RING**[QLTARG 293,**

QLRING_QUAL 5]

FUSED*TO*CURRENT*RING**[QLTARG 293,**

QLRING_QUAL 6]

LARGER

The RING specifier allows atoms and bonds in rings to be referenced, as in

ADD 10 IF ATOM*1 IS IN A RING.

A variety of modifying adjectives may be used with RING. These modifiers are packed into a type-C word (see Figure 1.1). This word is either the second or third word of a qualifier, depending on whether a type-B word is present. The existence of the type-C word is signaled by QLTARG 293 in the first word of the qualifier.

When RING is followed by the keyword SIZE and one or two numerical arguments, rings of specific sizes can be accessed, as in

IF ATOM*1 IS IN A RING OF SIZE 5...

or IF BOND2*3 IS IN A RING OF SIZE 4 THROUGH 7....

When used with two numerical arguments, the keyword THROUGH must separate the numbers.

The keyword LARGER may be used as the second numerical argument, as in

IF BOND1*1 IS IN A RING OF SIZE 8 OR LARGER....

With LARGER, however, the keyword OR must be used in place of THROUGH as the numerical argument separator.

The target atom and bond sets are filled with all the atoms and bonds contained in rings of the designated size(s). If a ring has been designated as the CURRENT*RING (this is done automatically for special substructure chemistry), however, the target atom and bond sets will not contain the atoms and bonds in the current ring unless those atoms and bonds are also in another ring of the appropriate size.

The modifiers CARBOCYCLIC and HETEROCYCLIC may also be used, as in

IF ATOM*1 IS IN A CARBOCYCLIC RING OF SIZE 6....

The COMMON modifier is used to identify rings that are common to two locants, as in

IF BOND*4 AND SAVED*ATOM 3 ARE IN A COMMON &
CARBOCYCLIC RING OF SIZE 8 OR LARGER....

Note that in the special case of the use of COMMON, any operation specifier (e.g. ADD, SUBTRACT, or KILL) must follow the qualifier. For example, the following qualifier is illegal:

KILL IF x AND y ARE IN A COMMON RING.

The RING specifiers may be used to set up a particular ring as the CURRENT*RING using the DESIGNATE command, as in

DESIGNATE THE CARBOCYCLIC RING OF SIZE 5 &
CONTAINING ATOM*3 AS THE CURRENT*RING.

The QLQUAL specifiers ON*CURRENT*RING and OFF*CURRENT*RING may then be used to get locants on or off the ring just designated.

Also, atoms and bonds that are in rings that are either fused or bridged to the CURRENT*RING may be found by using the FUSED*TO*CURRENT*RING or BRIDGED*TO*CURRENT*RING modifiers, respectively:

BRDATS GETS ATOMS IN RINGS BRIDGED*TO*CURRENT*RING
REMOVE: CURRENT*RING ATOMS FROM BRDATS.

BRIDGE*1	[QLTARG 303]
BRIDGE*2	[QLTARG 304]
BRIDGE*3	[QLTARG 305]

These specifiers allow the CHMTRN writer to refer specifically to the first, second, and third (respectively) bridges on a ring. BRIDGE*1 is the shortest bridge, BRIDGE*2 the next longer one, and BRIDGE*3 the next longer again. These specifiers can return an empty set if there is no such bridge. An example follows:

```

IF THE CURRENT*RING HAS A BRIDGE*2 AND:IF &
    THERE ARE FEWER THAN SEVEN ATOMS ON*THE*BRIDGE
BEGIN SECOND*BRIDGE
    THERE ARE ATOMS IN THE CURRENT*RING
    SAVE*SEQUENTIALLY STARTING FROM 2 &
        THE ATOMS ALPHA TO THE PREVIOUS LOCANT ON*THE*BRIDGE
BLKEND SECOND*BRIDGE

```

CIS*BRIDGE(D)	[QLTARG 306]
TRANS*BRIDGE(D)	[QLTARG 307]

These specifiers are entirely analogous to the BRIDGE(D) specifier (*vide infra*) and can be used in all the ways that BRIDGE(D) can be used:

```

IF THE CURRENT*RING IS CIS*BRIDGED....

```

If stereochemistry is not specified in the structure being evaluated, the answer to questions like the one above will always be "no."

LOCKED*CHAIR	[QLTARG 308]
LOCKED*BOAT	[QLTARG 309]

These specifiers refer only to six-membered CURRENT*RINGs, e.g.

```

IF THE CURRENT*RING IS A LOCKED*CHAIR....

```

"Locked" rings are those whose conformations are well-defined as one of the standard chair or boat forms by virtue of the substitution about the ring. Locked rings can be immobile, such as the locked, immobile boat six ring in norbornane, but need not be.

BRIDGE(D)	[QLTARG 310]
------------------	---------------------

This specifier refers to bridges about the ring currently being examined. The BRIDGED specifier can be used to find out if the keying ring is bridged, as in

```

IF THE CURRENT*RING IS BRIDGED....

```

or to find out if the ring is bridged at a particular site, as in

IF THE CURRENT*RING IS BRIDGED AT ATOM*5....

Note that the BRIDGED specifier will not necessarily refer to all the bridgehead atoms on the ring since the ring under consideration may not itself be bridged at all those sites. Also note that unlike many chemists, LHASA and DEREK do not recognize one-bond (i.e. zero-atom) "bridges".

The atoms and bonds in a specific bridge may also be referenced, as in

IF THERE IS A BRIDGE FROM ATOM*3 TOWARDS ATOM*7...
 or IF THERE IS A TWO BOND BRIDGE FROM ATOM*1 TOWARDS ATOM*5...
 or IF THERE IS A THREE ATOM BRIDGE FROM ATOM*6 TOWARDS ATOM*8....

Once a bridge has been identified, the atoms and bonds in that bridge can be referenced by the specifiers ON*THE*BRIDGE and OFF*THE*BRIDGE.

3.2.5 Comparison Atom and Bond Specifiers

MINUS	[QLTARG 311]
PLUS	[QLTARG 312]
RADICAL	[QLTARG 313]

The MINUS specifier compares the relative stabilities of a negative charge on two different locants, as in

SUBTRACT 35 IF A MINUS CHARGE IS BETTER ON ATOM*2 THAN ON ATOM*4
 or IF A MINUS CHARGE IS NOT BETTER ON ATOM*3 THAN ON ATOM*4....

Note that a MINUS CHARGE NOT BETTER THAN qualifier is taken to mean that the negative charge stabilization on the first locant is either equivalent to or worse than on the second locant.

The negative charge stabilization for a particular locant is calculated as 4*(the number of withdrawing bonds activating the locant) + 2*(the number of times the locant is allylic and/or benzylic) - 2*(the number of donating bonds deactivating the locant) - 1*(the number of alkyl groups on the locant).

The PLUS specifier is analogous to the MINUS specifier except that the plus charge stabilization for a locant is calculated as 2*(the number of donating bonds activating the locant) + 2*(the number of times the locant is allylic and/or benzylic) + 1*(the number of alkyl groups on the locant) - 4*(the number of withdrawing bonds deactivating the locant).

The RADICAL specifier is also analogous to the MINUS charge specifier except that radical stabilization for a locant is calculated as 2*(the number of withdrawing bonds activating the locant) + 2*(the number of donating bonds activating the locant) + 2*(the number of times the locant is allylic and/or benzylic) + 1*(the number of alkyl groups on the locant).

MORE*HINDERED**[QLTARG 314]****LESS*HINDERED****[QLTARG 315]**

These specifiers compare the relative steric hindrances at two different locants simply by comparing the number of non-hydrogen attachments at each locant, as in

IF ATOM*1 IS NOT MOREHINDERED THAN ATOM*2....

SAME**[QLTARG 316]****(synonym CONTAINED)**

SAME and CONTAINED are used to determine whether two locants, either both atom locants or both bond locants, overlap. SAME and CONTAINED are defined as the same QLTARG value, so the choice of one or the other is purely a semantic one. They must always appear between the two locants, as in

IF BOND*3 IS THE SAME AS SAVED*BOND 1...

or IF ALPHA TO ATOM*2 OFFPATH IS THE SAME AS BETA TO ATOM*3 OFFPATH...

or IF ATOM*3 IS CONTAINED IN FRAGMENT*1....

In cases where one of the locant specifiers may refer to more than a single atom or bond (as in the last two examples given above), the qualifier will be taken as true if any of the atoms (or bonds) referenced by the two locant specifiers are identical.

FEWER**[QLTARG 317]****MORE****[QLTARG 318]**

These keywords are used in counting operations. They compare the number of times the target feature of a qualifier occurs on its specified locant with the value of the QLINCR field. For example, the qualifier

IF THERE ARE FEWER THAN TWO HYDROGENS ON ATOM*2 &
THEN GO TO FEWHYD

would count the number of occurrences of the target, HYDROGEN, on the locant, ATOM*2, and compare it with the QLINCR value, TWO. The word separator THAN must follow FEWER and MORE. Because FEWER and MORE require the QLINCR field to house a value for comparison, ADD or SUBTRACT commands or any other specifiers making use of the QLINCR field must be preceded by the THEN specifier, as in

IF MORE THAN THREE CARBONS BETA TO ATOM*4 THEN SUBTRACT 25.

ALL	[QLTARG 319]
SOME	[QLTARG 320]
NONE	[QLTARG 321]

These keywords refer to a connected set of atoms, such as ATOM*1 TOWARDS ATOM*3. These specifiers require two locant arguments. For example,

KILL IF ALL OF ATOM*4 TOWARDS ATOM*2 IS NOT IN A RING OF SIZE 6.

Only those atoms and bonds on the shortest possible path are returned in the target atom and bond sets. Note also that the comparison between target and locant sets to decide the truth or falsity of the qualifier is done on bonds, not on atoms.

APPENDAGE(S) [QLTARG 322]

This specifier is used to check for the identity of two appendages. Each appendage is directional and is defined by two locant atoms, one source atom and one directional atom. Since a total of four atom locants must be specified, three word separators, one between each pair of locants, must be used. Note that each of the four locants must be non-zero. The following protocol is recommended:

IF THE APPENDAGES FROM [SOURCE 1] TOWARDS &
[DIRECTIONAL 1] AND FROM [SOURCE 2] &
TOWARDS [DIRECTIONAL 2] ARE IDENTICAL...

in which the words TOWARDS and AND serve as word separators. Operation codes (KILL, ADD 25, etc.) may appear at the beginning of the qualifier. When the source atoms belong to the same symmetry class (i.e., are chemically equivalent), EQUIVALENT, which is evaluated more efficiently, can often be used instead of APPENDAGES. Note that for each appendage, the source and directional atoms should be adjacent for best results. Note also that it is not possible to combine APPENDAGE qualifiers with other qualifiers using logical specifiers OR:, AND:, OR:IF, or AND:IF.

EQUIVALENT [QLTARG 323]

EQUIVALENT is a comparison specifier used to find out whether two atom locants belong to the same symmetry class (i.e., are chemically equivalent). EQUIVALENT can be used as follows:

ADD 20 IF ATOM*1 IS EQUIVALENT TO ATOM*4.

3.2.6 Miscellaneous Atom and Bond Specifiers

PRIMARY*CENTER(S)	[QLTARG 341] (synonym PRIMARY*CENTRE)
SECONDARY*CENTER(S)	[QLTARG 342] (synonym SECONDARY*CENTRE(S))
TERTIARY*CENTER(S)	[QLTARG 343] (synonym TERTIARY*CENTRE)
QUATERNARY*CENTER(S)	[QLTARG 344] (synonym QUATERNARY*CENTRE)

These specifiers refer to the degree of substitution on the locant atom as determined by the number of carbon atoms attached. If one wants to find out whether a locant is in any of several of these (e.g. tertiary or quaternary) the qualifier should be phrased

IF THERE ARE MORE THAN TWO CARBONS ON ALPHA TO ATOM*1....

These specifiers are not to be confused with the PRIMARY, SECONDARY, and TERTIARY specifiers, which refer to functional groups.

SINGLE*BOND(S)	[QLTARG 345]
DOUBLE*BOND(S)	[QLTARG 346]
TRIPLE*BOND(S)	[QLTARG 347]

These specifiers refer to all bonds of the indicated order in the current molecule regardless of the types of atoms spanning the bonds, as in

IF BOND*3 IS A SINGLE*BOND....

Note that these specifiers do not take account of aromaticity, i.e. a single bond in an aromatic ring is still considered to be a SINGLE*BOND.

MULTIPLE	[QLTARG 348] (synonym MULTIPLY)
-----------------	--

This specifier refers atom and bond locants that are multiple (including aromatic atoms and bonds), as in

IF BOND*2 IS MULTIPLE...
or IF ATOM*3 IS A MULTIPLY BONDED ATOM....

PRESERVED	[QLTARG 349]
------------------	---------------------

Bonds that have been graphically designated as preserved by the chemist are never broken in a transform. In the case of long-range searches, the table writer may want to spare the LHASA program the trouble of doing subgoals for an orientation where the eventual goal disconnection will be blocked by the preserved bond criterion. Accordingly, it is possible in CHMTRN to find out whether a bond has been graphically designated as preserved. For example

IF THERE IS A PRESERVED BOND AT BOND*4 OR: BOND*6 THEN REORIENT.

If the keyword BOND(S) does not appear, the PRESERVED specifier can be used to see whether a particular stereocenter has been graphically designated as preserved:

IF ATOM*1 IS PRESERVED....

ALLYLIC

[QLTARG 350]

Allylic atoms are defined as those saturated carbon atoms that are alpha to a carbon-carbon double bond. "Allylic bonds" may also be referenced by this specifier, as in

IF BOND1*3 IS ALLYLIC...

and are defined as those bonds to allylic carbons that do not lead back to the carbon-carbon double bond.

BENZYLIC

[QLTARG 351]

Benzylic atoms are defined as those saturated carbon atoms that are alpha to an aromatic carbon atom. The BENZYLIC specifier allows such locants to be referenced, as in

IF CARBON1*1 IS BENZYLIC....

"Benzylic bonds" may also be referenced by this specifier and are defined as those bonds to benzylic atoms that do not lead back to an aromatic atom.

CONJUGATED

[QLTARG 352]

This target may refer to either atoms or bonds, as in

IF BOND*2 IS CONJUGATED...

or FOR EACH CONJUGATED ATOM ALPHA TO ATOM*1....

For an atom locant to be considered conjugated, it must be alpha to a multiply bonded atom that is not in a keying group. Conjugated bonds, which need not be multiple themselves, are those that have a spanning atom alpha to a multiply bonded atom.

ENOLIZABLE

[QLTARG 353]

This specifier refers to the set of all enolizable atoms in the current molecule, as in

FOR EACH ENOLIZABLE ATOM ALPHA TO ATOM*1...

To be enolizable, a locant must be saturated, nonbridgehead, have at least one hydrogen, and be activated by a withdrawing bond.

GOAL*BOND	[QLTARG 372]
GOAL*FGI	[QLTARG 373]
GOAL*FGA	[QLTARG 374]
WITHDRAWING*REQUIRED	[QLTARG 377]
DONATING*REQUIRED	[QLTARG 378]

These specifiers allow the CHMTRN writer to query the goal list of objectives, identified by the LHASA starting-material-oriented-search executive, to be used in the conversion of target to starting material, as in

IF GOAL*FGI ON ATOM*1 THEN....

These specifiers are currently used only in the LHASA strategy table, PCHEMS1.

DOUBLY	[QLTARG 379]
TRIPLY	[QLTARG 380]

These specifiers refer to atom and bond locants that are double or triple (excluding aromatic atoms and bonds), as in

IF ATOM*3 IS A TRIPLY BONDED ATOM....

DOUBLY and TRIPLY exist primarily to provide access to the corresponding atoms in the CHMTRN language, since specifiers like DOUBLE*BOND refer exclusively to bonds.

NEGATIVE*CHARGE	[QLTARG 381]
CHARGE	[QLTARG 382]
RADICAL*CENTER	[QLTARG 383]
	(synonym RADICAL*CENTRE)

These specifiers are used to ascertain whether a particular locant has the indicated attribute, as in

IF ATOM*1 HAS A POSITIVE*CHARGE THEN....

ANY*GROUP	[QLTARG 384]
------------------	---------------------

This specifier is used to ask whether a locant is functionalized, independently from functional group recognition. As a consequence, ANY*GROUP does not set up a functional group which can be referred to in later qualifiers, as is the case with FUNCTIONAL (and FUNCTIONAL can set up only recognized functional groups). Atoms attached by aromatic bonds are excluded; for example, none of the atoms in the pyrrole ring bear ANY*GROUP, unless there are hetero atom attachments which are not connected by aromatic bonds. Thus, any atom bearing an N-pyrrole substituent does bear ANY*GROUP.

3.3 Miscellaneous Specifiers

FRAGMENT*COUNT

[QLTARG 501]

This specifier refers to the number of fragments produced after a transform mechanism has been performed, as in

IF THE FRAGMENT*COUNT IS TWO....

This is a simple way to determine whether the synthetic reaction corresponding to the current transform is intra- or intermolecular. Since the QLINCR field is used to store the number with which the fragment count is to be compared, ADD, SUBTRACT, and KILL operation codes must be preceded by the keyword THEN, as in

IF THE FRAGMENT*COUNT IS ONE THEN ADD 25.

Section 4: LOCANT SPECIFICATION

The locant specifications of a qualifier can be contained in as many as five fields - QLFROM, QLBOND, QLDIST, QLQUAL, and/or QLNOTL. The QLFROM field stores a number corresponding to the atom or bond actually mentioned in the qualifier sentence (e.g. ATOM*1, BOND*1, etc.).

The single-bit QLBOND field, set by the keyword BOND(S), causes a locant specifier that would otherwise refer to atoms (e.g. ATOM*3, SAVED*ATOM 2) to refer to bonds emanating from the atom locants (e.g. THE BONDS ATTACHED TO ATOM*3, THE BONDS ATTACHED TO SAVED*ATOM 2). The QLDIST field is used to refer to locations up to three atoms away (ALPHA, BETA, GAMMA) from the QLFROM atom locant.

The single-bit QLNOTL (NOT Locant) field (set by the keywords NOT*ON and NOT*LOCATED) negates the locant, i.e. causes the locant atom and bond sets to be filled with all locants other than those identified by the rest of the locant fields.

Additional restrictions (QUALifications) regarding the locant are housed in the QLQUAL field.

During transform or rule evaluation the locant atom and bond sets are filled with the atoms and bonds satisfying the locant specifications of the present qualifier. These sets are then compared with the target atom and bond sets (see Section 3) to determine whether or not the conditions of the qualifier have been met by the current molecule. The conditions of the qualifier are met (i.e. the qualifier is "true") if the logical AND of the target atom set with the locant atom set (or the target bond set with the locant bond set for qualifiers referring explicitly to bonds) is nonzero.

When no locant specifiers are mentioned in a qualifier, as in

ADD 25 FOR EACH OLEFIN ANYWHERE

the locant atom and bond sets are filled, respectively, with all atoms and bonds in the molecule.

4.1 Distance Specification

The QLDIST field allows the query of relative distance between locants.

ALPHA	[QLDIST 1]
BETA	[QLDIST 2]
GAMMA	[QLDIST 3]

These specifiers refer, respectively, to distances of one, two, and three away from the atom locant specified by the QLFROM field. Distance specifiers may not be used when the QLFROM field explicitly refers to a bond. The locant atom set is filled with the atoms that are the specified distance away, and the locant bond set with the bonds emanating from those atoms that do not lead back towards the reference QLFROM atom. The growing operations for these specifiers will never include or grow through any keying group atoms or path atoms.

Unless explicitly referenced by a HYDROGEN*ATOM or EXPLICIT*HYDROGEN specifier, explicit hydrogen atoms will not be included in any final locant atom sets. Bonds to explicit

hydrogens are included in the locant bond set in the absence of a growing operation (QLDIST = 0) but are excluded when QLDIST $\neq$ 0.

[QLDIST 4]

Although the word WITHIN is defined as QLDIST 4, it must always be used in conjunction with a distance specifier (ALPHA, BETA, or GAMMA), which causes the final value of QLDIST to range from 5 to 7 (*vide infra*). Since no combination of distance specifiers can ever yield a final QLDIST value of 4, and since it is only legal to specify distances away from atom locants, all bond locants have been defined as QLDIST 4 in addition to a QLFROM value. Thus, atom and bond locants defined by the same QLFROM values (e.g. ATOM*1 and BOND*1 are both defined as QLFROM 1) can be differentiated.

WITHIN

[QLDIST 5, 6, 7]

This keyword, which itself is defined as QLDIST 4, indicates that all locants up to and including the specified distance from the reference atom are acceptable. In

IF THERE IS AN OXYGEN ATOM WITHIN BETA TO ATOM*1...

for example, the locant atom and bond sets are filled with ATOM*1 and its attached bonds and with locants alpha and beta to ATOM*1. The word WITHIN must always appear with an ALPHA, BETA, or GAMMA specifier and, therefore, will cause the value of the QLDIST field to range from 5 (WITHIN ALPHA) to 7 (WITHIN GAMMA).

4.2 Atom and Bond Specification

Atom and bond locants are stored in the QLFROM field. Each QLFROM value can be used to define a unique atom locant and a unique bond locant. Differentiation between atoms and bonds is possible since all bond locants are defined additionally as QLDIST 4 (see Section 4.2.2).

4.2.1 Atom Locants

ATOM*1 - ATOM*32

[QLFROM 1-32, QLDIST $\neq$ 4]

Each transform has a path of atoms associated with it. The length and nature of the path depends on the executive reading the transform entry. The specifiers ATOM*1 through ATOM*32 are used to label the atoms along the path. For transforms with path length L, only ATOM*1 through ATOM*(L+1) will be defined. Path atom specifiers with numbers more than one higher than the path length of the current transform must not be used. Exceptions to this rule are the disconnective path change 0 FGA's (in FGATB0) and FGI's (in FGITB0), which may refer to BOND*1 (the bond to be broken by the transform) and ATOM*2.

SAVED*ATOM(S)**[QLFROM 33-64, QLDIST ≠ 4]**

This locant specifier references the atom locant(s) saved by a previous SAVE command, as in

```
SAVE AS 1 THE ATOMS ALPHA TO ATOM*6
```

```
IF SAVED*ATOM 1 IS ALLYLIC....
```

The word SAVED*ATOM refers only to atom locants. Saved bonds must be referenced by the SAVED*BOND specifier. Note that a SAVED*ATOM set may contain more than one atom or may in fact be empty. Care must be exercised in this latter case. In the example above, for instance, if there are no atoms ALPHA TO ATOM*6 then an error will result when the ALLYLIC qualifier that follows is evaluated.

Up to thirty-two atom sets may be saved. The SAVED*ATOM expression must be followed by an integer between 1 and 32 or by an expression that computes to such an integer. Note that SAVED*ATOM 1 is defined as QLFROM 33, SAVED*ATOM 2 as QLFROM 34, etc.

SPECIFIED*ATOM(S)**[QLFROM 65-69, QLDIST ≠ 4]**

A CHMTRN table SUBROUTINE may be called with up to five atom arguments and five bond arguments. (See CALL.) The atom arguments are referenced within the called subroutine by the SPECIFIED*ATOM X specifier. SPECIFIED*ATOM 1 is also used to refer to an atom argument within an iterative DO block (see DO). Note that a SPECIFIED*ATOM, like a SPECIFIED*BOND or a SAVED*ATOM or SAVED*BOND, is stored as a set and may contain more than one atom.

Only atom locants are referenced by this specifier. Bond arguments to subroutine CALLs and DO blocks must be referenced by the SPECIFIED*BOND specifier.

All specified atoms can be referenced collectively in the called subroutine via the ON*SPECIFIED*PATH and OFF*SPECIFIED*PATH keywords. Note that SPECIFIED*ATOM 1 is defined as QLFROM 65, SPECIFIED*ATOM 2 as QLFROM 66, etc. The SPECIFIED*ATOM expression must be followed by an integer between 1 and 5 or by an expression that computes to such an integer.

CARBON1*1**[QLFROM 70, QLDIST ≠ 4]****CARBON1*2****[QLFROM 71, QLDIST ≠ 4]****CARBON1*3****[QLFROM 72, QLDIST ≠ 4]****HETERO1*1****[QLFROM 73, QLDIST ≠ 4]****HETERO1*2****[QLFROM 74, QLDIST ≠ 4]****HETERO1*3****[QLFROM 75, QLDIST ≠ 4]**

These specifiers are the labels of the atoms in the functional group currently designated as the first group. In LHASA for all the group chemistry tables, a keying functional group is automatically designated as the first group. In a special substructure or pattern table, however, any group may be designated as the first group by using the DESIGNATE command.

The labeling is done by starting at the group origin (labeled CARBON1*1) and growing into the group by successively picking up neighboring atoms. All adjacent atoms joined to the origin by single bonds are picked up first, then double bonds, then triple bonds. If two or more adjacent atoms are joined to the origin by bonds of identical order, they are then picked up in order of decreasing hydrogen count. The growing operation is repeated as often as necessary.

Note that if a transform mechanism changes any atoms or bonds in the first group (designated implicitly or explicitly), it is illegal to refer to any of the atom labels for that group in post-mechanism qualifiers.

CARBON2*1	[QLFROM 76, QLDIST ≠ 4]
CARBON2*2	[QLFROM 77, QLDIST ≠ 4]
CARBON2*3	[QLFROM 78, QLDIST ≠ 4]
HETERO2*1	[QLFROM 79, QLDIST ≠ 4]
HETERO2*2	[QLFROM 80, QLDIST ≠ 4]
HETERO2*3	[QLFROM 81, QLDIST ≠ 4]

These specifiers are the labels of the atoms in the functional group currently designated as the second group. For the two-group chemistry table, a keying functional group is automatically designated as the second group. In any other table, however, any group may be designated as the second group by using the DESIGNATE command.

The second group atom labels are assigned analogously to the first group labels (*vide supra*).

Note that if a transform mechanism changes any atoms or bonds in the second group (designated implicitly or explicitly), it is illegal to refer to any of the atom labels for that group in post-mechanism qualifiers.

LOCANT(S) [QLFROM 82] (synonym IT, THEM)

This keyword, correctly used only with the buzzword PREVIOUS, refers to all the locants that satisfied the target specification in the previous qualifier. For example, in

```

      ADD 25 IF THERE IS A CARBON BETA TO ATOM*1 OFFPATH
      ADD 20 IF ALPHA TO THE PREVIOUS LOCANT IS AN ALCOHOL
or    ADD 20 IF ALPHA TO IT IS AN ALCOHOL

```

the word LOCANT (or IT) refers to all the carbons that are beta to ATOM*1 offpath, as set up in the previous qualifier. Note that the qualifier that sets up the PREVIOUS LOCANT for the next qualifier must contain both a target and a locant. For example,

```

      SAVE AS 1 ALPHA TO ATOM*1 OFFPATH

```

will save the correct locant but will not generate a valid PREVIOUS LOCANT for the next qualifier, since it does not contain any target specifier. When a distance specifier is used with LOCANT, the atoms used for growing out the specified distance will not include any atoms grown over in the previous qualifier. In the first example above, therefore, the ALPHA TO THE PREVIOUS LOCANT specifier will not include any atoms alpha to ATOM 1 offpath since they were grown over previously.

To use the PREVIOUS LOCANT specifier for a previous bond locant the keyword BOND (QLBOND 1) must be explicitly included in the qualifier, e.g.

SAVE AS 1 THE PREVIOUS BOND LOCANT.

POSITION*1	[QLFROM 83]
POSITION*2	[QLFROM 84]
POSITION*3	[QLFROM 85]
POSITION*4	[QLFROM 86]
POSITION*5	[QLFROM 87]
POSITION*6	[QLFROM 88]
POSITION*7	[QLFROM 89]

The POSITION*n locants refer to the atoms in a new ring attached in the current mechanism via

INTRODUCE A PHENYL GROUP ON ATOM*1
or INTRODUCE A RING OF SIZE 5 ON THE NEWEST*ATOM.

After such a new ring introduction, the POSITION*n locants may be used like any other locants. For example, to introduce a new pyrrole ring:

INTRODUCE A RING OF SIZE 5 TO ATOM*1
JOIN POSITION*1 AND POSITION*2
JOIN POSITION*4 AND POSITION*5
CONVERT POSITION*3 TO A NITROGEN.

KEPT*ORIGIN 1, 2, 3	[QLFROM 90,91,92]
HETERO*ORIGIN	[QLFROM 90]

The KEPT*ORIGIN and HETERO*ORIGIN specifiers are used to refer to atoms created in a mechanism but not yet perceived and labeled. Both of these types of atoms are set up by KEEP commands.

In general, any functional group that includes a hetero atom whose valence is not explicitly filled out by the definition of the group will have that hetero atom labeled as the HETERO*ORIGIN after attachment. For example

ATTACH AN AMIDE*1 TO ATOM*1
KEEP AS 2 THE HETERO*ORIGIN
INTRODUCE AN AMIDE*3 ON KEPT*ORIGIN 2

After ATTACH AN AMIDE TO ATOM*1, the nitrogen atom may be referred to with the HETERO*ORIGIN specifier. Complex functional groups (in this case, an N-acyl-N',N'-dimethylurea) can be built by adding further heteroatoms to the kept atom. Note that it is illegal to use HETERO*ORIGIN in any qualifier except a KEEP command and that it is only legal to use KEPT*ORIGIN in ATTACH, JOIN, and INTRODUCE commands.

4.2.2 Bond Locants

SPANNING

[QLFROM 0, QLDIST 4]

The SPANNING bond specifier refers to all bonds spanning a pair of locants, as in

ADD 10 IF THE BOND SPANNING SAVED*ATOM 2 AND &
BOND*3 IS MULTIPLE.

As the above example illustrates, the two "spanning" locants can reference either atoms or bonds, and they must be separated by the keyword AND. Note that it is illegal to use other QLDIST specifiers (e.g. ALPHA, BETA, and GAMMA) as part of the first locant in a SPANNING qualifier.

BOND*1 - BOND*32

[QLFROM 1-32, QLDIST 4]

Each transform or rule has a path of atoms associated with it. The length and nature of the path depends on the executive reading the transform. Path bond numbering is usually consistent with path atom numbering, i.e. ATOM*1 and ATOM*2 span BOND*1, etc. The LHASA Olefin Table is currently the only knowledge-base table where consistent atom/bond path numbering cannot be assumed (see comment at top of file "olftb.src").

For transforms with path length L, only bonds BOND*1 through BOND*L will be defined. Path bond specifiers with numbers higher than the path length must not be referenced. Exceptions to this rule are the subgoal tables FGATB0 and FGITB0, where path change 0 disconnective transforms break BOND*1 and may thus refer to it.

SAVED*BOND(S)

[QLFROM 33-64, QLDIST 4]

This locant specifier references the bond locant(s) saved by a previous SAVE command, as in

SAVE AS 8 THE BONDS ON ALPHA TO ATOM*1 OFFPATH .
.
.
IF SAVED*BOND 8 IS ALLYLIC....

The word SAVED*BOND refers only to bond locants. Saved atoms must be referenced by the SAVED*ATOM specifier. Note that a SAVED*BOND set may contain more than one bond or may in fact be empty. Care must be exercised in this latter case. In the example above, for instance, if there is no atom ALPHA TO ATOM*1 OFFPATH then an error will result when the ALLYLIC qualifier that follows is evaluated.

Up to thirty-two bonds may be saved. The SAVED*BOND expression must be followed by an integer between 1 and 32 or by an expression that computes to such an integer. Note that SAVED*BOND 1 is defined as QLFROM 33, SAVED*BOND 2 as QLFROM 34, etc.

SPECIFIED*BOND(S)**[QLFROM 65-69, QLDIST 4]**

A CHMTRN SUBROUTINE may be called with up to five atom arguments and five bond arguments. The atom arguments are referenced within the called subroutine by the SPECIFIED*ATOM X specifier (see CALL). SPECIFIED*ATOM X is also used to refer to an atom argument within an iterative DO block (see DO). Note that a SPECIFIED*BOND, like a SPECIFIED*ATOM, a SAVED*ATOM, or a SAVED*BOND, is stored as a set and may contain more than one bond.

All specified bonds can be referenced collectively in the called subroutine via the ON*SPECIFIED*PATH and OFF*SPECIFIED*PATH keywords. Note that SPECIFIED*BOND 1 is defined as QLFROM 65, SPECIFIED*BOND 2 as QLFROM 66, etc. As with SPECIFIED*ATOMs, SPECIFIED*BONDs must be referred to by an integer or by an expression that computes to an integer.

BOND1*1**[QLFROM 70, QLDIST 4]****BOND1*2****[QLFROM 71, QLDIST 4]****BOND1*3****[QLFROM 72, QLDIST 4]****BOND1*4****[QLFROM 73, QLDIST 4]**

These specifiers are the labels of the bonds in that functional group currently designated as the first group. In LHASA, for all the group chemistry tables, a keying functional group is automatically designated as the first group. In a ring or pattern table, however, any group may be designated as the first group by using the DESIGNATE command.

The labeling is done by starting at the group origin (labeled CARBON1*1) and growing into the group by successively picking up neighboring atoms.

Once all the atoms have been labeled, the bond labels BOND1*1 through BOND1*4 are assigned. They are assigned labels successively according to the priority scheme shown below which is based on the labeled atoms spanning the bond.

CARBON1*1	-	CARBON1*2	(highest priority)
CARBON1*1	-	HETERO1*1	
HETERO1*1	-	HETERO1*2	
CARBON1*1	-	HETERO1*2	
HETERO1*1	-	CARBON1*2	
CARBON1*2	-	HETERO1*2	
CARBON1*1	-	CARBON1*3	
CARBON1*2	-	CARBON1*3	
HETERO1*1	-	CARBON1*3	
HETERO1*2	-	CARBON1*3	
CARBON1*1	-	HETERO1*3	
CARBON1*2	-	HETERO1*3	
HETERO1*1	-	HETERO1*3	
HETERO1*2	-	HETERO1*3	
CARBON1*3	-	HETERO1*3	(lowest priority)

Note that if a transform mechanism changes any atoms or bonds in the first group (designated implicitly or explicitly), it is illegal to refer to any of the bond labels for that group in post-mechanism qualifiers.

BOND2*1	[QLFROM 74, QLDIST 4]
BOND2*2	[QLFROM 75, QLDIST 4]
BOND2*3	[QLFROM 76, QLDIST 4]
BOND2*4	[QLFROM 77, QLDIST 4]

These specifiers are the labels of the bonds in that functional group currently designated as the second group. In LHASA, for the two-group chemistry table, a keying functional group is automatically designated as the second group. In any other table, however, any group may be designated as the second group by using the DESIGNATE command.

The second group bond labels are assigned analogously to the first group labels (*vide supra*).

Note that if a transform mechanism changes any atoms or bonds in the second group (designated implicitly or explicitly), it is illegal to refer to any of the bond labels for that group in post-mechanism qualifiers.

OFFPATH*ON*ATOM*1	[QLFROM 1, QLDIST 0]
OFFPATH*ON*ATOM*2	[QLFROM 2, QLDIST 0]
OFFPATH*ON*ATOM*3	[QLFROM 3, QLDIST 0]
OFFPATH*ON*ATOM*4	[QLFROM 4, QLDIST 0]
OFFPATH*ON*ATOM*5	[QLFROM 5, QLDIST 0]
OFFPATH*ON*ATOM*6	[QLFROM 6, QLDIST 0]
OFFPATH*ON*ATOM*7	[QLFROM 7, QLDIST 0]
OFFPATH*ON*ATOM*8	[QLFROM 8, QLDIST 0]
OFFPATH*ON*ATOM*9	[QLFROM 9, QLDIST 0]
OFFPATH*ON*ATOM*10	[QLFROM 10, QLDIST 0]
OFFPATH*ON*ATOM*11	[QLFROM 11, QLDIST 0]
OFFPATH*ON*ATOM*12	[QLFROM 12, QLDIST 0]

The OFFPATH*ON*ATOM specifier, although defined as an atom specifier (QLDIST 0), refers to the bonds to ATOM*n that are offpath. For example, the malonic ester synthesis transform (for 1,3 dicarbonyl) breaks the bond OFFPATH*ON*ATOM*2. These locants are typically used when creating the special directory sets.

OTHER*BONDS	[QLFROM 127, QLDIST 0]
--------------------	-------------------------------

There are cases when all the bonds broken by a transform cannot be identified. In these cases the special directory set BROKEN*BONDS (*vide infra*) includes the specifier OTHER*BONDS.

4.3 Additional Locant Specification

The QLQUAL field is used to store additional specifications or "qualifications" that may be applied to a designated locant. For example,

ADD 10 IF ALPHA TO ATOM*2 OFFPATH ONRING &
OFF*THE*BRIDGE IS ENOLIZABLE.

The additional locant specifiers (e.g. those underlined above) place further restrictions on the types of atoms and bonds that satisfy the locant specification of the qualifier. Each of these "restrictive" specifiers is stored as a unique bit in the QLQUAL field, making it possible for any combination of restrictive specifiers to be used in a single qualifier.

In qualifiers where locants alpha, beta, or gamma to an atom are queried, the program "grows out" the required distance from a specified source (QLFROM) atom. QLQUAL restrictions are applied to all atoms and bonds encountered during this growing operation but are not applied to the original source atom. In this regard, it is important that QLQUAL specifiers not be used in the same qualifier with a locant atom unless a distance specifier is also used. The phrases

IF SAVED*ATOM 1 IS OFFPATH...
 and IF SPECIFIED*ATOM 3 IS ON THE ALPHA*FACE...

are illegal and must be written using the comparison keyword SAME or CONTAINED, as in

IF SAVED*ATOM 1 IS THE SAME AS AN OFFPATH ATOM...
 and IF SPECIFIED*ATOM 3 IS CONTAINED IN THE ALPHA*FACE....

A comparison keyword, however, is not necessary if a distance specifier is present. Thus, the qualifiers

IF ALPHA TO SAVED*ATOM 1 IS OFFPATH...
 and IF BETA TO SPECIFIED*ATOM 3 IS ON THE ALPHA*FACE...

will execute properly even though more descriptive wordings of these would be

IF THERE IS AN ATOM ALPHA TO SAVED*ATOM 1 OFFPATH...
 and IF THERE IS AN ATOM BETA TO SPECIFIED*ATOM 3 ON THE ALPHA*FACE....

As the above sample qualifiers illustrate, QLQUAL specifiers are normally used to "modify" QLFROM specifiers. However, qualifiers that contain a QLQUAL specifier but no QLFROM specifier such as

ADD 10 IF THERE IS A KETONE BETA TO THE CURRENT*RING
 or KILL IF THERE IS A FUNCTIONAL GROUP ONPATH

are also legal.

ONPATH	[QLQUAL 1]
OFFPATH	[QLQUAL 2]

These keywords restrict the locants specified by QLFROM and QLDIST to be, respectively, either on or off a designated path. Each transform or rule has a path of atoms associated with it. The length and nature of the path depends on the executive reading the transform. For the special

substructure transforms, the path atoms and bonds are usually those in the ring currently under examination. Note that the order of specifiers in a qualifier that uses ONPATH or OFFPATH to qualify a locant is important.

...ALPHA TO ATOM*3 OFFPATH

is legal, while

...ALPHA OFFPATH TO ATOM*3

is not.

ONRING	[QLQUAL 3]
OFFRING	[QLQUAL 4]

These keywords restrict the locants designated by QLFROM and QLDIST to be, respectively, either on or off a ring of any size. As with ONPATH/OFFPATH, these specifiers must follow the QLFROM and QLDIST keywords if such exist.

ON*THE*BRIDGE	[QLQUAL 5]
OFF*THE*BRIDGE	[QLQUAL 6]

These keywords refer to locants either on or off the bridge mentioned in a previous BRIDGE qualifier, as in

IF ALPHA TO ATOM*1 ON*THE*BRIDGE IS HETERO....

FIRST	[QLQUAL 7] (synonyms INGROUP*1, THIS)
SECOND	[QLQUAL 8] (synonym INGROUP*2)

In the group chemistry tables in LHASA, these specifiers refer, respectively, to the atoms and bonds in the first and second (in the two-group table) keying functional groups. For example,

SAVE AS 3 ALPHA TO ATOM*1 IN THE FIRST GROUP

could be used to save as SAVED*ATOM 3 both oxygen atoms in an acetal or ketal group whose origin is ATOM*1.

For transforms that can be keyed by several functional group types, the FIRST and SECOND specifiers can be used within the body of the transform to distinguish between the possible keying group types, as in

IF THE SECOND GROUP IS AN ALDEHYDE....

They may also be used in mechanism commands, such as

REMOVE THE FIRST GROUP

or CONVERT THE SECOND GROUP TO AN ALCOHOL

and in DESIGNATE commands, for example

DESIGNATE THE GROUP ON ATOM*3 AS THE SECOND GROUP.

When a CHMTRN subroutine is called (see CALL), the functional groups designated as the first and second groups, if any, are stored and then restored after the return from the called subroutine. Any changes made to the first and/or second group in the called subroutine, therefore, are local to that subroutine.

Note that if a transform mechanism changes any atoms or bonds in the first or second group (designated implicitly or explicitly), it is illegal to refer to that group in post-mechanism qualifiers.

FRAGMENT*1	[QLQUAL 9]
FRAGMENT*2	[QLQUAL 10]

These specifiers are used in post-mechanism qualifiers of disconnective transforms to refer to the fragments produced by the disconnection. FRAGMENT*1 refers to the fragment containing ATOM*1 of the path after the mechanism has been performed, and FRAGMENT*2 refers to all other fragments. FRAGMENT*x specifiers are used most often in CONDITIONS statements, as in

CONDITIONS WITTIG AND IN FRAGMENT*2 ORGANOMETALLIC.

They may also be used in a CONTAINED qualifier to find out whether an atom or bond is contained in a particular fragment, as in

IF ATOM*3 IS CONTAINED IN FRAGMENT*1...

Note that since only FRAGMENT*1 remains "in-core" after a DISPLAY THE PRECURSOR command, any reference to atoms in FRAGMENT*2 after a displayed mechanism is likely to cause an error.

ALPHA*FACE	[QLQUAL 11]
BETA*FACE	[QLQUAL 12]

These specifiers allow the locant atoms and bonds on a particular side (face) of a molecule to be referenced. For example,

ADD 25 IF THERE IS A KETONE ALPHA TO ATOM*2 ON THE BETA*FACE

The alpha and beta faces of a molecule are defined in a DESIGNATE command. Only bonds attached to a "reference" ring (a ring whose faces have been previously defined) and only atoms alpha to that reference ring are considered "on" a face. It is, however, allowable to refer to a functional group as being on a particular face even if the origin atom of the group is on the reference ring instead of alpha to it. Thus, the qualifier

IF THERE IS AN ALCOHOL ON ATOM*2 ON THE ALPHA*FACE...

is legal even if ATOM*2 is in the reference ring.

ALPHA*FACE and BETA*FACE may also be used in conjunction with the DEFINED*CIS and DEFINED*TRANS commands to specify stereochemistry in MAKE and EXCHANGE qualifiers, as in

EXCHANGE THE GROUP FOR AN ALCOHOL ON ATOM*5 &
DEFINED*CIS TO THE ALPHA*FACE.

ON*SPECIFIED*PATH	[QLQUAL 13]
OFF*SPECIFIED*PATH	[QLQUAL 14]

Atom and bond arguments mentioned in a subroutine call may be referred to as SPECIFIED*ATOM X and SPECIFIED*BOND X within the called subroutine (see CALL). The keywords ON*SPECIFIED*PATH and OFF*SPECIFIED*PATH are used to refer collectively to all atoms and bonds mentioned in the CALL.

EXPLICIT*HYDROGEN(S)	[QLQUAL 15]
-----------------------------	--------------------

Explicit hydrogen atoms are usually ignored in locant specifications. They may be included by using the EXPLICIT*HYDROGEN keyword, as in

ADD 5 FOR EACH ATOM ALPHA TO ATOM*1 &
INCLUDING EXPLICIT*HYDROGEN.

ON*CURRENT*RING	[QLQUAL 16] (synonyms CURRENT*RING, ON*THE*RING)
OFF*CURRENT*RING	[QLQUAL 17] (synonym OFF*THE*RING)

These specifiers refer to atoms and bonds in the ring or rings currently being operated on by a special substructure table or DESIGNATED as the CURRENT RING in a transform table. For example, in the presently implemented version of the LHASA Diels-Alder transform where the atoms in the six-membered ring are numbered consecutively from ATOM*1 to ATOM*6, the qualifier

IF THERE IS A KETONE ON ALPHA TO ATOM*1 ON*CURRENT*RING...

would examine ATOM*2 and ATOM*6 for ketones.

IN*THE*GROUP	[QLQUAL 18]
---------------------	--------------------

This specifier refers to atoms and bonds in non-keying functional groups that have been mentioned ("set up") in a previous qualifier, e.g.

IF THERE IS NOT AN AMIDE ON ATOM*3 THEN GO TO NOTAMIDE
 IF THERE IS AN OLEFIN ON BETA TO ATOM*3 &
 IN*THE*GROUP THEN DISCONTINUE

NEWEST*ATOM	[QLQUAL 19]
NEWEST*BOND	[QLQUAL 20]

These specifiers are used in mechanism commands only to refer to the most recently introduced atom or bond as in

ATTACH AN OXYGEN ATOM TO ATOM*1
 INTRODUCE A PHENYL TO THE NEWEST*ATOM

or

ATTACH A SULFONE TO CARBON2*2
 ATTACH A PHENYL TO THE NEWEST*ATOM

or

KEEP AS 2 THE NEWEST*ATOM

They are required because the actual locants do not yet exist in the current structure. An additional locant can be used to indicate the most recently introduced atom alpha to - or bond on - an existing atom, as in

JOIN THE NEWEST*ATOM ON AOTM*1 AND SAVED ATOM*1

or

ATOM*3 IS DEFINED ABOVE THE PATH &
 FROM THE NEWEST*ATOM ON ATOM*2 TOWARDS ATOM*1

or

BOND*5 IS DEFINED ANTI TO THE NEWEST*BOND ON ATOM*4

Note that the NEWEST*ATOM/BOND specifiers have the same QLQUAL numbers as INTERFERING/PROTECTED (*vide infra*), but no conflict is possible since these are used only in DESIGNATE qualifiers.

INTERFERING	[QLQUAL 19]
PROTECTED	[QLQUAL 20]
PARTICIPATING	[QLQUAL 21]

These three specifiers are used in conjunction with the DESIGNATE (*vide supra*) command to attach qualities to particular functional groups. For example,

DESIGNATE THE GROUP AS PROTECTED

would ensure that the current group would be displayed to the chemist as needing protection.

Note that the INTERFERING and PROTECTED specifiers have the same QLQUAL numbers as NEWEST*ATOM/BOND (*vide supra*), but no conflict is possible since these are used only in mechanism qualifiers.

REQUESTED**[QLQUAL 22]**

This specifier is used in transforms that may have more than one precursor functional group type. In the LHASA alcohol to halide FGI transform, for instance, the object group type could be fluoride, chloride, bromide, or iodide, depending on which group type was needed by the executive requesting the transform. In transforms of this type, however, the qualifiers cannot refer to any specific object group by name so the REQUESTED specifier is used. For example,

CONVERT THE GROUP TO THE REQUESTED GROUP
or ATTACH THE REQUESTED GROUP TO ATOM*3.

It is possible to test the requested group type, as in

ADD 25 IF THE REQUESTED GROUP IS A BROMIDE.

In fact, the "requested group" is a set of group types, and it is possible for group- and pattern-chemistry transforms to loop (see LOOP) to attach each of the groups in a requested set. It is also possible to use the DESIGNATE specifier to add a group type to the set of requested groups (see DESIGNATE). It should be noted that it is not legal to use logical operators with REQUESTED, as in

IF THE REQUESTED GROUP IS ACID OR: ESTER....

Section 5: OPERATION CODES

The operation code or "action specifier" of a qualifier is contained in the QLOPER field. This field was originally designed to house the ADD, SUBTRACT, and KILL specifiers related to transform evaluation, but its use has been markedly expanded to include specifiers for mechanism commands, subgoal requests, subroutine returns, and other special operations. The QLOPER field may house a number from 1 to 7. Presently, only values 1-4 are in use.

ADD	[QLOPER 1]
SUBTRACT	[QLOPER 2]

These commands are used to modify the rating of a group- or pattern-oriented transform in LHASA, as in

ADD 25 IF ATOM*1 IS ENOLIZABLE.

The value of the rating increment, or decrement, (in this case 25) is stored in the QLINCR field, and is therefore limited to a maximum value of 63. A rating modification is, of course, executed conditionally depending on the truth or falsity of the qualifier containing it. However, when an ADD or SUBTRACT command appears alone on a line, it will be executed unconditionally. (A more expedient way of specifying an unconditional rating modification is available via the INCREMENT and DECREMENT commands which, unlike ADD and SUBTRACT, are not limited to a maximum modification value of 63.)

KILL	[QLOPER 3]
-------------	-------------------

The KILL command is used to terminate the processing of a transform or rule. It is used only when the presence (absence) of the structural or chemical feature mentioned in the KILL qualifier prohibits (is essential to) the successful application of the transform or rule. For example, in a transform corresponding to nucleophilic substitution at saturated carbon, the qualifier

KILL IF ATOM*1 IS A MULTIPLY BONDED ATOM

would be appropriate.

MUST*BE
MUST*NOT*BE
FORBIDDEN

These keywords, in combination with their required locants, allow alternative phrasing of KILL specifiers. For example,

ALPHA TO ATOM*1 MUST*BE BORON.
 ATOM*2 MUST*NOT*BE IN A RING OF SIZE 6.
 BOND*3 IS A DOUBLE*BOND FORBIDDEN.

[QLOPER 4]

The field QLOPER 4 is reserved for a variety of special operations, each of which is distinguished by a unique combination of QLTARG, QLINCR, and/or QLDIST fields. Presently, there are six major categories of special operations:

- 1) Mechanism and subgoal operations,
- 2) Operations requiring intact locant specifiers,
- 3) Operations using an address or table location,
- 4) Operations specifying text,
- 5) Miscellaneous operations,
- 6) Condition statement operators.

Each of the special [QLOPER 4] operators is explained below, along with the additional fields used to define them.

5.1 Mechanism and Subgoal Operations**REMOVE****[QLINCR 1]**

REMOVE is a mechanism command that causes the functional group on a specified locant to be removed. If it is certain that the specified locant contains only one functional group, then a command such as

REMOVE THE GROUP ON ATOM*1

is adequate. However, it is always safer to refer explicitly to the group that is to be removed, as in

REMOVE THE KETONE ON ATOM*1.

Used with the specifiers FIRST and SECOND, the keying functional groups can be removed, as in

REMOVE THE (FIRST) GROUP.

The word FIRST (but not SECOND) may actually be omitted, since an unqualified REMOVE command implicitly refers to the FIRST group.

CONVERT**[QLINCR 1]**

This is a mechanism command that can be used to convert one functional group to another functional group type or to convert an atom to a different atom type. Although the CONVERT command is defined as a synonym to REMOVE, it functions as a composite of the REMOVE and ATTACH commands for functional group conversions. CONVERT causes a keying (FIRST or SECOND) functional group to be removed and replaced by another group. For example, the command

CONVERT THE (FIRST) GROUP TO A KETONE

causes all hetero atoms and hetero bonds in the first group to be removed and replaced by a =O. As with the REMOVE command, the word FIRST is implied and may be omitted.

Unlike the REMOVE command, CONVERT may use only a FIRST or SECOND locant specifier to designate the group being removed. Referring to a group explicitly by specifying an atom or bond locant, as in

CONVERT THE GROUP ON ATOM*3 TO A KETONE

is illegal and would be interpreted as

REMOVE THE KETONE ON ATOM*3.

The CONVERT command may also be used to change atom types, as in

CONVERT POSITION*2 TO BORON.

MAKE	[QLINCR 2]
EXCHANGE	[QLINCR 3]

These commands set up subgoal requests from inside a transform or special substructure table. They request either a functional group interchange (FGI) subgoal (EXCHANGE), or a functional group addition (FGA) subgoal (MAKE). For example,

IF THERE IS A NITRO GROUP ON ATOM*5 THEN &
EXCHANGE THE GROUP FOR AN AMINE*1.

It should be emphasized that MAKE and EXCHANGE are not mechanism commands like REMOVE, CONVERT, etc. Rather than simply manipulating atoms and bonds, these commands request that LHASA search its CHMTRN knowledge base for transforms that can effect the desired conversions. This search for transforms can either succeed or fail, and the result is passed back to the requesting table by a success flag that can be examined with the SUCCESSFUL or UNSUCCESSFUL specifiers (*vide infra*).

Both the MAKE and EXCHANGE commands require the type of group requested to be stated along with the origin at which the exchange or addition is to occur. In cases where any of a number of functional groups sharing a particular characteristic will suffice for the subgoal request, these composite specifiers may be used: CARBONYL, CARBOXYL, DONATING, WITHDRAWING, NON*EXPANDABLE*WITHDRAWING, EXPANDABLE*WITHDRAWING, AMIDE, AMINE, LEAVING, GOOD*LEAVING, HALIDE.

In FGI requests, the functional group to be exchanged is usually mentioned in a previous qualifier, as illustrated above by the nitro to amine EXCHANGE qualifier. Functional groups designated as the FIRST or SECOND group may also be exchanged, as in

EXCHANGE THE FIRST GROUP FOR A KETONE

When a multi-originated group is to be exchanged, care must be taken to mention the proper group origin explicitly. For example, the qualifier

IF THERE IS AN OLEFIN ON ATOM*1 THEN &

EXCHANGE THE GROUP FOR AN ALCOHOL

correctly designates the origin of the C=C at which the exchange is to occur, ATOM*1. A qualifier such as

IF THERE IS AN OLEFIN ON BOND*2 THEN &
EXCHANGE THE GROUP FOR AN ALCOHOL

could put the alcohol on either atom spanning BOND*2.

ATTACH**[QLINCR 4]**

This mechanism command can be used to attach a functional group, atom, ring, or appendage to a specified atom locant or locants. The qualifier

ATTACH AN ACID TO ATOM*3,

for example, causes a =O and an -OH to be attached to ATOM*3. To introduce an entire functional group, including the origin atom (e.g. -COOH) to a locant, the INTRODUCE command (*vide infra*) must be used.

It is also possible to ATTACH a functional group to an atom just inserted into the target molecule in a previous mechanism command:

ATTACH AN ETHER TO THE NEWEST*ATOM
or ATTACH A METHYLENE TO THE NEWEST*ATOM ON ATOM*3.

Certain functional groups may be ATTACHed to two locants simultaneously. In a two-locant ATTACH, the first locant is the primary attachment point (labeled ORG1 in file "fgpats.src") and the second locant is the secondary attachment point (labeled ORG2A or ORG2R in file "fgpats.src"). If a functional group pattern in FGPATS does not have an ORG2x label, the functional group is not appropriate for two-locant attachment. If the second attachment point in the functional group pattern is labeled ORG2A, a bond is made in the mechanism between the second locant and the ORG2A atom of the new functional group. By contrast, if the second attachment point is labeled ORG2R, the second locant becomes the ORG2R atom after the mechanism. For example,

ATTACH AN ESTER TO ATOM*1 AND ATOM*5

puts a doubly bonded oxygen and a singly bonded oxygen on ATOM*1 and attaches the new singly bonded oxygen to ATOM*5, since the second carbon in the ESTER pattern in FGPATS is labeled ORG2R. On the other hand,

ATTACH AN OLEFIN TO ATOM*1 AND ATOM*5

attaches a doubly bonded carbon to ATOM*1 and then attaches this new carbon to ATOM*5 by a new single bond that is not part of the functional group pattern, since the second atom in the OLEFIN pattern is labeled ORG2A.

ATTACH may also be used to attach certain types of atoms (HYDROGEN, OXYGEN, NITROGEN, CARBON, PHOSPHORUS, SULFUR, FLUORINE, CHLORINE, BROMINE, IODINE, SILICON, SELENIUM, COPPER, BORON, ALUMINUM, TIN, and TITANIUM) and certain types of pre-defined appendages (PHENYL, T*BUTYL, and ISOPROPYL) to a locant:

ATTACH A SELENIUM ATOM TO ATOM*2
or ATTACH AN ISOPROPYL GROUP TO POSITION*1.

Note that, as with functional groups, this latter application of ATTACH does not add the carbon "origin" of the appendage, i.e. the above qualifier would add two methyl groups to POSITION 1. To get the full ISOPROPYL appendage, the INTRODUCE command should be used.

Elements, like most functional groups, may be ATTACHed to two locants if so desired. This capability allows a new atom to be positioned equidistant from the two locants, which is not guaranteed by the otherwise equivalent single-locant ATTACH followed by a JOIN to the second locant.

It is also possible to ATTACH a ring to an existing locant, as in

ATTACH A RING OF SIZE 6 TO THE NEWEST*ATOM.

Since the ATTACH is done on an existing atom, only 5 new atoms would be created by the above command. After a ring attachment, the new ring atoms may be referred to in subsequent mechanism commands as POSITION*1 through POSITION*n.

Finally, ATTACH may be used to attach a copy of an appendage existing in the target onto another locant. Two wordings of the ATTACH command are acceptable:

ATTACH A COPY OF THE APPENDAGE FROM x TOWARDS y ON*TO z
or ATTACH A DUPLICATE OF THE APPENDAGE FROM x TOWARDS y ON*TO z,

in which x and y are locants in the target and z may be an existing locant or a newly created one, like THE NEWEST*ATOM or KEPT*ORIGIN 2. Note that it is not legal to make further attachments to a newly copied appendage.

INTRODUCE [QLINCR 5]

This is a mechanism command that can be used to introduce a functional group, ring, or appendage on a specified atom locant. For example, the qualifier

INTRODUCE A KETONE ON ATOM*3

causes a methyl ketone (three atoms) to be introduced on ATOM*3. By contrast, the ATTACH command does not introduce an entire group intact but rather adds the atoms and bonds necessary to convert the locant itself to that functional group type.

It is also possible to INTRODUCE a functional group on an atom just inserted into the target molecule in a previous mechanism command:

INTRODUCE AN ETHER ON THE NEWEST*ATOM
 or INTRODUCE A METHYLENE ON THE NEWEST*ATOM ON ATOM*3.

One can also INTRODUCE a functional group to two locants simultaneously, provided that the group INTRODUCed can bear two attachments. For example, it is possible to

INTRODUCE A KETONE ON ATOM*1 AND ATOM*2

whereas it would be impossible to INTRODUCE an aldehyde to two locants. Details are as described under ATTACH (*vide supra*) except that INTRODUCE creates the ORG1 atom (bonded to the first locant) instead of converting the first locant to the ORG1 atom. It is not legal to INTRODUCE an element to two locants.

It is also possible to INTRODUCE certain selected appendages that have been defined in CHMTRN – PHENYL, T*BUTYL, and ISOPROPYL (see Section 3.1.3):

INTRODUCE A PHENYL GROUP ON ATOM*3
 or INTRODUCE A T*BUTYL GROUP ON POSITION*4

Note that, as with functional groups, INTRODUCE adds appendages with their carbon "origins". In the above examples, six new atoms and four new atoms would be added, respectively. To add only the three methyl groups of a T*BUTYL, one could use an ATTACH command (*vide supra*).

It is also possible to INTRODUCE a ring to a locant, as in

INTRODUCE A RING OF SIZE 6 ON THE NEWEST*ATOM.

The introduction of a ring creates all the ring atoms, drawing a bond from one of them to the specified locant. After a ring introduction, the new ring atoms may be referred to in subsequent mechanism commands as POSITION*1 through POSITION*n.

DESIGNATE	[QLINCR 6]
CLEAR	[QLINCR 6, QLNOT 1]

The most common use of the DESIGNATE specifier is to label the atoms and bonds in a functional group. The group is designated either as the first group or the second group, as in

DESIGNATE THE KETONE ON ALPHA TO ATOM*2 &
 OFFPATH AS THE FIRST GROUP.

If it is not clear that the particular functional group type exists on a locant, the preferred wording of the DESIGNATE qualifier would be:

IF THERE IS A KETONE ON ALPHA TO ATOM*2 &
 OFFPATH THEN DESIGNATE THE GROUP AS THE FIRST GROUP.

This construction makes use of the ability to "remember" a group set up in a previous qualifier (see also EXCHANGE). In fact, multiple functional groups can be designated, using the construction

IF THERE IS A FUNCTIONAL GROUP ON ALPHA TO &
ATOM*2 OFFPATH THEN DESIGNATE THE GROUP AS THE FIRST GROUP.

DESIGNATE for groups should not be used in LHASA in the two-group tables (PAIRTx) where both the first group and the second group have been specified before entering the transform. It can, however, be used to designate a second group in any other group chemistry table, and to designate up to two functional groups in a pattern or special substructure transform. When a group is designated as the first group, the following labels are given to the atoms and bonds in the group: CARBON1*1, CARBON1*2, CARBON1*3, HETERO1*1, HETERO1*2, HETERO1*3, BOND1*1, BOND1*2, BOND1*3, BOND1*4. Second group atoms and bonds are labeled similarly (CARBON2*1, CARBON2*2, etc.).

The CLEAR specifier is used to clear a designated group label, as in

CLEAR THE SECOND GROUP.

When used in conjunction with the keyword INTERFERING, the DESIGNATE command causes a functional group to be designated as reactive and unprotectable toward the reaction conditions. For example, a particular olefin could be explicitly designated as interfering to the conditions of the alpha-diazoketocarbene three-membered ring forming reaction. When the transform is displayed, interfering groups will appear enclosed in dashed boxes.

When DESIGNATE is used with the keyword PROTECTED, as in

DESIGNATE THE GROUP AS PROTECTED

the protected group(s) is displayed enclosed in a solid box.

The keyword PARTICIPATING can also be used with the DESIGNATE or CLEAR command to specify that a functional group participates in a reaction and thus should not be scrutinized for functional group interference by the functional group reactivity module.

Note that the above uses of DESIGNATE should not be employed above the BRANCH or LOOP statement(s) if a transform or long-range table contains one or more such qualifiers. DESIGNATE for functional group interference applications can be used between the lowest-level BRANCH or LOOP qualifier and the end of the transform mechanism (or PERFORM command).

The DESIGNATE and CLEAR commands can also be used to designate the faces of the current molecule as the alpha and beta faces by specifying an atom on one of the faces, e.g.

DESIGNATE THE FACE CONTAINING ATOM*3 AS THE ALPHA*FACE

It is possible to "swap" designated faces using the command

DESIGNATE THE ALPHA*FACE AS THE BETA*FACE.

The DESIGNATE command may also be used in conjunction with the BOW*ATOM specifier to fix a particular conformation of a boat cyclohexane ring, as in

DESIGNATE ATOM*3 AS A BOW*ATOM.

DESIGNATE may be used to prevent certain bonds from being broken in a mechanism, as in

DESIGNATE THE BONDS IN THE FIRST GROUP AS PRESERVED,
or DESIGNATE SAVED*BOND 3 AS A PRESERVED BOND

or to prevent certain stereocenters from being disrupted, as in

DESIGNATE ATOM*1 AS PRESERVED.

CLEAR performs the reverse operations. Note that the keyword BOND(S) is used to distinguish between these two types of PRESERVED qualifiers.

The DESIGNATE command can also be used to set a default functional group type in the requested group set, as in

DESIGNATE BROMIDE AS THE REQUESTED GROUP.

This capability is used in transforms that LOOP on REQUESTED GROUP types (see LOOP for details).

A final use of the DESIGNATE specifier serves to set up a particular ring as the CURRENT*RING. Qualifiers of this type make use of the same conventions for specifying the reference ring as are used in CIS and TRANS qualifiers. Note that the entire range of RING modifiers may also be used, for example

DESIGNATE AS THE CURRENT*RING THE &
CARBOCYCLIC RING OF SIZE 6 CONTAINING BOND*4
or DESIGNATE THE HETEROCYCLIC RING CONTAINING &
BOND*4 AS THE CURRENT*RING.

CATIONIZE	[QLTARG 235, QLINCR 6]
NEUTRALIZE	[QLTARG 235, QLINCR 7]
ANIONIZE	[QLTARG 235, QLINCR 8]
RADICALIZE	[QLTARG 235, QLINCR 9]

These are the mechanism commands used to modify charge. They can be used with an explicit atom locant, as in

CATIONIZE CARBON1*1

or with an atom just added in a previous mechanism command, as in

RADICALIZE THE NEWEST*ATOM ON ATOM*2.

JOIN
SEPARATE

[QLTARG 235, QLINCR 10]
[QLTARG 235, QLINCR 17]

These commands instruct the mechanism handler that two specified atom locants are to be joined or separated, as in

SEPARATE ATOM*1 AND ATOM*2.

When a JOIN command refers to two locants that are already bonded, the order of bonding between them will be increased by one. The SEPARATE command, however, will completely break the bond between the two locants regardless of its order.

It is also possible to join an existing locant to an atom just added in a previous mechanism command. For example, an azirine could be fused to atom 1 and atom 2 as follows:

ATTACH AN IMINE TO ATOM*1
 JOIN ATOM*2 AND THE NEWEST*ATOM ON ATOM*1

DEFINED*AXIAL
DEFINED*EQUATORIAL

[QLTARG 235, QLINCR 11]
[QLTARG 235, QLINCR 12]

These specifiers are used in stand-alone mechanism commands to fix a substituent on a six-membered ring as axial or equatorial, respectively. Use of a DEFINED*AXIAL (EQUATORIAL) specifier overrides the automatic mode of operation of the conformational analysis modules and allows the table writer to fix the conformation of a six-membered-ring target in order to ask further questions about the structure. The bond to be defined as axial or equatorial must be specified as a single CHMTRN specifier, so that the DEFINED*AXIAL(EQUATORIAL) command is packed into the first word of the qualifier. For example,

SAVE AS 1 THE BOND SPANNING ATOM*1 AND ALPHA TO ATOM*1 OFFRING
 SAVED*BOND 1 IS DEFINED*AXIAL

is correct, and a construction like

THE BOND FROM ATOM*1 TOWARDS ALPHA TO ATOM*1 &
 OFFPATH IS DEFINED*AXIAL

is not legal CHMTRN.

INVERT
RETAIN
RACEMIZE

[QLTARG 235, QLINCR 18]
[QLTARG 235, QLINCR 19]
[QLTARG 235, QLINCR 20]

These specifiers are mechanism commands that cause the indicated stereo-operation to be performed on a single, specified atom locant, as in

INVERT AT ATOM*3.

When a bond to a stereocenter is broken during the execution of a mechanism, the default stereo-operation is to racemize the center.

DELETE	[QLTARG 235, QLINCR 21]
BREAK	[QLTARG 235, QLINCR 21]
SINGLE	[QLTARG 235, QLINCR 22]
DOUBLE	[QLTARG 235, QLINCR 27]
TRIPLE	[QLTARG 235, QLINCR 28]
DOUBLE STEREOSPECIFICALLY*CIS	[QLTARG 235, QLINCR 29]
DOUBLE STEREOSPECIFICALLY*TRANS	[QLTARG 235, QLINCR 30]

With the exception of DELETE, these specifiers instruct the mechanism handler to modify a bond locant, as in

```
DOUBLE BOND1*2
or BREAK BOND1*3.
```

The DELETE mechanism command operates on atoms rather than bonds. It instructs the mechanism handler to delete an atom locant along with all of its attached bonds, as in

```
DELETE ATOM*4.
```

Because stereospecific addition across a double bond is a common reaction in organic synthesis, the phrases STEREOSPECIFICALLY*CIS (+QLINCR 2) and STEREOSPECIFICALLY*TRANS (+QLINCR 3) have been added to CHMTRN and may be used in conjunction with a DOUBLE command. For example, the mechanism for a trans bromination of an olefin could be written as follows:

```
BREAK BOND1*1 (a bond to one of the Br's)
BREAK BOND1*2 (a bond to one of the Br's)
DOUBLE BOND*1 STEREOSPECIFICALLY*TRANS
```

This mechanism will cause only one double bond isomer to be generated if the following conditions are met at the time the DOUBLE command is executed:

- 1) the atoms at both ends of the new double bond must have been stereocenters in the target structure;
- 2) one and only one bond must have been removed (broken) on each of these two atoms. The other three bonds must remain unchanged.

If these conditions are not satisfied, then a normal double command (which may result in both olefin isomers) is performed.

It is also possible to use the NEWEST*BOND specifier in conjunction with either the DOUBLE or the TRIPLE command, as in

```
ATTACH A NITROGEN TO ATOM*2
TRIPLE THE NEWEST*BOND.
```

DEFINED*CIS
DEFINED*TRANS

[QLTARG 235, QLINCR 23]
[QLTARG 235, QLINCR 24]

These specifiers are used to define the stereo-relationship about a ring of a newly created stereocenter to a previously existing stereocenter or "face" of the molecule. DEFINED commands may be used as part of an ATTACH, CONVERT, or INTRODUCE mechanism command, as in

INTRODUCE AN ALCOHOL ON ATOM*6 DEFINED*TRANS TO BOND*2
 or ATTACH AN EPOXIDE TO ATOM*1 AND ATOM*2 &
 DEFINED*CIS TO THE ALPHA*FACE

or in an FGI or FGA request, as in

EXCHANGE THE GROUP FOR AN ETHER &
 DEFINED*CIS TO THE GROUP ON ATOM*4

or as a stand-alone in-table mechanism command such as

THE BOND FROM ATOM*1 TOWARDS ALPHA TO ATOM*1 &
 OFFPATH IS DEFINED*CIS TO BOND2*1.

When used in this third manner, the command must be followed by a PERFORM or PERFORM AND DISPLAY command.

Both the new stereolocator, which appears before the DEFINED command, and the reference locator, which appears after the DEFINED command, must define unique bonds and, hence, unique faces of the molecule. The identifying bonds, which must both be exo to a common ring or ring pair, can be defined in one of five ways:

- 1.) by an explicit bond locator, as in

BOND*2 IS DEFINED*TRANS TO....

- 2.) by two ordered atom locants, as in

THE BOND FROM ATOM*1 TOWARDS CARBON1*1....

- 3.) by a single atom locator. In this case, a functional group is assumed to be either on that atom locator or alpha to it. An atom in the group provides the second locator needed to define the bond. For example,

THE GROUP ON ATOM*1 IS DEFINED*CIS TO....

In cases where there is a functional group both on the atom locator and alpha to it, the DEFINED command will apply to the group actually on the atom locator. Functional groups on the locator but contained in the defining ring (cyclic ethers, sulfides, etc.) will be ignored.

- 4.) for the reference locator, by specifying a previously designated face in the molecule, as in

BOND*1 IS DEFINED*TRANS TO THE BETA*FACE.

- 5.) for the reference locant, by using the NEWEST BOND specifier. Often, the DEFINED command must refer to a bond not present in the target structure (e.g. a bond just created in the mechanism by a JOIN command). In such cases, the NEWEST*BOND keyword is used in conjunction with the DEFINED command. For example,

```
JOIN ATOM*1 AND ATOM*5
BOND1*1 IS DEFINED*CIS TO THE NEWEST*BOND ON ATOM*5
```

Without the NEWEST*BOND keyword, it would be impossible to refer to the correct bond. Note that the following wording, which sounds equivalent, is not legal CHMTRN:

```
THE NEWEST*BOND ON ATOM*5 IS DEFINED*CIS TO BOND1*1.
```

The DEFINED mechanism command is always processed after all others. In fact, it is processed after perception on the newly created precursor is complete. This permits the DEFINED specifier to be used even before the ring about which the stereocenters are defined is created, as is commonly the case with reconnection transforms.

DEFINED*SYN	[QLTARG 235, QLINCR 25]
DEFINED*ANTI	[QLTARG 235, QLINCR 26]

These specifiers can be used to control stereorelationships in new carbon-carbon double bonds or diastereorelationships between appendages on new asymmetric centers. The syntax is different in the two applications.

When a carbon-carbon double bond is produced in a precursor structure generated by the LHASA program, both stereoisomers of the double bond will be generated unless a DEFINED*SYN or DEFINED*ANTI command is used to specify the double bond stereochemistry, e.g.

```
      BOND*2 IS DEFINED*SYN TO SAVED*BOND 1
or    BOND*3 IS DEFINED*ANTI TO THE BOND FROM &
      CARBON1*1 TOWARDS ATOM*2
or    SAVED*BOND 2 IS DEFINED*SYN TO THE &
      NEWEST*BOND ON CARBON1*1.
```

Note that the rules for specifying locants for DEFINED*SYN/ANTI commands for double bond stereochemistry are the same as those for DEFINED*CIS/TRANS (*vide supra*).

For each new asymmetric center created in a mechanism, both epimers will be generated in the absence of stereodefinition qualifiers. The DEFINED*SYN and DEFINED*ANTI commands may be used to define new stereochemistry relative to an existing center or to define a diastereomeric relationship between two new appendages on different atoms, e.g.

```
      ATOM*2 IS DEFINED*SYN TO THE NEWEST*BOND &
      ABOUT THE PATH FROM ATOM*1 TOWARDS SPECIFIED*ATOM 1
or    ATTACH AN ALCOHOL TO ATOM*2 DEFINED*SYN TO BOND*4 &
      ABOUT THE PATH FROM ATOM*1 TOWARDS SPECIFIED*ATOM 1
```

As the examples show, locants for this type of DEFINED*SYN/ANTI command can be atoms, bonds, or even full qualifiers. More complete descriptions of the chemical significance of and the allowed syntax for these specifiers can be found under SYN and ANTI (*vide infra*).

DEFINED*ABOVE
DEFINED*BELOW

[QLTARG 235, QLINCR 31]
 [QLTARG 235, QLINCR 32]

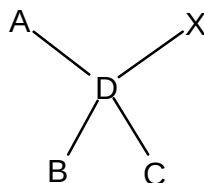
When an sp^3 center is created or modified in a precursor structure generated by the program, both epimers will be generated unless a stereodefinition command is used to specify the stereochemistry before the next PERFORM (either explicit or implicit). DEFINED*ABOVE and DEFINED*BELOW can be used for spatial (i.e. nomenclature independent) definition of new sp^3 centers. These specifiers allow full qualifiers (i.e. both targets and locants) to describe the first atom or bond, e.g.

THE HYDROGEN*ATOM ALPHA TO ATOM*1 IS &
 DEFINED*ABOVE THE PATH FROM POSITION*1 &
 TOWARDS THE NEWEST*BOND ON ATOM*1.

DEFINED*ABOVE and DEFINED*BELOW qualifiers must follow the generalized form:

A IS DEFINED*ABOVE THE PATH FROM B TOWARDS C.

All three locants A, B, and C are attached to a common atom, which will be called D. The molecule is oriented with respect to the plane defined by B, C and D, putting B in the lower left corner, C in the lower-right corner and D in the center as illustrated.



In this orientation, ABOVE is defined by A being in front of or above the plane, and BELOW is defined by A being behind or below the plane.

DEFINED*R
DEFINED*S

[QLTARG 235, QLINCR 33]
 [QLTARG 235, QLINCR 34]

When an sp^3 center is created or modified in a precursor structure generated by the program, both enantiomers will be generated unless a stereodefinition command is used to specify the stereochemistry before the next PERFORM (either explicit or implicit), e.g.

THE CARBON ATOM ALPHA TO CARBON1*1 IS DEFINED*R

DEFINED*R and DEFINED*S both follow the Cahn, Ingold, Prelog rules of stereochemical nomenclature.

KEEP**[QLTARG 235, QLINCR 35-37]**

This specifier is used to store for future reference up to three atoms that have been created in previous mechanism commands but not yet perceived and labeled by LHASA. These atoms may be specified with NEWEST*ATOM, NEWEST*ATOM ON an existing atom, POSITION*n, or HETERO*ORIGIN, as in

```

      KEEP AS 1 THE NEWEST*ATOM ON ATOM*4
or    KEEP AS 3 POSITION*2
or    KEEP THE HETERO*ORIGIN.

```

Note that if no number is explicitly given, the KEPT*ORIGIN number defaults to 1. Use of this capability in combination with the NEWEST*ATOM specifier allows the construction of quite complicated substructures in mechanism commands:

```

ATTACH A CARBON ATOM TO ATOM*1
KEEP THE NEWEST*ATOM
ATTACH A CARBON ATOM TO THE KEPT*ORIGIN
ATTACH A CARBON ATOM TO THE KEPT*ORIGIN
ATTACH AN OXYGEN ATOM TO THE KEPT*ORIGIN
ATTACH A TRIALKYSILYL GROUP TO THE NEWEST*ATOM

```

Note that KEPT*ORIGIN is only allowed in ATTACH, JOIN, and INTRODUCE commands, not with BREAK or SEPARATE.

**HIGHLIGHT
LOWLIGHT****[QLTARG 235, QLINCR 38]
[QLTARG 235, QLINCR 39]**

These specifiers are used to add and remove, respectively, bonds from a set of bonds to be displayed either enclosed in a diamond or distinguished by a solid/dashed line difference (on monochrome terminals) or differently colored (on color terminals). HIGHLIGHT and LOWLIGHT are used principally in the DEREK rulebase, but can be used in LHASA as well for flagging substructures in a precursor. Any bond locant can be used as an argument to either specifier, as in:

```

HIGHLIGHT THE BONDS ONPATH
LOWLIGHT BOND*2.

```

**SCHEDULE
DEFUNCTIONALIZE****[QLTARG 236, QLINCR 1]
[QLTARG 236, QLINCR 2]
(synonym DEFUNCTIONALISE)**

SCHEDULE is used to update the goal list of structural changes identified by the starting-material-oriented-search executive as necessary in the conversion of target to starting material. SCHEDULE is used only in the pattern chemistry strategy table, PCHEMS1. When a command like

```

SCHEDULE THE STRUCTURAL CHANGE REQUIRED ON ATOM*1

```

is issued, the initial rating of the strategy transform is added to the current increment to give a priority for the structural change. The change is then inserted onto the goal list with the calculated priority.

DEFUNCTIONALIZE is a command analogous to MAKE and EXCHANGE that directs LHASA to look for functional group removal (FGR) subgoal transforms to satisfy a request from within a goal transform or special substructure table, as in

IF THERE IS A CHLORIDE ON ATOM*1 THEN DEFUNCTIONALIZE ATOM*1

5.2 Operations Requiring Locant Specifiers

SAVE [QLTARG 237]
SAVE*SEQUENTIALLY [QLTARG 238]

The SAVE command permits a locant, either atom or bond, to be saved for later reference. For example,

SAVE AS 5 CARBON1*3
 or SAVE AS 1 THE BOND IN BOND*6

allows the saved atom(s) or bond(s) to be referred to later as SAVED*ATOM X or SAVED*BOND X respectively. Note that it is very important when saving a bond locant to include the keyword BOND, which turns on the QLBOND bit in the CHMTRN word. Note also that NEWEST*ATOM(BOND) is illegal as a locant in a SAVE command. The SAVE command may be followed by an integer or by an arithmetic expression. Up to thirty-two atoms and thirty-two bonds may be saved. If no number is specified, as in

SAVE THE BOND IN BOND*6,

the number 1 is assumed, and the saved bond will be in SAVED*BOND 1.

It is also possible to save the atoms or bonds of a keying functional group. The commands

SAVE AS 17 THE FIRST GROUP
 and SAVE AS 14 THE BONDS IN THE FIRST GROUP

for example, allow the atoms of the first keying group to be referenced by the SAVED*ATOM(S) 17 specifier and the bonds of the first group to be referenced by the SAVED*BOND(S) 14 specifier, respectively.

The atoms attached to a previously saved bond and the bonds attached to a previously saved atom can also be saved, as in

SAVE AS 4 THE BONDS ATTACHED TO SAVED*ATOM 8
 SAVE AS 22 THE ATOMS ATTACHED TO SAVED*BONDS 9.

When more than one atom or bond satisfies a locant specifier (e.g. THE BONDS ATTACHED TO ATOM*5 or THE ATOMS ALPHA TO ATOM*4), the SAVE*SEQUENTIALLY command can

be used to put a unique SAVED label on each atom or bond. For example, if there were three bonds emanating from ATOM*3, the command

SAVE*SEQUENTIALLY STARTING FROM 6 THE BONDS ATTACHED TO ATOM*3

will label the emanating bonds individually as SAVED*BOND 6 through SAVED*BOND 8.

It is often desirable to narrow down a locant and save only a single atom (or bond) with a certain property. This is conveniently done using the LOCANT specifier with its accompanying buzzword PREVIOUS, as in

IF THERE IS AN OXYGEN ON ALPHA TO ATOM*2 &
OFFPATH THEN SAVE AS 1 THE PREVIOUS LOCANT.

PERMUTE

[QLTARG 239]

This specifier takes as an argument a bond locant that is part of an aromatic system, as in

PERMUTE AT BOND*2
IF UNSUCCESSFUL THEN GO TO BADPERM

The PERMUTE command causes a permutation of the aromatic system to occur such that the formal bond order of the locant bond is altered, either from single to double or from double to single. A check should be made following a permute command to see if the aromatic system in question could successfully be permuted (an attempt to permute tropolone, for example, would be unsuccessful). This command is presently used in the LHASA Diels-Alder mechanism to avoid displaying certain benzyne intermediates as cyclic cumulenes.

CHECK*BOND*PAIR ENTER*BOND*PAIR

**[QLTARG 240, QLINCR 0]
[QLTARG 240, QLINCR 1]**

The use of these specifiers is restricted to the perception table for strategic bond pairs. Each specifier takes two bond locant arguments, the first and second members of the bond pair in question:

CHECK*BOND*PAIR AT SPECIFIED*BOND 1 AND SPECIFIED*BOND 2
IF SUCCESSFUL THEN RETURN

CHECK*BOND*PAIR is used to find out whether the specified bond pair has already been perceived as strategic. As with PERMUTE, CALL, MAKE, and EXCHANGE, a success flag may be tested in the line following the CHECK*BOND*PAIR qualifier. ENTER*BOND*PAIR is used to enter a pair of bonds that has been determined to be strategic into the list of strategic bond pairs in LHASA.

5.3 Operations Using an Address or Table Location

DO [QLTARG 242]
ENDDO [QLTARG 248]

DO is a very powerful specifier that allows blocks of qualifiers to be executed iteratively. The QLADDR field of the CHMTRN word containing DO is loaded with the table address specifying the end of the iterative code. Each DO statement must be part of a FOR EACH statement which indicates the parameter that is to be varied during the iteration, for example,

```
FOR EACH CARBON ALPHA TO ATOM*2 DO CRBALPHA
```

Within the iterative code, the variable parameter is referred to as SPECIFIED*ATOM X (or SPECIFIED*BOND X). Just as with the CALL specifier (*vide infra*), the atoms and bonds presently designated as specified atoms and specified bonds when the iteration begins are stored and then restored after the iteration. Any changes made to these locant parameters within the DO loop, therefore, are in effect only during the iteration.

Iterations may also be performed on selected rings in the target:

```
FOR EACH RING OF SIZE 8 OR LARGER &  
    FUSED*TO*CURRENT*RING DO BIGFUSRING  
or  FOR EACH RING CONTAINING ATOM*3 DO FUSIONLOOP.
```

Any of the ring modifiers except COMMON may be used in this construct. Within a ring DO loop, SPECIFIED*ATOM 1 contains the atoms and SPECIFIED*BOND 1 the bonds in the ring.

ENDDO is used to terminate an iterative DO block. The ENDDO statement bears the address label referenced by the DO statement, as in

```
FOR EACH ... DO DOLABEL  
.  
.  
.  
DOLABEL ENDDO.
```

Note that it is not legal to jump past an ENDDO (for example, via a GO TO statement), without completing the iteration. This is an infraction that the CHMTRN compiler does not test for, so caution should be exercised.

GO (TO) [QLTARG 243]
JUMP (USING) [QLTARG 243, QLCONT 1]

GO is a specifier that transfers control to a different table location when used in conjunction with an address label such as ENDLOC or FIRSTPLACE. as in

```
GO TO FIRSTPLACE.
```


A GO statement is similar to an EXIT statement but differs in that the EXIT statement jumps to one statement past the location referenced by the address label, while the GO statement jumps to the exact location of the address label.

JUMP is a computed GO TO specifier allowing the table writer to jump to one of several locations depending on the value of an index variable, as in

JUMP USING (PROCNO) TO LOCATIONS LOC1 LOC2 LOC3

In the above qualifier, a GO TO LOC1 would be executed for PROCNO = 1, GO TO LOC2 for PROCNO = 2, etc.

Note that it is not legal to use GO TO to jump out of the range of a DO loop without completing the iteration. This is an infraction that the CHMTRN compiler does not test for, so caution should be exercised.

BEGIN	[QLTARG 244]
EXIT	[QLTARG 245]
BLKEND	[no value associated]

These keywords are used for the block structuring of qualifiers. When the keyword BEGIN (followed by an address label such as ENDLOC or FIRSTPLACE) is encountered, the success of the previous qualifier is queried. If the previous qualifier was true, then the block is entered. If it was false, then the address label is examined and processing continues from the site of the label (generally present as a BLKEND LOCX statement).

The EXIT keyword is used to leave a block of code that ends with a BLKEND LOCX statement. An EXIT statement is similar to a GO (TO) statement but differs in that the GO (TO) statement jumps to the exact location referenced by the address label whereas the EXIT statement jumps one table location past the address label.

A sample block structure is shown below.

```

IF BOND*1 IS IN A RING
BEGIN RINGCODE
    ADD 25 IF BOND*1 IS IN A &
        RING OF SIZE 5 THEN EXIT RINGCODE
    KILL
BLKEND RINGCODE

```

(The BLKEND statement is not actually assembled into the tables. It merely serves as a place marker denoting the end of a block.)

CALL	[QLTARG 247]
SUBROUTINE	[no value associated]
RETURN	[QLTARG 249]
RETURN SUCCESS	[QLTARG 249, QLINCR 1]
RETURN FAIL	[QLTARG 249, QLINCR 2]

The keyword **CALL** sets up in-table subroutine calls. The **QLADDR** field is set the address of the location where control is to be transferred, as in

```
CALL CHECKDB
.
.
.
SUBROUTINE CHECKDB
```

in which **CHECKDB** is the starting address of a table subroutine. Table subroutines are declared through the use of the **SUBROUTINE** keyword. Up to five atom and up to five bond calling arguments may also be specified in the **CALL**, as in

```
CALL CHKFG AT ATOM*3 AND ATOM*5
or CALL BONDCK AT ATOM*2 AND BOND*3 AND BOND*4.
```

Atom and bond calling arguments are referred to as **SPECIFIED*ATOM X** and **SPECIFIED*BOND X** (where **X** is an integer between 1 and 5) within the called subroutine.

Before a subroutine call is executed, the atoms and bonds presently referred to as **SPECIFIED*ATOMS 1-5** and **SPECIFIED*BONDS 1-5**, if any, are stored and then restored after returning from the called subroutine. The same is true for functional groups designated as the **FIRST** or **SECOND** group. In this manner, any changes to the aforementioned locant specifiers made within the subroutine will be local to that subroutine.

RETURN is used to exit a called subroutine. The keyword **SUCCESS (QLINCR 1)** or **FAIL (QLINCR 2)** may accompany the **RETURN** statement to indicate whether or not the objective of the subroutine was achieved, as in

```
SUBROUTINE TERTIARY_CHECK
...
...TERTIARY_CHECK checks for tertiary centers. The
...routine is called with one argument, the center for
...evaluation. Successful return if found.
...
IF SPECIFIED*ATOM 1 IS A TERTIARY*CENTER &
    THEN RETURN SUCCESS OTHERWISE RETURN FAIL.
.
.
.
```

As shown in this example, the word **OTHERWISE** can be used effectively with the **RETURN**, **EXIT**, and **GO TO** specifiers, but should not be used with specifiers that do not transfer control to another part of the table.

The result is passed back to the calling transform or subroutine by a success flag that can be examined with the SUCCESSFUL or UNSUCCESSFUL specifiers.

It is also possible to return from a subroutine without affecting the success flag (that is, returning with the current state of the success flag), by using a simple RETURN. This is generally done after a qualifier that sets the success flag (e.g. MAKE or EXCHANGE), as in

```
SUBROUTINE GET_ACID
...
...GET_ACID is used to obtain an acid on the atom
...locant at which it is called.
...
IF THERE IS AN ACID ON SPECIFIED*ATOM 1 &
    THEN RETURN SUCCESS
IF SPECIFIED*ATOM 1 HAS A FUNCTIONAL GROUP
BEGIN EXCHACID
    EXCHANGE THE GROUP FOR AN ACID
    RETURN
BLKEND EXCH*ACID
MAKE AN ACID AT SPECIFIED*ATOM 1
RETURN
```

Thus, RETURN is equivalent to, but shorter than, IF SUCCESSFUL THEN RETURN SUCCESS OTHERWISE RETURN FAIL. Care should be taken not to RETURN while in a DO loop. This is an infraction that the compiler does not test for, so caution should be exercised.

INCREMENT	[QLTARG 250]
DECREMENT	[QLTARG 251]

These commands are used to specify unconditional rating modifications, as in

```
INCREMENT 20.
```

They are similar to the ADD and SUBTRACT commands (QLOPERs 1 and 2), except that the numerical arguments following INCREMENT and DECREMENT are not stored in the QLINCR field. Rather, they are stored in the space normally reserved for the QLDIST, QLFROM, and QLINCR arguments. Thus, rating modifications larger than 64 may be specified. INCREMENTs and DECREMENTs are rounded to the nearest 5 before being added to the rating.

BRANCH	[QLTARG 252]
THEN*TO	[no value associated]

The BRANCH specifier is used to branch sequentially to various table locations, as in

```
BRANCH TO PP1 THEN*TO ELIM THEN*TO DOIT
```

in which PP1, ELIM, and DOIT are table addresses. These addresses must be separated by the word separator THEN*TO. When the BRANCH statement is reached, control is transferred to the first branch address. When a REORIENT/DISCONTINUE or FINISHED (in the long-range search tables) or a KILL or END*TRANSFORM (in the transform-oriented tables) command is encountered during the first branch, the second branch is taken, then the third, etc. At the start of each branch, the structure that was in core at the time the BRANCH statement was first encountered is retrieved. Thus, this facility provides for multiple precursors for a given transform or long-range table.

Note that while most CHMTRN information is stored on entering the first limb of a BRANCH and restored for each successive limb, mechanism commands and CONDITIONS are not saved and must be specified afresh in each limb. See also LOOP and REGENERATE. Additional examples of the use of BRANCH and LOOP are included in the CHMTRN Writers' Guide.

A REJECT command should never be encountered when executing code below a BRANCH statement. Thus, if an impossible situation occurs in a subroutine called from the routine containing the BRANCH, the subroutine must RETURN FAIL and the CALL statement must be followed by an IF UNSUCCESSFUL THEN REJECT.

If the keyword CONDITIONALLY (QLNOT 1) follows BRANCH as in,

BRANCH CONDITIONALLY TO PP1 THEN*TO FGCHK

branch addresses after the first address are executed only if the previous branch code resulted in a REORIENT/DISCONTINUE (long-range tables) or a KILL (transform tables). In other words, conditional branches are terminated whenever a successful sequence (a sequence ending via a FINISHED or an END*TRANSFORM command) is found.

Up to ten different BRANCHes and/or LOOPs may be nested within one another.

5.4 Operations Specifying Text

The keywords specifying text must be assigned a unique number by the table writer. Please refer to the CHMTRN Writers' Guide for the numbers allocated for use. Note that the text specified may be in mixed case, but must not exceed 58 characters in length.

MESSAGE

[QLTARG 254]

This specifier is used in conjunction with the buzzword WRITE to transmit messages to the chemist. Messages are a particularly welcome feature in the long-range search tables, since LHASA often spends considerable time in preprocessor evaluation or in generating subgoal sequences without displaying any structures to the chemist. Again, the message requires a unique number by the table writer, as in

WRITE MESSAGE 3945: Entering Prior Procedure Evaluation.

Note that MESSAGEs should not be used before the BRANCH or LOOP statement(s) in transforms or long-range tables containing these types of qualifiers.

TITLE [QLTARG 255]

This specifier is used in LHASA to assign a transform name to an upcoming subgoal or long-range goal transform. TITLE may also be used to name individual steps of a multi-step mechanism with separate PERFORM and DISPLAY commands. In either case, the format for the TITLE qualifier is

TITLE 3961: Simmons-Smith Cyclopropanation

Note that TITLES should not be used before the branch of loop statement(s) in transforms or long-range tables containing these types of qualifiers.

WARNING [QLTARG 256]

This special operator is used to transmit various warning messages to the chemist. For example, in an S_N2 displacement transform the following qualifiers might appear.

IF THERE IS NOT AN OLEFIN ALPHA TO &
 ATOM*1 THEN GO TO NODB
 WARNING 3871: Equatorial Carboxyl Group
 NODB

In this example, the WARNING qualifier is encountered only if an S_N2' attack is possible. When a WARNING qualifier is encountered, the message it contains is stored and will be transmitted to the chemist only if the transform in which it is contained passes evaluation by LHASA.

Unlike MESSAGES, WARNING numbers for each transform in LHASA are stored on the synthesis tree and thus become part of the permanent record of the retrosynthetic analysis. Note that WARNINGS should not be used before the BRANCH or LOOP statement(s) in transforms or long-range tables containing these types of qualifiers.

ACTUAL*CONDITION(S) [QLTARG 257]

This operator allows the CHMTRN writer to specify actual conditions in a LHASA transform. Actual conditions can be included for each set of prototype conditions. If the set of prototype conditions chosen most favorable (based on minimum interference from non-participating functional groups) has actual conditions associated with it, the actual conditions will be displayed to the chemist in place of the prototype conditions. Only one actual conditions number is stored for each set of prototype conditions, so successive ACTUAL*CONDITIONS statements with no intervening CONDITIONS statement will overwrite previous actual conditions text. ACTUAL*CONDITIONS may be specified in transforms lacking CONDITIONS statements, in which case the most recent set of actual conditions will be displayed.

ACTUAL*CONDITIONS should be short, standard reagents analogous to the CONDITIONS specifiers. Reaction temperatures, reaction times, and solvents should in general not be included unless there is no other reagent:

CONDITIONS PX5

IF THERE ARE TWO FLUORINE ATOMS ALPHA TO &
]ATOM*2 THEN ACTUAL*CONDITIONS 1: SF4
 IF THERE ARE TWO CHLORINE ATOMS ALPHA TO &
 ATOM*2 THEN ACTUAL*CONDITIONS 2: PC15

NEW*HAZARD**[QLTARG 255] (synonym NEW*RISK)**

In DEREK rulebase entries, NEW*HAZARD can be used to replace the HAZARD stated at the top of each rule. NEW*HAZARDS in DEREK can occupy up to two lines. The line break is indicated by //, for example,

NEW*HAZARD 528: Mutagenicity //
 Sensitization

NEW*TOXOPHORE**[QLTARG 257]**

In DEREK rulebase entries, NEW*TOXOPHORE can be used to replace the TOXOPHORE stated at the top of each rule. NEW*TOXOPHORES in DEREK can occupy up to two lines. The line break is indicated by //, for example,

NEW*TOXOPHORE 801: Organic Isocyanate

5.5 Miscellaneous Operations**BIT****[QLTARG 253]****SPECIAL*SET****[QLTARG 253, QLINCR 1]****(synonym SUBCLASSIFICATION)****TOGGLE BIT****[QLTARG 253, QLINCR 2]****TURN ON BIT****[QLTARG 253, QLINCR 3]****TURN OFF BIT****[QLTARG 253, QLINCR 3, QLNOT 1]**

There is a 64 bit set available to the table writer to do with as s/he wishes. The keyword BIT is used to turn bits ON and OFF in this set, as in

IF SUCCESSFUL THEN TURN ON BIT 39
 or IF BIT 26 IS OFF THEN RETURN SUCCESS.
 or IF ATOM*1 IS ALLYLIC THEN TURN ON THE NEXT BIT.
 or IF THE NEXT BIT IS OFF THEN GO TO BITOFF.

As with FLAGS (*vide infra*), one can toggle the value of a particular bit:

TOGGLE BIT 15.

The keyword SPECIAL*SET is used to determine whether the entire set is empty or not, e.g.

IF THE SPECIAL*SET IS EMPTY....

The SUBCLASSIFICATION synonym for SPECIAL*SET is used exclusively in the LHASA SBTAB table for functional group reactivity subclassification. Note that the buzzword EMPTY should not be used in other types of CHMTRN qualifiers, as it can lead to incorrect results through misinterpretation.

PERFORM	[QLTARG 261]
DISPLAY	[QLTARG 261, QLINCR 1]
REGENERATE	[QLTARG 261, QLINCR 2]
REDRAW	[QLTARG 261, QLINCR 4]
GOAL	[QLINCR = QLINCR + 5]

The PERFORM specifier indicates that an in-table mechanism has been set up and is to be performed by the LHASA mechanism handler, as in

PERFORM THE MECHANISM.

PERFORM causes a new structure to be created but not displayed. Successive PERFORMs without intervening displays cause the new structure to replace the structures created in previous performs. To display and retain the new structure, a DISPLAY command is required, as in

PERFORM THE MECHANISM AND DISPLAY THE PRECURSOR.

In a LHASA transform-oriented table, a structure created with a PERFORM is also displayed by the end of the transform or by an END*TRANSFORM command. When the mechanism corresponding to the goal of a special substructure table is to be executed, the keyword GOAL must be added to the command, as in

PERFORM AND DISPLAY THE GOAL MECHANISM
or PERFORM THE GOAL MECHANISM
or DISPLAY THE GOAL PRECURSOR.

The keyword GOAL causes the value of QLINCR to be incremented by 5, so the specifier DISPLAY THE GOAL PRECURSOR is packed as QLINCR 6.

The REGENERATE command is used to bring back into core the last parent structure that had been displayed. If several PERFORM commands are given to generate successive nodes without any of those nodes being displayed, the REGENERATE command will bring back into core the parent of the node generated by the first PERFORM command.

Unstable or highly reactive intermediates (enols, aromatic tautomers, etc.) may be generated using the PERFORM command and then examined via CHMTRN. However, the LHASA program will not allow structures containing such unstable features to be displayed.

The REDRAW specifier mimics the use of the REDRAW button on the Precursor Display from within a transform, as in

PERFORM THE MECHANISM AND REDRAW THE PRECURSOR.

For LHASA transforms in which there is no explicit PERFORM command, the REDRAW THE PRECURSOR command should be the first command after the set of four dots (...) terminating the mechanism. The REDRAW feature should not be used indiscriminately, as it tends to make the chemistry hard to follow by reorienting acyclic precursors. For simple cyclic precursors, especially those resulting from rearrangement or reconnection transforms, REDRAW can be quite useful.

END*TRANSFORM [QLTARG 262, QLINCR 0]
END*RULE [QLTARG 262, QLINCR 0]

In LHASA, when an END*TRANSFORM is executed in the group-chemistry or pattern-chemistry tables, processing of the transform is stopped immediately and, on the basis of the current rating value, the transform is either displayed or killed. The END*RULE specifier provides equivalent functionality for a DEREK rule. These specifiers eliminate the need to explicitly jump to the end of the transform or rule to avoid a block of qualifiers. If these specifiers are used within the scope of a BRANCH or LOOP, the next iteration is performed.

FINISHED [QLTARG 262, QLINCR 1]

The FINISHED specifier denotes the end of a block of BRANCH or LOOP code in a special substructure table. This differs from the group- and pattern-chemistry case primarily in that no display is implied by the end of a BRANCH or LOOP in a special substructure table. In these tables, a PERFORM and/or DISPLAY must be explicitly requested.

DISCONTINUE [QLTARG 262, QLINCR 2](synonym **REORIENT**)

This command, used only by the special substructure transforms, causes processing to be halted on the current ring procedure and/or ring orientation and causes the table to be reentered at the beginning with a new procedure and/or orientation. If used within the scope of a BRANCH or LOOP, the next iteration is performed after any nodes generated in the BRANCH or LOOP iteration leading to the DISCONTINUE have been pruned.

REJECT [QLTARG 262, QLINCR 3]

This command, which is issued only from a special substructure transform, indicates that the current substructure is to be completely rejected from further consideration. No other orientations of the current substructure will be examined. For example,

IF THERE IS A HETERO ATOM IN THE CURRENT*RING THEN REJECT.

RENUMBER [QLTARG 262, QLINCR 5]

RENUMBER is used in a special substructure table to reorient a cyclic substructure according to the renumbering definitions at the top of the table without clearing all the LHASA information that REORIENT normally clears when renumbering a ring. RENUMBER is currently used only in the Quinone-Diels-Alder search table.

SUCCESSFUL	[QLTARG 263]
UNSUCCESSFUL	[QLTARG 263, QLNOT 1]

These specifiers ascertain whether a previously mentioned subgoal request, subroutine call, or PERMUTE was successful. For example,

```

CALL CHECKFG AT ATOM*3
IF UNSUCCESSFUL THEN GO TO BADCHECK
or  EXCHANGE THE GROUP FOR A KETONE
    IF SUCCESSFUL THEN RETURN SUCCESS.

```

FLAG	[QLTARG 264]
TOGGLE FLAG	[QLTARG 264, QLINCR 2]
TURN ON FLAG	[QLTARG 264, QLINCR 3]
TURN OFF FLAG	[QLTARG 264, QLINCR 3, QLNOT 1]

As their names imply, these keywords refer to a set of flags that are accessible to the table writer to do with as s/he wishes. Each flag is global, and thus may be set in one subroutine and tested in another. Up to 64 flags may be used simultaneously.

A flag is referenced by an alphanumeric “name”, corresponding to a unique number packed by the CHMTRN compiler into the QLNUMB field of the qualifier:

```

TURN ON FLAG ALPHA_CH2
or  IF FLAG ALREADY_DONE IS OFF THEN....

```

Note that additional operations (ADD, SUBTRACT, KILL, etc.) can be used in a FLAG qualifier but must be preceded by THEN, as in

```
IF FLAG ALPHA_CH2 IS OFF THEN ADD 25.
```

The “value” of a FLAG may be inverted, i.e. ON changed to OFF or vice versa, using the TOGGLE specifier, as in

```
TOGGLE FLAG UNCHANGED.
```

LOOP	[QLTARG 266]
UP*TO	
DOWN*TO	
EQUALING	
CONDITIONALLY*AFTER	[QLTARG 267]

LOOP allows iterative execution of large blocks of code, each time with a different value of a “loop parameter.” When a LOOP statement is encountered, one element from the loop parameter is chosen and execution continues until a FINISHED or DISCONTINUE/REORIENT (in the LHASA long-range tables); or an END*TRANSFORM, END*RULE or KILL (in the other tables) is

reached. The table is then re-entered at the loop statement and processing continues with the next element of the loop parameter. For each re-entry, as with returns to successive BRANCHes, the status (values of flags, sets, variables, saved and specified atoms, etc.) is restored to its condition prior to the first entry. Thus, values of the items saved cannot be passed from one arm of a BRANCH or iteration of a LOOP to another.

There are four types of loop parameters:

1. An atom or bond loop parameter. The set of atoms (or bonds) to be looped on is established by a normal comparison of the target and locant specifiers in the LOOP qualifier, e.g.

```

      LOOP ON THE ENOLIZABLE ATOMS BETA TO THE CURRENT*RING
or    LOOP ON THE BONDS ALPHA TO ATOM*3 OFFPATH
or    LOOP ON OLEFIN BONDS
or    LOOP ON THE NEXT ATOM IN CARBOXYLSET [ SETNUMBER ]

```

The LOOP command selects members of the resultant set, one at a time. Within the loop, the atom (or bond) parameter is referred to as SPECIFIED*ATOM 1 (or SPECIFIED*BOND 1).

2. A path parameter. The "path loop" is used to grow and label connected paths of atoms and bonds. All paths of a specified length are grown from a starting atom locant, as in

```

      LOOP ON PATHS OF FIVE ATOMS STARTING FROM ATOM*3
or    LOOP ON PATHS OF ( VAR2 ) BONDS STARTING FROM ATOM*1

```

The loop is performed once for each path found, with the CHMTRN labels ATOM*1 – ATOM*n and BOND*1 – BOND*(n-1) assigned to the atoms and bonds along the grown path.

In addition to growing paths of a specified length, LOOP may be used to grow the shortest paths from one locant to another, as in

```

      LOOP ON THE SHORTEST PATHS FROM ATOM*2 &
      TOWARDS SAVED*ATOM*3

```

When the second (destination) locant of a "shortest path" loop itself specifies a set of connected atoms or bonds (e.g. CURRENT*RING), the path can be grown from the first locant up to and AROUND the second locant, as in

```

      LOOP ON THE SHORTEST PATHS OF SIX ATOMS &
      STARTING FROM SAVED*ATOM*1 AROUND THE CURRENT*RING.

```

If desired, the atoms over which the paths will be grown can be restricted by a USING ONLY phrase, as in

```

      LOOP ON PATHS OF FOUR ATOMS FROM ATOM*1 &
      USING ONLY THE ATOMS IN GOODAT.

```

3. A requested group parameter. This capability is used in LHASA only in group- and pattern-chemistry goal transforms, where the executive requesting the transform may or may not have filled the set of requested group types before entering the transform. To prevent an error when the LOOP qualifier is read, a DESIGNATE command is used to establish a default group type if the requested set was previously empty:

```
IF THERE IS NOT A REQUESTED GROUP THEN &
  DESIGNATE BROMIDE AS THE REQUESTED GROUP
LOOP ON REQUESTED GROUPS.
```

The LOOP command iterates on successive members of the requested group set. Within the loop, the current group type is referred to as the REQUESTED GROUP.

To protect the transform from situations in which an executive has sent in a set containing an inappropriate requested group type, it is necessary to check within the loop that each value of the loop parameter is a group type that the current transform can generate, e.g.

```
IF THE REQUESTED GROUP IS NOT CHLORIDE AND:IF &
  THE REQUESTED GROUP IS NOT BROMIDE AND:IF &
  THE REQUESTED GROUP IS NOT IODIDE THEN KILL
```

The LOOP statement and the lines following it are executed $N + 1$ times for N items to be looped on. Thus, an IF UNSUCCESSFUL statement must follow each LOOP statement, as in

```
LOOP ON ACETYLENE BONDS
IF UNSUCCESSFUL THEN DISCONTINUE.
```

4. An integer parameter. Progressive values of an integer variable may be looped upon, e.g.

```
LOOP ON VAR1 VALUES EQUALING FROM (-3) UP*TO (10)
or LOOP ON BVAR VALUES EQUALING FROM (VAR2 - 3) &
  DOWN*TO (VAR4 + 1)
```

The endpoint variable values may be any integer or any variable expression that computes to an integer.

This type of “variable loop” may be continued conditionally after a certain value is reached, as in

```
LOOP ON ATMVAR VALUES EQUALING FROM (30) DOWN*TO (1) &
  CONDITIONALLY* AFTER (15)
```

The loop is continued past the conditional value only if no successful sequences were generated prior to that value (e.g. if prior variable values all lead to sequences ending in REORIENT/DISCONTINUE).

ZERO VARIABLE*ARRAY [QLTARG 273, QLINCR 1]

These specifiers are used to explicitly zero sets in an array of sets and variables, as in

ZERO 20 WORDS IN THE SET*ARRAY STARTING AT TEMP [2]
 ZERO 10 WORDS IN THE VARIABLE*ARRAY STARTING AT TEMP(1)

**SORT ASCENDING [QLTARG 274]
 SORT DESCENDING [QLTARG 274, QLINCR 1]**

These special operators sort the elements of arrays in either ASCENDING or DESCENDING order, as in

SORT IN ASCENDING ORDER 4 ELEMENTS OF THE &
 ARRAYS AT ATM(20) AND ATM(10).

The above command will parallel sort the two arrays starting at ATM(20) and ATM(10) so that the array at ATM(20) is in ascending order and each associated number in the array ATM(10) is moved correspondingly, e.g.

Before Sort		After Sort	
ATM(20) = 10	ATM(10) = 3	ATM(20) = 1	ATM(10) = 5
ATM(21) = 4	ATM(11) = 25	ATM(21) = 3	ATM(11) = 16
ATM(22) = 1	ATM(12) = 5	ATM(22) = 4	ATM(12) = 25
ATM(23) = 3	ATM(13) = 16	ATM(23) = 10	ATM(13) = 3

During evaluation, if the arrays overlap, then an error message is given and no sorting is performed. Any number of arrays can be listed in the SORT command. All will be sorted in order of the first array listed.

WRITE*STRUCTURE [QLTARG 275]

WRITE*STRUCTURE instructs LHASA or DEREK to write a copy of the current “in-core” structure to the structure dump file for debugging purposes. This qualifier is rarely used.

NO*OP [QLTARG 276, QLINCR 0]

NO*OP is the null CHMTRN specifier, whose function is to occupy a CHMTRN word without resulting in any CHMTRN operations. NO*OP can be used to allocate several adjacent locations for manual addition of code for debugging purposes. This qualifier has fallen into disuse.

TIME [QLTARG 277]

TIME is a debugging specifier used to write entries for elapsed time and CPU time into the LHASA or DEREK debug log file. TIME is used with an index that is also written into the log as an identifier to locate the TIME call in the CHMTRN code. For example,

TIME 2

would make a timer entry labeled "Time 2." Note that no entry will be made in the log unless the TIMER debug option has been selected graphically by the user from the DEBUG menu while running LHASA or DEREK.

SYN	[QLTARG 278]
NOT*SYN	[QLTARG 278, QLNOT 1]
ANTI	[QLTARG 279]
NOT*ANTI	[QLTARG 279, QLNOT 1]

These specifiers can be used to ask about stereorelationships between appendages on carbon-carbon double bonds or about diastereoisomerism on rings or acyclic chains. The syntax is different in the two applications. A qualifier of the form

IF ATOM*2 IS ANTI TO BOND*5...

is used for olefin stereochemistry. Two locants are required, one on each side of the SYN or ANTI specifier. Note that the negative specifiers NOT*SYN and NOT*ANTI must be used to ask whether a specific stereorelationship does not exist. The clause NOT ANTI(SYN) is invalid.

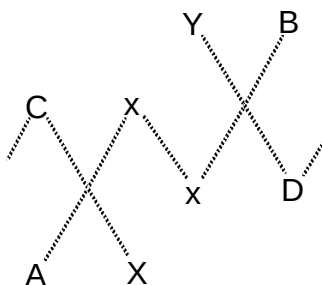
A qualifier of the form

IF THE HYDROGEN*ATOM ALPHA TO ATOM*2 IS &
SYN TO BOND*4 ABOUT THE PATH FROM ATOM*1 &
TOWARDS SPECIFIED*ATOM 1...

can be used to determine the diastereomeric relationship between two appendages. The meanings of these specifiers in this context follow the definitions of SYN/ANTI proposed by Masamune and coworkers (S. Masamune, Sk.A. Ali, D.L. Snitman and D.S. Garvey, *Angew. Chem. Int. Ed.*, 19, 557 (1980)).

Qualifiers using SYN/ANTI to ask about diastereorelationships must be of the form:

IF A IS SYN TO B ABOUT THE PATH FROM C TOWARDS D...



A and C are attached to a common atom, as are B and D. The portion of the molecule defined by the path from C towards D through these common atoms is drawn in an extended (zig-zag) fashion.

The relationship between A and B is now determined by the plane of this extended chain. In the SYN isomer, A and B are on the same side of the plane, while in the ANTI isomer, they are on opposite sides. Since the terms are conventional, it does not matter if the molecule can actually adopt this configuration.

Note that A, B, C, and D in the above generalized qualifier can be full qualifiers, i.e. each may include both a target and a locant. Also note that it is not legal to have an operation, e.g. KILL, as part of A, B, C, or D. One must say IF ... THEN KILL.

In order to define olefin stereochemistry or diastereomeric relationships between stereocenter appendages in a mechanism, the DEFINED*SYN(ANTI) specifiers may be used.

ABOVE	[QLTARG 281]
NOT*ABOVE	[QLTARG 281, QLNOT 1]
BELOW	[QLTARG 282]
NOT*BELOW	[QLTARG 282, QLNOT 1]

These specifiers give the CHMTRN writer spatial (i.e. nomenclature independent) information about sp^3 centers. ABOVE and BELOW allow both targets and locants in describing all atoms or bonds, e.g.

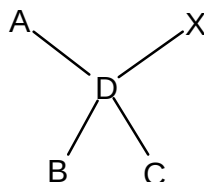
IF THE HYDROGEN*ATOM ALPHA TO ATOM*1 &
 IS ABOVE THE PATH FROM POSITION*1 &
 TOWARDS THE NEWEST*BOND ON ATOM*1....

Note that the negative specifiers NOT*ABOVE and NOT*BELOW must be used to find out whether specific spatial arrangements do not exist. The clause NOT ABOVE(BELOW) is invalid.

Qualifiers using ABOVE and BELOW must be of the form:

IF A IS ABOVE THE PATH FROM B TOWARDS C....

A, B, and C can each be a full qualifier including a target and a locant, but must all be attached to a common atom, D. The molecule is oriented with respect to the plane defined by B, C and D, putting B in the lower left corner, C in the lower right corner and D in the center as illustrated.



In this orientation, ABOVE is defined by A being in front of or above the plane, and BELOW is defined by A being behind or below the plane.

It is important to note that it is not legal to have an operation, e.g. KILL, as part of A, B, or C. One must say IF ... THEN KILL.

ENANTIOSELECTIVE*SYNTHESIS	[QLTARG 283]
DIASTEREOSELECTIVE*SYNTHESIS	[QLTARG 283, QLINCR 1]

These specifiers give the CHMTRN writer the ability to inquire about the stereochemical mode of processing. When the chemist selects STEREOSELECTIVE from the LHASA PROCESS menu, s/he is given a choice of ENANTIO- or DIASTEREOSELECTIVE methodology. This selection can be queried from within the transform as follows:

```

IF ENANTIOSELECTIVE*SYNTHESIS THEN KILL
IF NOT DIASTEREOSELECTIVE*SYNTHESIS THEN &
    GO TO NOSTEREO
... Stereochemical qualifiers
NOSTEREO &
... Qualifiers common to all processing modes

```

LHASA	[QLTARG 288, QLINCR 1]
DEREK	[QLTARG 288, QLINCR 2]
PROTECT	[QLTARG 288, QLINCR 3]
APSO	[QLTARG 288, QLINCR 4]

These keywords allow CHMTRN qualifiers to be executed conditionally, dependent upon the currently executing program. They are intended primarily for use in the student version of LHASA. For example,

```

IF APSO THEN &
    WARNING 3361: Alcohol must be converted to a better leaving group

```

5.6 Condition Statement Operators

CONDITION(S)	[QLTARG 290]
FOLLOWED	[QLINCR 1]

In LHASA condition statements appear in almost every group- and pattern-chemistry transform. These statements specify the prototype reagent(s) necessary to bring about the synthetic reaction corresponding to the transform under consideration. The CONDITIONS keyword marks the beginning of a condition statement qualifier. The actual prototype reaction conditions (see list below) are packed into the QLTARG fields of the succeeding words.

In a disconnective transform (i.e. one that retrosynthetically produces two or more fragments), certain reagents contained in the condition statement apply to only one of the reacting fragments. In such instances, the keywords FRAGMENT*1 and FRAGMENT*2 precede those reagents. For example,

```

CONDITIONS WITTIG AND IN FRAGMENT*1 Ph3P FOLLOWED BY RLi

```

specifies that the Ph₃P and RLi conditions apply only to fragment 1 while the WITTIG condition applies to all reacting fragments.

Alternative conditions for a transformation are listed as separate **CONDITIONS** statements. LHASA chooses the "best" set of conditions based on the functional group interference it perceives in the non-participating functional groups in the precursor.

It is also possible to specify an **ACTUAL*CONDITIONS** string for each set of prototype conditions (see **ACTUAL*CONDITIONS**). If there is an **ACTUAL*CONDITIONS** statement for the chosen set of prototype conditions, the **ACTUAL*CONDITIONS** string will be displayed to the chemist in place of the prototype conditions.

Like most other qualifiers, **CONDITIONS** statements can be made conditional on the result of a previous qualifier, as in

IF THERE IS AN ACID ON ATOM*1 THEN **CONDITIONS** LDA.

The **QLTARG** values associated with the individual reagents are listed below.

Reagent	Field
pH<1/100	[QLTARG 1]
pH<1	[QLTARG 2]
pH:1	[QLTARG 3]
pH2:4	[QLTARG 4]
pH4:6	[QLTARG 5]
pH6:8	[QLTARG 6]
pH9:10	[QLTARG 7]
pH10:12	[QLTARG 8]
pH>12	[QLTARG 9]
pH>12/150	[QLTARG 10]
NaH <i>or</i> Sodium*Hydride	[QLTARG 11]
Ph ₃ CNa <i>or</i> [C ₆ H ₅] ₃ CNa	[QLTARG 12]
Na*Naphthalide	[QLTARG 13]
NaH/DMSO <i>or</i> NaH/CH ₃ SOCH ₃	[QLTARG 14]
KotBu <i>or</i> tButoxide	[QLTARG 15]
LDA <i>or</i> LiNR ₂ <i>or</i> LiN[iPr] ₂	[QLTARG 16]
Pyridine <i>or</i> Pyr	[QLTARG 17]
R ₃ N <i>or</i> NR ₃	[QLTARG 18]
NaNH ₂	[QLTARG 19]
NaOR	[QLTARG 20]
Enolate	[QLTARG 21]
Ph ₃ P <i>or</i> R ₃ P	[QLTARG 22]
TsNHNH ₂	[QLTARG 23]
RNH ₂ <i>or</i> NH ₃	[QLTARG 24]

Reagent	Field
RNHOH <i>or</i> H2NOH <i>or</i> NH2OH	[QLTARG 25]
RNHNH2	[QLTARG 26]
NH2NH2	[QLTARG 27]
RSH <i>or</i> NaSH	[QLTARG 28]
NaN3	[QLTARG 29]
NaSCN	[QLTARG 30]
NaI <i>or</i> LiI	[QLTARG 31]
NaCN	[QLTARG 32]
Acetylide	[QLTARG 33]
NaCl	[QLTARG 34]
NaBr	[QLTARG 35]
HCN <i>or</i> HCN/ca*CN	[QLTARG 36]
ROH	[QLTARG 37]
NaOAc	[QLTARG 38]
RLi <i>or</i> ORGANOLITHIUM	[QLTARG 39]
RMgX <i>or</i> ORGANOMAGNESIUM	[QLTARG 40]
RZnX <i>or</i> ORGANOZINC	[QLTARG 41]
R2Cd <i>or</i> ORGANOCADMIUM	[QLTARG 42]
RCu <i>or</i> ORGANOCOPPER	[QLTARG 43]
R2CuLi <i>or</i> ORGANOCUPRATE	[QLTARG 44]
WITTIG <i>or</i> Phos*ylide	[QLTARG 45]
Sulf*ylide <i>or</i> Sulph*ylide	[QLTARG 46]
Oxosulf*ylide <i>or</i> Oxosulph*ylide	[QLTARG 47]
H2Ni	[QLTARG 48]
H2/Pt/HOAc	[QLTARG 49]
H2/Pd	[QLTARG 50]
H2/Lindlar	[QLTARG 51]
H2/Rh	[QLTARG 52]
Zn/HCl	[QLTARG 53]
Zn/HOAc	[QLTARG 54]
Sn/Cl2/HCl	[QLTARG 55]
Cr[II]/pH:5	[QLTARG 56]
Na/NH3	[QLTARG 57]
Al[Hg]	[QLTARG 58]
Na[Hg]	[QLTARG 59]
SnCl2/Pyr	[QLTARG 60]

Reagent	Field
NaHSO ₃	[QLTARG 61]
H ₂ S	[QLTARG 62]
LiAlH ₄ <i>or</i> LAH	[QLTARG 63]
KR ₃ BH/MINUS*50	[QLTARG 64]
Selectride	[QLTARG 65]
R ₂ BH <i>or</i> B ₂ H ₆ <i>or</i> Diborane	[QLTARG 66]
NaBH ₄	[QLTARG 67]
Zn[BH ₄]	[QLTARG 68]
NaCNBH ₃ /HMPA/60	[QLTARG 69]
Dibal <i>or</i> R ₂ AlH	[QLTARG 70]
Li[tBuO] ₃ AlH	[QLTARG 71]
AlCl ₃ /80	[QLTARG 72]
AlCl ₃ /25	[QLTARG 73]
SnCl ₄ /25	[QLTARG 74]
Et ₂ O:BF ₃ <i>or</i> BF ₃ :Et ₂ O	[QLTARG 75]
LiClO ₄ <i>or</i> Mg[II]	[QLTARG 76]
TsOH/80	[QLTARG 77]
TsOH/0	[QLTARG 78]
Hg[OAc] ₂	[QLTARG 79]
Hg[OOCCF ₃] ₂	[QLTARG 80]
HgCl ₂	[QLTARG 81]
HgO	[QLTARG 82]
Ag[I]	[QLTARG 83]
Cu[II]/Pyridine <i>or</i> Cu[II]/Pyr	[QLTARG 84]
HBr/Radical	[QLTARG 85]
HX/Radical	[QLTARG 86]
NBS/Radical	[QLTARG 87]
BrCCl ₃ /Radical <i>or</i> CCl ₄ /Radical	[QLTARG 88]
OsO ₄	[QLTARG 89]
KMnO ₄	[QLTARG 90]
Ozone/MINUS*50	[QLTARG 91]
Peracid/0 <i>or</i> RCO ₃ H/0	[QLTARG 92]
Peracid/50 <i>or</i> RCO ₃ H/50	[QLTARG 93]
CrO ₃ /Pyridine <i>or</i> CrO ₃ /Pyr	[QLTARG 94]
H ₂ Cr ₂ O ₇ <i>or</i> CrO ₃ /pH:1	[QLTARG 95]
Peroxide/alk <i>or</i> HOOH/alk	[QLTARG 96]

Reagent	Field
Quinone	[QLTARG 97]
Singlet*Oxygen	[QLTARG 98]
DMSO/100	[QLTARG 99]
NaOCl/pH:10	[QLTARG 100]
NBS/HOH <i>or</i> NBS/dioxane	[QLTARG 101]
I2 <i>or</i> I2/Oxaziridine	[QLTARG 102]
PhSCl <i>or</i> C6H5SCl	[QLTARG 103]
PhSeCl <i>or</i> C6H5SeCl	[QLTARG 104]
Cl2	[QLTARG 105]
Br2	[QLTARG 106]
XCN	[QLTARG 107]
MnO2/CH2Cl2	[QLTARG 108]
NaIO4/pH5:8 <i>or</i> HIO4	[QLTARG 109]
SeO2/pH2:4	[QLTARG 110]
SeO2/Pyridine <i>or</i> SeO2/Pyr	[QLTARG 111]
K3FeCN6/pH7:10	[QLTARG 112]
Pb[IV]/25	[QLTARG 113]
Pb[IV]/80	[QLTARG 114]
Pyrolysis/150	[QLTARG 115]
Pyrolysis/250	[QLTARG 116]
Pyrolysis/350	[QLTARG 117]
:CCl2 <i>or</i> Dichlorocarbene	[QLTARG 118]
N2CHCOOEt/Cu/80	[QLTARG 119]
CH2I2/ZnCu	[QLTARG 120]
nBu3SnH <i>or</i> R3SnH	[QLTARG 121]
Ni[CO]4 <i>or</i> Ni[CO]4/Pd[0]L4	[QLTARG 122]
CH2N2	[QLTARG 123]
SOCl2	[QLTARG 124]
PX5	[QLTARG 125]
PX3	[QLTARG 126]
Ac2O/TsCl/25 <i>or</i> AcOAc/TsCl/25	[QLTARG 127]
Ac2O/100 <i>or</i> AcOAc/100	[QLTARG 128]
P2O5	[QLTARG 129]
DCCD	[QLTARG 130]
CH3I	[QLTARG 131]
R3OBF4 <i>or</i> [CH3]3OBF4 <i>or</i> [Me]3OBF4	[QLTARG 132]

Reagent	Field
LDA/CH ₃ I	[QLTARG 133]
K ₂ CO ₃ /CH ₃ I	[QLTARG 134]
Tl[NO ₃] ₃ <i>or</i> Tl[III]	[QLTARG 135]
MILDLY*ELECTROPHILIC	[QLTARG 136]
ELECTROPHILIC	[QLTARG 137]
STRONGLY*ELECTROPHILIC	[QLTARG 138]

Section 6: CONDITIONAL AND LOGICAL QUALIFIERS

The QLTHEN and QLLOGC fields allow qualifiers to be combined logically and/or executed conditionally.

6.1 Conditional Execution of Qualifiers**THEN****[QLTHEN 1] (synonym AND*THEN)**

The execution of any CHMTRN qualifier can be made conditional on the truth or falsity of a previous qualifier by using the keyword THEN (and/or AND*THEN), as in

```

      IF ATOM*2 IS ENOLIZABLE THEN GO TO A2ENOL
or    IF BOND*6 IS IN A RING THEN ADD 30 IF BOND*3 IS AROMATIC
or    IF CARBON1*3 IS A STEREOCENTER THEN &
      TURN ON FLAG YESFLAG AND*THEN GO TO YESTEREO

```

When such a qualifier is false, all THEN and AND*THEN qualifiers immediately following the false qualifier are skipped.

6.2 Logical Combination of Qualifiers

The words defined by QLLOGC values allow separate qualifiers and/or portions of qualifiers to be combined.

OR:**[QLLOGC 1]****AND:****[QLLOGC 2]**

The specifiers allow either multiple locants or multiple targets to be mentioned in a standard target/locant qualifier, e.g.

```

      ADD 25 IF THERE IS AN ENOLIZABLE ATOM ALPHA TO ATOM*3 OR &
      BETA TO ATOM*1 OFFPATH OR GAMMA TO ATOM*2 ONRING
or    KILL IF ATOM*6 IS ALLYLIC AND: A BRIDGEHEAD
or    IF ATOM*1 HAS AN OLEFIN AND: ETHER THEN GO TO ENOLETHER

```

The (target or locant) specifier that is not varied must appear in the first portion of the qualifier. Thus, the ATOM*1 qualifier above would be illegal if worded

```

      IF THERE IS AN OLEFIN AND: ETHER ON ATOM*1 THEN GO TO ENOLETHER

```

While many OR: and AND: combination qualifiers are unambiguous, a multiple locant AND: qualifier such as

```

      ADD 15 IF THERE IS AN ALLYLIC ATOM ALPHA TO &
      ATOM*3 AND: BETA TO SAVED*ATOM 4 OFFPATH

```

can easily be misinterpreted and should be avoided.

The four possible types of multiple target/locant OR: and AND: qualifiers are interpreted as follows (T, T1, and T2 are targets; L, L1, and L2 are locants):

- | | | | |
|-------------|---------|----------------|--------------------|
| 1. IF T ON | L1 OR: | L2 as IF T ON | L1 OR:IF T ON L2 |
| 2. IF T ON | L1 AND: | L2 as IF T ON | L1 AND:IF T ON L2 |
| 3. IF L HAS | T1 OR: | T2 as IF L HAS | T1 OR:IF L HAS T2 |
| 4. IF L HAS | T1 AND: | T2 as IF L HAS | T1 AND:IF L HAS T2 |

In case 2 above, QLOLDC is assigned as the logical OR of the two individual resultant sets of atoms(bonds) if the qualifier is true. If the qualifier is false (either of the resultant sets is 0), QLOLDC is set to 0.

Care must be exercised when using OR: and AND: to find functional groups. A qualifier like

IF ATOM*1 HAS AN ALCOHOL OR: ETHER OR: &
ESTERX THEN DESIGNATE THE GROUP AS THE FIRST GROUP

will function properly if there is in fact one of the desired group types on the specified locant. However, if the qualifier

IF ATOM*3 HAS AN ETHER AND: OLEFIN...

fails because of the absence of only one of the desired group types. The group that is “remembered” for future DESIGNATE or EXCHANGE qualifiers will not be the group that is present on ATOM*3, but rather whatever group it had in memory before it processed the current qualifier.

The logical comparisons in OR: and AND: qualifiers are normally made on atoms. Bonds will be used, however, if the non-varying (target or locant) specifier explicitly refers to a bond, as in

ADD 25 IF THERE IS AN OLEFIN ON ALPHA &
TO SAVED*ATOM 2 OR: BETA TO SAVED*ATOM 4

or IF BOND*3 IS ALLYLIC OR: ATTACHED TO A BRIDGEHEAD....

Bonds will also be used for logical comparisons if every locant of a multiple locant qualifier explicitly refers to a bond, e.g.

IF THERE IS A FUSION ON BOND*2 OR: ANY BOND &
BETA TO ATOM*3 OR: ANY BOND ALPHA TO ATOM*1 OFFPATH....

The NOT specifier negates the entire combination qualifier, e.g.

KILL IF THERE IS NOT AN OLEFIN ON ATOM*2 OR: &
ALPHA TO ATOM*4 OFFPATH

Individual targets in a multiple target qualifier should be negated by the NON specifier (QLNOTT 1), as in

KILL IF ALPHA TO ATOM*7 IS ALLYLIC OR: NON ENOLIZABLE

Individual locants in a multiple locant qualifier should be negated by the NOT*ON or NOT*LOCATED ON specifiers (QLNOTL 1), as in

KILL IF THERE IS AN OLEFIN ON ATOM*2 OR: &
ON ANY ATOM NOT*ON BETA TO ATOM*5

Note that CHMTRN does not provide for combinations of logical qualifiers and counting operations. For example:

IF BETA TO ATOM*1 HAS TWO ALCOHOLS OR: ETHERS...

is NOT a legal CHMTRN question. Instead, one must ask:

IF BETA TO ATOM*1 HAS TWO ALCOHOLS OR:IF &
BETA TO ATOM*1 HAS TWO ETHERS....

Also, logical specifiers should not be used in APPENDAGE or WITHDRAWING BOND qualifiers.

OR:IF	[QLLOGC 3] (synonym AND:FOR)
AND:IF	[QLLOGC 4]

These specifiers allow complete qualifiers to be combined, as in

KILL IF ATOM*2 IS ENOLIZABLE AND:IF THERE IS &
A LEAVING GROUP ON SPECIFIED*ATOM 1
or IF BIT 2 IS ON OR:IF BETA TO ATOM*6 IS ARYL &
OR:IF THERE IS NOT A KETONE ON ATOM*1 THEN ADD 25

When several FOR EACH qualifiers are combined, the specifier AND:FOR should be used to improve readability, as in

ADD 15 FOR EACH ENOLIZABLE ATOM BETA TO ATOM*5 AND:FOR &
EACH ALLYLIC ATOM ALPHA TO ATOM*1 OFFPATH

The CHMTRN interpreter allows OR:/AND: combination qualifiers to appear within OR:IF/AND:IF qualifiers, as in

KILL IF FLAG PREKILL IS ON AND:IF THERE IS &
AN EPOXIDE ON BOND*3 OR: SAVED*BOND 2 &
AND:IF ALPHA TO ATOM*1 HAS AN OLEFIN AND: ETHER

It is not legal, however, to use both AND:IF and OR:IF in the same qualifier. Note also that it is not advisable to use THE GROUP after an AND:IF or OR:IF qualifier, since a particular group is not "remembered" very well in these situations (see descriptions of EXCHANGE and DESIGNATE). Note further that it is not legal to use HYDROGEN, METHYL, or ALKYL in compound FOR EACH/AND:FOR qualifiers.

Section 7: CHMTRN ARITHMETIC

CHMTRN, just like most other computer programming languages, supports a wide variety of arithmetic expressions. CHMTRN expressions may be used with a large number of CHMTRN qualifiers. The various arithmetic specifiers and keywords, together with their usage, are documented in this section.

7.1 Numerical Values

ONE	[QLNUMB 1]
TWO	[QLNUMB 2]
THREE	[QLNUMB 3]
FOUR	[QLNUMB 4]
FIVE	[QLNUMB 5]
SIX	[QLNUMB 6]
SEVEN	[QLNUMB 7]
EIGHT	[QLNUMB 8]
NINE	[QLNUMB 9]
TEN	[QLNUMB 10]
ELEVEN	[QLNUMB 11]
TWELVE	[QLNUMB 12]
THIRTEEN	[QLNUMB 13]
FOURTEEN	[QLNUMB 14]
FIFTEEN	[QLNUMB 15]
SIXTEEN	[QLNUMB 16]
SEVENTEEN	[QLNUMB 17]
EIGHTEEN	[QLNUMB 18]
NINETEEN	[QLNUMB 19]
TWENTY	[QLNUMB 20]

These keywords explicitly specify the value of the QLNUMB field. They are used most commonly to express the numerical arguments in counting qualifiers, such as

KILL IF THERE ARE NOT TWO HYDROGENS ON ATOM*1.

7.2 Arithmetic Declarations

Any word previously undefined in CHMTRN can be declared to refer to user-defined storage. There are two declaration types: sets (which may contain collections of atoms or bonds) and variables (which may contain positive or negative integers). Arrays of either of these types may also be declared. Identifiers for user-defined sets or variables may contain the characters A to Z, 0 to 9 and . A typical declaration is show below:

```
SETS: GOOD_FUSIONS, CURRENT_FUSION, START_BONDS, &
      ANTI_MARKOV, FGSET, TO_BE_DELETED, RXN_ATOMS, TMP_SET
SET*ARRAYS: FIRST_BOND(12), REST_BONDS(12), PATH_BONDS(13)
VARIABLES: PATHLENGTH, INDEX, PATH_TERM, DIFELC, NHYDRO, &
           MINACT, RNGACT, VAR1, VAR2, FG_ID_CODE, KINETIC_FLAG
```


SET(S):	[QLTARG 401-464]
SET*ARRAY(S):	[QLTARG 401-464]
VARIABLE(S):	
VARIABLE*ARRAY(S):	

The above keywords, if present, must be specified at the top of a CHMTRN source file. The keyword is followed by one or more identifiers separated by commas and may be continued on more than one line. Array sizes are indicated by enclosing a numerical constant in parenthesis. [User-defined set QLTARGs are assigned in the order of the set declaration statements in the table. If the declarations above were the first ones in a table, for example, GOOD_FUSIONS would be QLTARG 401, CURRENT_FUSION QLTARG 402, etc.]

EQUIVALENCE

A maximum of 64 sets and arrays of sets may be declared. Occasionally, a table will require more than this maximum. Since it is unlikely that such a large number of sets would be in use at any one time, however, provision has been made to reuse set locations via the EQUIVALENCE specifier:

EQUIVALENCE HETBNDS AS ENON*ALPHA.

In this example, HETBNDS is defined by the EQUIVALENCE statement and does not need a previous SET: declaration statement. HETBNDS and ENON ALPHA are identified by the same QLTARG values, and may be used in the table, but not simultaneously.

7.3 Set Expressions

Individual members of SET*ARRAYS are accessed by specifying the subscript in square brackets []. For example,

FUSION_BONDS [3]

accesses the 3rd set in the set array FUSION_BONDS.

ZERO	
GETS	[QLTARG 401-464]
PUT:	[QLTARG 401-464]
REMOVE:	[QLTARG 401-464, QLNOT 1]

Atoms and bonds are placed into and taken out of the 64 user-defined sets by the GETS, PUT:, and REMOVE: commands, for example

PUT: ALPHA TO ATOM*1 INTO BADATS [1]
or REMOVE: BOND*3 FROM GOODBD [3].

An entire set can be cleared using a ZERO with the GETS command, as in

BETABDS GETS ZERO.

GETS can be used to fill a named set with the contents of another named set or of a legitimate target set (e.g. ENOLIZABLE ATOMS). The PUT: and REMOVE: commands operate on named sets but take locants, not targets, as arguments. Note that complex specifiers as arguments to these commands frequently lead to trouble. For example,

MEDRNG GETS ATOMS IN RINGS OF SIZE 4 THROUGH 9

is illegal.

ALSO:	[QLFROM 1]
PLUS:	[QLFROM 2]
MINUS:	[QLFROM 3]

The special operators ALSO:, PLUS:, and MINUS are used with the GETS command to perform, respectively, logical AND, inclusive OR, and AND complement operations on a set.

GOODAT GETS ENOLIZABLE ATOMS WHICH ARE ALSO: ALLYLIC

The GETS operator has been defined to allow any number of logical operations to be performed in a single CHMTRN statement. For example,

**ATMSET GETS ATMSET MINUS: BADATS PLUS: &
CARBONS WHICH ARE ALSO: IN THE &
BONDSTOATOMS OF MULTIPLE BONDS.**

ATOMS*TO*ATOMS	[QLDIST 1]
ATOMS*TO*BONDS	[QLDIST 2]
BONDS*TO*ATOMS	[QLDIST 3]
BONDS*TO*BONDS	[QLDIST 4]

The special operators ATOMS*TO*ATOMS, ATOMS*TO*BONDS, BONDS*TO*ATOMS, and BONDS*TO*BONDS can also be used to operate on a set within a GETS command, as in

GOODAT GETS GOODAT MINUS: THE ATOMS*TO*ATOMS OF NITROGENS
or **SET 3 GETS MULTIPLE BONDS PLUS: THE &
ATOMS*TO*BONDS OF HETERO ATOMS.**

The ATOMS*TO*ATOMS operator returns all the atoms alpha to the atoms in the specified target set excluding any originally in that set. The ATOMS*TO*BONDS operator replaces all of the atoms in the specified target set with their adjacent bonds. The BONDS*TO*ATOMS and BOND*TO*BONDS operators function analogously.

7.4 Variable Expressions

CHMTRN variable expression syntax has changed considerably as the language has evolved. The storage in the binary output file has also undergone various transformations. The current syntax is modeled fairly closely to the Fortran computer programming language. The field [QLTARG 270] is used for the main arithmetic operators.

Arithmetic operations of addition (+), subtraction (-), multiplication (*), division (/), and modulus (%) are defined. Parentheses () may be used to specify evaluation order. Both variables (BDINDEX, ATMKNT, etc.) and constants (1, 2,...) can appear expressions. For example

```
REMAINDER = 7 % 3
TOTAL = VAR1 + VAR2
AVG = (CNT1 + CNT2 + CNT3) / 3
```

Individual array members are accessed by specifying the subscript in parentheses. Variables may also be used in computed array indices. For example

```
ATMARR(4)
ATMARR(ATMKNT + 5)
ATMARR(ATMKNT)
ATMARR(ATMARR(2) + 5)
```

The indexing of variables is very flexible, allowing any operation normally used in arithmetic expressions. Beware of using variables with a fractional part as array indices; they will be truncated (not rounded off) before the array lookup.

```
.EQ.
.NE.
.GT.
.GE.
.LT.
.LE.
```

The six relational operators of Fortran shown above are defined in CHMTRN and may be included in arithmetic expressions. For example

```
IF (ONENUM .NE. -4) THEN ...
or IF (NUTHERNUM + 6 .LT. THIRDNUM) THEN ...
```

```
.OR.
.AND.
```

These logical operators may be included also within CHMTRN expressions. For example

```
IF (VAR1 .EQ. 2 .OR. VAR1 .EQ. 3) THEN ...
or IF (CNT1 .EQ. 3 .AND. AVG .GT. 4) THEN ...
```

Additionally these two operators may be used to perform bit-wise masking operations. For example

```
IF ((MASKVAR .AND. 16) .NE. 0) THEN....
```

A number of CHMTRN specifiers contain an isolated non-zero integer as part of their definition. Currently, these are SAVED*ATOM X, SAVED*BOND X, SPECIFIED*ATOM X, SPECIFIED*BOND X, SAVE AS X, TURN ON BIT X, TURN OFF BIT X, IF BIT X IS ON (OFF), SET X, ADD X, SUBTRACT X, INCREMENT X, and DECREMENT X, where X is an integer. In all of these specifiers the integer X may be replaced by any arithmetic expression that computes to a non-zero integer. For example,

```
IF SAVED*ATOM ((ATMKNT * 4) / 3 ) IS ENOLIZABLE THEN GO TO SAVENOL
or  TURN ON BIT ( BITKNT - BNDKNT - 1 )
or  ADD ( RATING ) IF SPECIFIED*BOND ( KOUNT ) IS IN BADBND [ VAR2 + 1 ]
```

It is also legal to use an expression in a counting operation, as in

```
KILL IF FEWER THAN ( ATMKNT ) HYDROGENS ON ATOM*5.
```

An example of variable use in CHMTRN is given in the six lines below, which reverse the order of ten elements of an array.

```
INDEX = 1
SORT10  TEMP = ATMARR(INDEX)
        ATMARR(INDEX) = ATMARR(11 - INDEX)
        ATMARR(11 - INDEX) = TEMP
        INDEX = INDEX + 1
        IF (INDEX .LE. 5) THEN GO TO SORT10
```

7.5 Calculated Values

A number of properties may be calculated and then stored in a CHMTRN variable.

NUMBER [QLINCR 31]

The keyword NUMBER can be used to count and store in a CHMTRN variable the number of atoms or bonds (but not HYDROGENs, METHYLs, or ALKYLs) referred to in a normal qualifier, e.g.

```
OXATKNT = THE NUMBER OF OXYGEN ATOMS ALPHA TO ATOM*1.
```

LOWER*DESTABILIZATION*ENERGY [QLINCR 32]

LOWER*DESTABILIZATION*ENERGY allows the chemist to extract into a CHMTRN variable the destabilization energy of the lower energy conformation of the six-membered ring currently designated (see DESIGNATE and CURRENT*RING), as in

ENERGY1 = THE LOWER*DESTABILIZATION*ENERGY OF THE CURRENT*RING.

FG*TYPE**[QLINCR 33]**

The keyword FG*TYPE can be used to store in a CHMTRN variable the internal LHASA index for the functional group type on a particular locant, as in

FGON2 = THE FG*TYPE OF THE GROUP on ATOM*2.

HOMO*COEFFICIENT	[QLINCR 34]
LUMO*COEFFICIENT	[QLINCR 35]
ELECTRON*DENSITY	[QLINCR 36]
FREE*VALENCY	[QLINCR 37]
ELECTROPHILIC*LOCALIZATION*ENERGY	[QLINCR 38]
NUCLEOPHILIC*LOCALIZATION*ENERGY	[QLINCR 39]
RADICAL*LOCALIZATION*ENERGY	[QLINCR 40]
HOMO*ENERGY	[QLINCR 41]
LUMO*ENERGY	[QLINCR 42]
DIFF*ELECTROPHILIC*REACTIVITY	[QLINCR 43]
REL*ELECTROPHILIC*REACTIVITY	[QLINCR 44]
REL*NUCLEOPHILIC*REACTIVITY	[QLINCR 45]
ABS*ELECTROPHILIC*REACTIVITY	[QLINCR 46]

These specifiers can be used to extract Huckel parameters into CHMTRN variables for use in determining preferred sites of reaction in aromatic substitutions, as in

LUMO10 = THE LUMO*COEFFICIENT AT SAVED*ATOM 10.

INDEX

- , 107
- %, 107
- &, 24
- (), 107
- *, 107
- ..., 25
-, 25
- .AND., 107
- .EQ., 107
- .GE., 107
- .GT., 107
- .LE., 107
- .LT., 107
- .NE., 107
- .OR., 107
- /, 107
- :CCl₂, 99
- @@@@@, 10, 11
- [], 105
- [C₆H₅]₃CNa, 96
- [CH₃]₃OBF₄, 99
- [Me]₃OBF₄, 99
- +, 107
- 1-D patterns, 7, 11, 14
- 1D*DEFAULTS, 15
- 1D*PATTERN, 14
- 2-D patterns, 7, 14, 20, 25
- 2D*PATTERN, 14
- 3*OR*MORE*RINGS, 39
- A, 7
- About this Manual, 5
- ABOVE, 9, 94
- ABS*ELECTROPHILIC*REACTIVITY, 109
- Ac₂O/100, 99
- Ac₂O/TsCl/25, 99
- ACETAL, 28
- ACETYLENE, 28
- Acetylide, 97
- ACID, 27
- ACID*HALIDE, 27
- AcOAc/100, 99
- AcOAc/TsCl/25, 99
- ACROSS1, 22
- ACROSS2, 22
- ACROSS3, 22
- ACTUAL*CONDITION(S), 85
- ADD, 64
- Additional Locant Specification, 57
- Ag[I], 98
- Al[Hg], 97
- AlCl₃/25, 98
- AlCl₃/80, 98
- ALCOHOL, 28
- ALDEHYDE, 27
- ALKYL(S), 36
- ALKYL*ATOM(S), 35
- ALL, 45
- ALLENE, 28
- ALLYLIC, 47
- ALPHA, 50
- ALPHA*FACE, 60
- ALSO, 7
- ALSO:, 106
- ALUMINUM, 34
- AMIDE, 30
- AMIDE*1, 27
- AMIDE*2, 27
- AMIDE*3, 27
- AMIDZ, 28
- AMINE, 30
- AMINE*1, 27
- AMINE*2, 27
- AMINE*3, 27
- AN, 7
- AND, 26, 31, 45, 50, 55, 106
- AND*THEN, 101
- AND:, 45, 101, 102
- AND:FOR, 103
- AND:IF, 45, 103
- ANHYDRIDE, 27
- ANIONIZE, 71
- ANOTHER, 7
- ANTI, 9, 93
- ANY, 7
- ANY*GROUP, 48
- ANYWHERE, 7
- APPENDAGE, 45
- APPENDAGE(S), 45
- APSO, 21, 95
- ARE, 7
- Arithmetic Declarations, 104
- arithmetic expressions, 104
- Arithmetic operations, 107
- AROMATIC, 38
- AROMATIC*ATOM, 22
- Arrays, 104
- ARYL, 38
- AS, 7

- AT, 7
- ATOM, 7
- Atom and Bond Specification, 51
- Atom and Bond Specifiers, 33
- Atom Locants, 51
- ATOM*1 - ATOM*32, 51
- ATOM*NUMBERING, 24
- ATOMS*TO*ATOMS, 106
- ATOMS*TO*BONDS, 106
- ATTACH, 67
- ATTACHED, 7
- AWAY, 7
- AXIAL, 38
- AZIDE, 28
- AZIRIDINE, 27
- AZO, 27

- B2H6, 98
- BAD, 12
- BE, 7
- BEARING, 7
- BEGIN, 81
- BELOW, 9, 94
- BENZYLIC, 47
- BETA, 50
- BETA*FACE, 60
- BETTER, 7
- BF3:Et2O, 98
- binary output file, 7, 9, 13, 107
- BIT, 86
- bit-wise masking, 108
- BLKEND, 81
- Bond Locants, 55
- BOND*1, 17
- BOND*1 - BOND*32, 55
- BOND*NUMBERING, 24
- BOND1*1, 17, 56
- BOND1*2, 56
- BOND1*3, 56
- BOND1*4, 56
- BOND2*1, 57
- BOND2*2, 57
- BOND2*3, 57
- BOND2*4, 57
- BONDED, 7
- BONDS*TO*ATOMS, 106
- BONDS*TO*BONDS, 106
- BORANE*FOR*RL, 22
- BORON, 34
- BOW*ATOM, 38
- Br2, 99
- BRANCH, 83
- BrCCl3/Radical, 98
- BREAK, 73
- BREDT*STRAINED, 39
- BRIDGE, 9
- BRIDGE(D), 42
- BRIDGE*1, 42
- BRIDGE*2, 42
- BRIDGE*3, 42

- BRIDGED*TO*CURRENT*RIN, 41
- BRIDGED*TO*CURRENT*RING, 40
- BRIDGEHEAD(S), 38
- BROKEN*BOND(S), 17
- BROMIDE, 28
- BROMINE, 34
- buzzwords, 7
- BY, 7

- C*SULFONATE, 27
- C*SULPHONATE, 27
- C6H5SCl, 99
- C6H5SeCl, 99
- Calculated Values, 108
- CALL, 82
- CAN, 7
- CARBAMATE*C, 28
- CARBAMATE*H, 28
- CARBOCYCLIC, 40, 41
- CARBON(S), 34
- CARBON1*1, 52
- CARBON1*2, 52
- CARBON1*3, 52
- CARBON2*1, 53
- CARBON2*2, 53
- CARBON2*3, 53
- CARBONIUM, 27
- CARBONYL, 29
- CARBOXYL, 30
- CARCINOGENICITY, 23
- CATIONIZE, 71
- CCl4/Radical, 98
- CENTER, 7
- CENTRE, 7
- CH2I2/ZnCu, 99
- CH2N2, 99
- CH3I, 99
- CHANGE, 7
- CHARACTER, 7
- CHARGE, 7
- CHECK*BOND*PAIR, 79
- CHLORIDE, 28
- CHLORINE, 34
- CHMTRN arithmetic, 104
- CHMTRN qualifiers, 7
- CIS, 9, 36
- CIS*BRIDGE(D), 42
- CIS*FUSION, 37
- CIS*OLEFIN, 37
- CIS1, 22
- CIS2, 22
- CIS3, 22
- Cl2, 99
- CLEAR, 69, 71
- CODE, 7
- COLLABORATIVE, 23
- comment, 25
- COMMON, 40, 41
- Comparison Atom and Bond Specifiers, 43
- Condition Statement Operators, 95

CONDITION(S), 95
 CONDITION*FLEXIBILITY, 12, 14
 CONDITIONAL AND LOGICAL QUALIFIERS, 101
 Conditional Execution of Qualifiers, 101
 CONDITIONALLY*AFTER, 89
 CONJUGATED, 47
 constants, 107
 CONTAIN, 7
 CONTAINED, 9, 44
 CONTAINING, 7
 continuation, 24
 CONVERT, 65
 COPPER, 34
 COPY, 7
 COPYRIGHT*LUK, 21, 22
 CORE, 39
 CORROSIVITY, 23
 Cr[II]/pH:5, 97
 CREATES*STEREO, 20
 CrO₃/pH:1, 98
 CrO₃/Pyr, 98
 CrO₃/Pyridine, 98
 Cu[II]/Pyr, 98
 Cu[II]/Pyridine, 98
 CUPRATE*FOR*RC, 22
 CUPRATE*FOR*RT, 22
 CURRENT*RING, 41, 61
 CYANO, 27
 CYTOTOXICITY, 23

DCCD, 99
 DECREMENT, 83
 DEFINED*ABOVE, 9, 76
 DEFINED*ANTI, 9, 75
 DEFINED*AXIAL, 72
 DEFINED*BELOW, 9, 76
 DEFINED*CIS, 9, 74
 DEFINED*EQUATORIAL, 72
 DEFINED*R, 76
 DEFINED*S, 76
 DEFINED*SYN, 9, 75
 DEFINED*TRANS, 9, 74
 DEFUNCTIONALIZE, 77
 DELETE, 73
 DELTA, 23
 DEREK, 95
 DEREK Screening Set Operators, 22
 DEREK Title Operators, 11
 DESIGNATE, 69
 DEVELOPMENTAL, 23
 DIARYL*FUSION, 40
 DIASTEREOSELECTIVE*SYNTHESIS, 95
 DIAZO, 27
 Diborane, 98
 Dichlorocarbene, 99
 DIFF*ELECTROPHILIC*REACTIVITY, 109
 DIMENSIONED, 7
 directory sets, 7, 11, 16, 17, 21, 57
 DISCONNECTIVE, 17
 DISCONTINUE, 88

DISPLAY, 87
 Distance Specification, 50
 DISULFIDE, 28
 DITHIOACETAL, 28
 DITHIOKETAL, 28, 32
 DMSO/100, 99
 DO, 80
 DOES, 7
 DONATING, 31
 DONATING*REQUIRED, 48
 DOUBLE, 73
 DOUBLE STEREOSPECIFICALLY*CIS, 73
 DOUBLE STEREOSPECIFICALLY*TRANS, 73
 DOUBLE*BOND(S), 46
 DOUBLY, 48
 DOWN*TO, 89
 DUPLICATE, 7

E*DISUBSTITUTED, 22
 EIGHT, 104
 EIGHTEEN, 104
 ELECTRON*DENSITY, 109
 ELECTROPHILIC, 100
 ELECTROPHILIC*LOCALIZATION*ENERGY, 109
 ELEMENT, 7
 ELEVEN, 104
 EMPTY, 7
 ENAMINE, 27
 ENANTIOSELECTIVE*SYNTHESIS, 95
 END*DEFAULTS, 15
 END*PATTERN, 15
 END*REFERENCE(S), 10
 END*REMARK(S), 11
 END*RULE, 88
 END*TRANSFORM, 88
 ENDDO, 80
 ENOL*ETHER, 28
 Enolate, 96
 ENOLIZABLE, 47
 ENTER*BOND*PAIR, 79
 EPISULFIDE, 27
 EPISULPHIDE, 27
 EPOXIDE, 28
 EQUALING, 89
 EQUATORIAL, 38
 EQUIVALENCE, 105
 EQUIVALENT, 45
 ESTER, 27
 ESTERX, 28
 Et₂O:BF₃, 98
 ETHER, 28
 EXACTLY, 7
 EXCELLENT, 12
 EXCHANGE, 66
 EXIT, 81
 EXO, 7
 EXPANDABLE*WITHDRAWING, 31, 32
 Explicit Atom Types, 33
 Explicit Functional Groups, 27
 EXPLICIT*HYDROGEN, 50, 61

- EXTENDED*SUBGOAL, 17
- FACE, 7
- FAIR, 12
- FDA, 22
- FEWER, 44
- FG*TYPE, 109
- FGA*NOT*ALLOWED, 17
- fields, 8
- FIFTEEN, 104
- FINISHED, 88
- FIRST, 59
- FIVE, 104
- FLAG, 89
- FLUORIDE, 28
- FLUORINE, 34
- FOLLOWED, 95
- FOR, 7
- FOR*EACH*ATOM, 23
- FOR*EACH*PATH, 23
- FOR*EACH*RING, 23
- FORBIDDEN, 64
- FOUR, 104
- FOURTEEN, 104
- FRAGMENT*1, 60
- FRAGMENT*2, 60
- FRAGMENT*COUNT, 49
- FREE*VALENCY, 109
- FROM, 7
- FUNCTIONAL, 33
- Functional Group Specifiers, 27
- FUNCTIONAL*GROUP, 22
- FUSED, 39
- FUSED*TO*CURRENT*RING, 40, 41
- FUSION(S), 39
- GAMMA, 50
- GEM*DIHALIDE, 28
- Generic Atom Types, 35
- Generic Functional Groups, 29
- GETS, 41, 105
- GLYCOL, 28
- GO (TO), 80
- GOAL, 87
- GOAL*BOND, 48
- GOAL*FGA, 48
- GOAL*FGI, 48
- GOOD, 12
- GOOD*LEAVING, 32
- GROUP, 7, 33
- GROUP*1, 16
- GROUP*2, 16
- H2/Lindlar, 97
- H2/Pd, 97
- H2/Pt/HOAc, 97
- H2/Rh, 97
- H2Cr2O7, 98
- H2Ni, 97
- H2NOH, 97
- H2S, 98
- HALIDE, 30
- HALOAMINE, 27
- HALOGEN*ATOM(S), 35
- HALOHYDRIN, 28
- HAS, 7
- HAVE, 7
- HAZARD, 11
- HBr/Radical, 98
- HCN, 97
- HCN/ca*CN, 97
- HEADER OPERATORS, 10
- HEMIACETAL, 28
- HEPATOTOXICITY, 23
- HETERO, 35
- HETERO*ORIGIN, 54
- HETERO1*1, 52
- HETERO1*2, 52
- HETERO1*3, 52
- HETERO2*1, 53
- HETERO2*2, 53
- HETERO2*3, 53
- HETEROCYCLIC, 40, 41
- HETEROSELECTIVITY, 12, 14
- Hg[OAc]2, 98
- Hg[OOCFF3]2, 98
- HgCl2, 98
- HgO, 98
- HIGHLIGHT, 77
- HIO4, 99
- HOMO*COEFFICIENT, 109
- HOMO*ENERGY, 109
- HOMOSELECTIVITY, 12, 13
- HOOH/alk, 98
- HX/Radical, 98
- HYDRAZONE, 27
- HYDROGEN(S), 33
- HYDROGEN*ATOM, 34, 50
- HYDROXYL, 28
- HYDROXYLAMINE, 27
- I2, 99
- I2/Oxaziridine, 99
- IDENTICAL, 7
- Identifiers, 104
- IF, 7
- IMINE, 27, 72
- IN, 7
- IN*THE*GROUP, 61
- INCLUDING, 7
- INCREMENT, 83
- INGROUP*1, 59
- INGROUP*2, 59
- INTERFERING, 62
- INTERMOLECULAR, 17
- INTO, 7
- INTRODUCE, 68
- Introduction, 7
- INVERT, 72
- INVERTS*STEREO, 20

- IODIDE, 28
 IODINE, 34
 ION, 7, 27
 IRRITANCY, 23
 IS, 7
 ISOCYANATE, 27
 ISOCYANIDE, 27
 ISOPROPYL, 30
 IT, 53
- JOIN, 72
 JUMP (USING), 80
- K₂CO₃/CH₃I, 100
 K₃FeCN₆/pH7:10, 99
 KEEP, 54, 77
 KEPT*ORIGIN 1, 2, 3, 54
 KETONE, 27
 KILL, 64
 KMnO₄, 98
 KotBu, 96
 KR3BH/MINUS*50, 98
- LAH, 98
 LARGER, 40, 41
 LAST*RESORT, 18
 LDA, 96
 LDA/CH₃I, 100
 LEAVING, 32
 LEGAL*REQUEST, 18
 LENGTH, 7
 LESS*HINDERED, 44
 LHASA, 95
 LHASA Functional Group Screening Set Operators, 15
 LHASA Parameters for Calculations, 23
 LHASA Ring Parameters, 23
 LHASA Screening Operators for Olefin Chemistry, 21
 LHASA Screening Set Operators, 15
 LHASA Screening Set Operators for Transforms, 16
 LHASA Title Operators, 10
 Li[tBuO]₃AlH, 98
 LiAlH₄, 98
 LiClO₄, 98
 LiI, 97
 LiN[iPr]₂, 96
 LiNR₂, 96
 LIST, 7
 LOCANT, 53
 LOCANT SPECIFICATION, 50
 LOCANT(S), 53
 LOCKED*BOAT, 42
 LOCKED*CHAIR, 42
 Logical Combination of Qualifiers, 101
 logical operators, 107
 LONE1, 22
 LONE2, 22
 LONE3, 22
 LOOP, 89
 LOW*UTILITY, 18
 LOWER*DESTABILIZATION*ENERGY, 108
- LOWLIGHT, 77
 LUMO*COEFFICIENT, 109
 LUMO*ENERGY, 109
- MAKE, 66
 master index files, 7
 MAXIMUM*SIZE, 24
 MECHANISM, 7
 Mechanism and Subgoal Operations, 65
 MECHANISTIC, 18
 MEDIUM*UTILITY, 18
 MEMBER, 7
 MESSAGE, 84
 METHAEMOGLOBINAEMIA, 23
 METHYL(S), 35
 METHYL*ATOM(S), 35
 METHYLENE, 29
 Mg[II], 98
 MILDLY*ELECTROPHILIC, 100
 MINIMUM*SIZE, 24
 MINUS, 43
 MINUS:, 106
 Miscellaneous Language Constructs, 24
 Miscellaneous Operations, 86
 Miscellaneous Specifiers, 49
 MnO₂/CH₂Cl₂, 99
 MOLECULE, 7
 MORE, 44
 MORE*HINDERED, 44
 MULTIPLE, 46
 MULTIPLY, 46
 MUST, 7
 MUST*BE, 64
 MUST*NOT*BE, 64
 MUTAGENICITY, 23
- N*CARBAMATE, 28
 N*UREA*C, 28
 N*UREA*H, 28
 N₂CHCOOEt/Cu/80, 99
 Na*Naphthalide, 96
 Na/NH₃, 97
 Na[Hg], 97
 NaBH₄, 98
 NaBr, 97
 NaCl, 97
 NaCN, 97
 NaCNBH₃/HMPA/60, 98
 NaH, 96
 NaH/CH₃SOCH₃, 96
 NaH/DMSO, 96
 NaHSO₃, 98
 NaI, 97
 NaIO₄/pH5:8, 99
 NAME, 10, 12
 NaN₃, 97
 NaNH₂, 96
 NaOAc, 97
 NaOCl/pH:10, 99
 NaOR, 96

- NaSCN, 97
 NaSH, 97
 NBS/dioxane, 99
 NBS/HOH, 99
 NBS/Radical, 98
 nBu₃SnH, 99
 NEGATIVE*CHARGE, 48
 NEUROTOXICITY, 23
 NEUTRALIZE, 71
 NEW*HAZARD, 86
 NEW*RISK, 86
 NEW*TOXOPHORE, 86
 NEWEST*ATOM, 62
 NEWEST*BOND, 62
 NEXT, 7
 NH₂NH₂, 97
 NH₂OH, 97
 NH₃, 96
 Ni[CO]₄, 99
 Ni[CO]₄/Pd[0]L₄, 99
 NINE, 104
 NINETEEN, 104
 NITRILE, 27
 NITRO, 27
 NITROGEN(S), 34
 NITROSO, 27
 NO*C₂, 22
 NO*C₃, 22
 NO*FG*WITHIN*ALPHA, 22
 NO*L₂, 22
 NO*OP, 92
 NO*T₂, 22
 NON, 26
 NON*DIASTEREOSELECTIVE, 18
 NON*ENANTIOSELECTIVE, 18
 NON*OLEFINIC*FG, 22
 NON*OPPORTUNISTIC, 18
 NONE, 45
 NONEXPANDABLE*WITHDRAWING, 31, 32
 NOT, 26
 NOT*ABOVE, 94
 NOT*ANTI, 93
 NOT*APPLICABLE, 12
 NOT*BELOW, 94
 NOT*SYN, 93
 NR₃, 96
 NUCLEOPHILIC*LOCALIZATION*ENERGY, 109
 NUMBER, 108
 NUMBER*OF*RENUMBERS, 24
 NUMBER*OF*REORIENTS, 24
 NUMBER*OF*RINGS, 24
 Numerical Values, 104

 O*CARBAMATE, 28
 O*CARBONATE, 28
 O*SULFONATE, 28
 O*SULPHONATE, 28
 OBJECT, 16
 OF, 7
 OFF*CURRENT*RING, 41, 61
 OFF*SPECIFIED*PATH, 52, 61
 OFF*THE*BRIDGE, 43, 59
 OFF*THE*RING, 61
 OFFPATH, 58
 OFFPATH*ON*ATOM*1, 57
 OFFPATH*ON*ATOM*10, 57
 OFFPATH*ON*ATOM*11, 57
 OFFPATH*ON*ATOM*12, 57
 OFFPATH*ON*ATOM*2, 57
 OFFPATH*ON*ATOM*3, 57
 OFFPATH*ON*ATOM*4, 57
 OFFPATH*ON*ATOM*5, 57
 OFFPATH*ON*ATOM*6, 57
 OFFPATH*ON*ATOM*7, 57
 OFFPATH*ON*ATOM*8, 57
 OFFPATH*ON*ATOM*9, 57
 OFFPATH*ON*ATOM1, 17
 OFFRING, 59
 OFFSPRING, 7
 OLD, 7
 OLEFIN, 28, 50, 62, 85
 OLEFIN*TABLE, 22
 OLEFIN*TYPE, 7
 ON, 7
 ON*CURRENT*RING, 41, 61
 ON*SPECIFIED*PATH, 52, 61
 ON*THE*BRIDGE, 43, 59
 ON*THE*RING, 61
 ONE, 104
 ONLY, 7
 ONPATH, 58
 ONRING, 59
 OPERATION CODES, 64
 Operations Requiring Locant Specifiers, 78
 Operations Specifying Text, 84
 Operations Using an Address or Table Location, 80
 OR, 28, 31, 41, 106
 OR:, 45, 101, 102
 OR:IF, 45, 103
 ORDER, 7
 ORGANOCADMIUM, 97
 ORGANOCOPPER, 97
 ORGANOCUPRATE, 97
 ORGANOLITHIUM, 97
 ORGANOMAGNESIUM, 97
 ORGANOMETALLIC*AT*C1, 22
 ORGANOMETALLIC*AT*L1, 22
 ORGANOMETALLIC*AT*T1, 22
 ORGANOZINC, 97
 ORIENTATIONAL*SELECTIVITY, 12, 14
 ORIGIN*INDEPENDENT, 19
 OsO₄, 98
 Other Generic Functional Group Specifiers, 31
 OTHER*BONDS, 17, 20, 57
 OTHER*TOXICITY, 23
 Output files, 7
 OXIME, 27
 Oxosulf*ylide, 97
 Oxosulph*ylide, 97
 OXYGEN(S), 34

Ozone/MINUS*50, 98

P2O5, 99

PARENT, 7

Parentheses, 107

PARTICIPATING, 62

PATH, 7

PATH*INDEPENDENT, 19

PATH*ONE*SET, 19

PATRAN, 14

pattern chemistry, 14, 17, 18, 21, 77

Pattern Operators, 14

Pb[IV]/25, 99

Pb[IV]/80, 99

Peracid/0, 98

Peracid/50, 98

PERFORM, 87

PERMUTE, 79

PEROXIDE, 28

Peroxide/alk, 98

pH:1, 96

pH<1, 96

pH<1/100, 96

pH>12, 96

pH>12/150, 96

pH10:12, 96

pH2:4, 96

Ph3CNa, 96

Ph3P, 96

pH4:6, 96

pH6:8, 96

pH9:10, 96

PHENYL, 30

Phos*ylide, 97

PHOSPHINE, 27

PHOSPHONATE, 28

PHOSPHOROUS, 34

PHOSPHORUS, 34

PhSCl, 99

PhSeCl, 99

PLUS, 43

PLUS:, 106

POOR, 12

POSITION*1, 54

POSITION*2, 54

POSITION*3, 54

POSITION*4, 54

POSITION*5, 54

POSITION*6, 54

POSITION*7, 54

POSITIVE*CHARGE, 48

PRECURSOR, 7

PRESENT, 7

PRESERVED, 46

PREVIOUS, 7

PREVIOUS LOCANT, 53

PRIMARY, 9, 33

PRIMARY*CENTER(S), 46

PROTECT, 95

PROTECTED, 62

PROVISIONAL, 23

PUT, 105

PX3, 99

PX5, 99

Pyr, 96

Pyridine, 96

Pyrolysis/150, 99

Pyrolysis/250, 99

Pyrolysis/350, 99

QL2WRD, 8

QLADDR, 8, 9, 80, 82

QLBOND, 8, 50, 54, 78

QLBRKT, 8

QLCONST, 8

QLCONT, 8, 9

QLDIST, 8, 50, 51, 55, 57, 58, 59, 65

QLFORE0, 8

QLFROM, 8, 50, 51, 52, 55, 56, 58, 59

QLINCR, 8, 44, 49, 64, 65, 73

QLLOGC, 8, 101

QLNEXT, 9

QLNOT, 8

QLNOTL, 8, 50, 103

QLNOTT, 8, 26, 102

QLNUMB, 8, 9, 104

QLOPER, 8, 26, 64, 65

QLQUAL, 8, 50, 57, 58

QLRING_ISIZE, 8, 40

QLRING_JSIZE, 8, 40

QLRING_QUAL, 8, 40

QLTARG, 8, 26, 65, 95

QLTHEN, 8, 101

QUAT*CENTER, 22

QUATERNARY*CENTER(S), 46

Quinone, 99

R*CENTER(S), 37

R2BH, 98

R2Cd, 97

R2CuLi, 97

R3N, 96

R3OBF4, 99

R3P, 96

R3SnH, 99

RACEMIZE, 72

RACEMIZES*STEREO, 20

RADICAL, 43

RADICAL*CENTER, 48

RADICAL*LOCALIZATION*ENERGY, 109

RADICALIZE, 71

RATING, 12

rating block, 13

Rating Operators, 12

rating weightings, 12

RC*LARGER*THAN*METHYL, 22

RCO3H/0, 98

RCO3H/50, 98

RCu, 97

REARRANGEMENT, 19

- RECONNECTIVE, 19
 REDRAW, 87
 reference text file, 7
 REFERENCE(S), 10
 REGENERATE, 87
 REJECT, 84, 88
 REL*ELECTROPHILIC*REACTIVITY, 109
 REL*NUCLEOPHILIC*REACTIVITY, 109
 relational operators, 107
 RELIABILITY, 12, 13
 REMARK(S), 11
 REMOVE, 65
 REMOVE:, 105
 REMOVES*STEREO, 20
 RENUMBER, 88
 REORIENT, 88
 REORIENT*DEFINITION, 24
 REPRODUCTIVE*TOXICITY, 23
 REPUTATION, 12, 13
 REQUESTED, 63
 REQUIRED, 7
 REQUIRES*STEREO, 20
 RETAIN, 72
 RETAINS*STEREO, 20
 RETURN, 82
 RETURN FAIL, 82
 RETURN SUCCESS, 82
 RHO, 23
 Ring Atom and Bond Specifiers, 38
 RING(S), 40
 RING*4, 20
 RING*4*5, 20
 RING*4*6, 20
 RING*4*7, 20
 RING*5, 20
 RING*5*7, 20
 RING*6, 20
 RING*6*7, 20
 RING*7, 20
 RING*DEFINITION, 24
 RING*PARAMETERS, 23
 RING*PARAMETERS*END, 23
 RING*REQUIRED, 20
 RINGBOND, 22
 RISK, 11
 RLi, 97
 RMgX, 97
 RNH₂, 96
 RNHNH₂, 97
 RNHOH, 97
 ROH, 97
 RSH, 97
 RULE, 11
 RZnX, 97

 S*CENTER(S), 37
 SAME, 9, 44
 SAVE, 78
 SAVE*SEQUENTIALLY, 78
 SAVED*ATOM(S), 52
 SAVED*BOND(S), 55
 scale keywords, 12
 SCHEDULE, 77
 SCREEN, 12
 SECOND, 59
 SECONDARY, 9, 33
 SECONDARY*CENTER(S), 46
 Selectride, 98
 SELENIDE, 28
 SELENIUM, 34
 SENSITIZATION, 23
 SeO₂/pH₂:4, 99
 SeO₂/Pyr, 99
 SeO₂/Pyridine, 99
 SEPARATE, 72
 separators, 9
 Set Expressions, 105
 SET*ARRAYS:, 105
 sets, 104
 SETS:, 105
 SEVEN, 104
 SEVENTEEN, 104
 SHORTEST, 7
 SILICON, 34
 SILYLENOLETHYR, 28
 SIMULTANEOUSLY, 7
 SINGLE, 73
 SINGLE*BOND(S), 46
 Singlet*Oxygen, 99
 SIX, 104
 SIXTEEN, 104
 SIZE, 40
 SIZE(S), 40
 SMALL*FUSION, 39
 SnCl₂/HCl, 97
 SnCl₂/Pyr, 97
 SnCl₄/25, 98
 SOCl₂, 99
 Sodium*Hydride, 96
 SOME, 45
 SORT ASCENDING, 92
 SORT DESCENDING, 92
 SPACING*1*3, 22
 SPACING*1*4, 22
 SPACING*1*5, 22
 SPANNING, 55
 special operations, 64, 65
 SPECIAL*SET, 86
 SPECIFIED*ATOM(S), 52
 SPECIFIED*BOND(S), 56
 SPIRO, 39
 square brackets, 105
 STANDARD, 22
 STANNUM, 34
 STEREOCENTER(S), 36
 Stereochemical Atom and Bond Specifiers, 36
 STEREOSELECTIVE, 20
 STEREOSPECIFIC, 20
 storage of CHMTRN, 7
 STRAINED, 40

STRONGLY*ELECTROPHILIC, 100
 STRUCTURAL, 7
 STUDENT, 21
 student version of LHASA, 95
 SUBGOAL(S) *ALLOWED, 21
 SUBGOAL*1, 16
 SUBGOAL*2, 16
 SUBJECT, 16
 SUBROUTINE, 52, 56, 82
 subscript, 105, 107
 SUBTRACT, 64
 SUCCESSFUL, 89
 Sulf*ylide, 97
 SULFIDE, 27
 SULFONE, 27
 SULFOXIDE, 27
 SULFUR, 34
 Sulph*ylide, 97
 SULPHIDE, 27
 SULPHONE, 27
 SULPHOXIDE, 27
 SULPHUR, 34
 SYMMETRICAL, 21
 SYN, 9, 93

T*BUTYL, 30
 T1*NOT*QUARTERNARY, 22
 TACTIC*SCREEN, 12
 tactical combinations, 12, 14, 17, 18, 19, 20
 TARGET, 7
 TARGET SPECIFICATION, 26
 tButoxide, 96
 TBUTYL, 30
 TEN, 104
 TERT*OR*QUAT*CENTER, 22
 TERTIARY, 9, 33
 TERTIARY*CENTER(S), 46
 TEST, 7
 TETRASUBSTITUTED, 22
 THAT, 7
 THE, 7
 THEM, 53
 THEN, 101
 THEN*TO, 83
 THERE, 7
 THERMODYNAMICS, 12, 14
 THIOCYANATE, 27
 THIOESTER, 27
 THIOL, 27
 THIRTEEN, 104
 THIS, 59
 THREE, 104
 THROUGH, 25, 40, 41
 TIME, 92
 TIN, 34
 TITANIUM, 34
 TITLE, 85
 Title Operators, 10
 Title Operators for Screens, 11
 TI[III], 100

TI[NO3]3, 100
 TO, 7
 TOGGLE BIT, 86
 TOGGLE FLAG, 89
 TOWARDS, 45
 TOXOPHORE, 11
 TRANS, 9, 36
 TRANS*BRIDGE(D), 42
 TRANS*FUSION, 37
 TRANS*OLEFIN, 37
 TRANS1, 22
 TRANS2, 22
 TRANS3, 22
 TRANSFORM, 10
 TRIALKYLSILOXY, 28
 TRIALKYLSILYL, 28
 TRIHALIDE, 28, 32
 TRIPLE, 73
 TRIPLE*BOND(S), 46
 TRIPLY, 48
 TRISUBSTITUED, 22
 TsNHNH2, 96
 TsOH/0, 98
 TsOH/80, 98
 TURN OFF BIT, 86
 TURN OFF FLAG, 89
 TURN ON BIT, 86
 TURN ON FLAG, 89
 TWELVE, 104
 TWENTY, 104
 TWO, 104
 TYPICAL*YIELD, 12, 13

UNDEFINED*C*FIRST, 22
 UNDEFINED*C*LAST, 22
 UNDEFINED*L*FIRST, 22
 UNDEFINED*L*LAST, 22
 UNDEFINED*T*FIRST, 22
 UNDEFINED*T*LAST, 22
 UNITS, 7
 UNMASKING, 21
 UNSUCCESSFUL, 89
 UP, 7
 UP*TO, 89

VALUE, 7
 Variable Expressions, 107
 VARIABLE(S):, 105
 VARIABLE*ARRAY(S):, 105
 variables, 104
 VIC*DIHALIDE, 28
 VINYL, 28
 VINYLSILANE, 28
 VINYLW, 28

WARNING, 85
 WELL*DEFINED*C*FIRST, 22
 WELL*DEFINED*C*LAST, 22
 WELL*DEFINED*L*FIRST, 22
 WELL*DEFINED*L*LAST, 22

WELL*DEFINED*T*FIRST, 22
WELL*DEFINED*T*LAST, 22
WHICH, 7
WITH, 7
WITHDRAWING, 31
WITHDRAWING*REQUIRED, 48
WITHIN, 51
WITTIG, 97
WORD, 7
WRITE, 7
WRITE*STRUCTURE, 92

XCN, 99

Z*DISUBSTITUTED, 22
ZERO, 105
ZERO SET*ARRAY, 91
ZERO VARIABLE*ARRAY, 92
Zn/HCl, 97
Zn/HOAc, 97
Zn[BH₄], 98